

File: sequence_control.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/teste_clp

"""

sequence_control.py

Módulo plug-and-play para executar uma sequência de posições (X,Y,Z)
num sistema CNC + câmera.

Dependências:

PyQt6>=6.5

(Opcional) o dataclass InspectionPosition já usado nos módulos anteriores.

Caso não exista no projeto hospedeiro basta
substituir por dicts {'x':..., 'y':..., 'z':...}.

"""

from __future__ import annotations

import time # ← NECESSÁRIO para wait_for_idle / trigger_dot

from typing import List, Protocol, Callable, Dict, Any

from dataclasses import dataclass

from PyQt6.QtCore import QThread, pyqtSignal, QObject, Qt

from PyQt6.QtWidgets import (

QWidget, QGroupBox, QVBoxLayout, QHBoxLayout, QPushButton,

QProgressBar, QLabel, QFileDialog, QMessageBox

)

1) Modelo de dados (reaproveitável)

@dataclass

class InspectionPosition:

name: str

x: float

y2: float

y1: float

z: float = 0.0

camera_params: Dict[str, Any] | None = None

2) Protocolos (interfaces) para back-ends

class MotionBackend(Protocol):

"""Funções mínimas exigidas do sistema de movimento (GRBL, PLC etc.)."""

def move_to_absolute_position(self,

x: float | None,

y2: float | None,

y1: float | None,

z: float | None,

feed_rate: float = 1000) -> bool: ...

```

def wait_for_idle(self) -> bool: ...
# opcional, mas ajuda:
def set_feed_rate(self, feed_rate: float): ...
# novo: habilita ou não o uso do offset câmera↔nozzle
def set_offset_mode(self, apply_offset: bool): ...
# opcional – grava SOMENTE a quantidade de dots
def apply_dot_qty(self, qty: int): ...

```

```

class CameraBackend(Protocol):
    """Funções mínimas exigidas da câmera (pode ser dummy)."""
    def capture(self, params: Dict[str, Any] | None = None): ...

```

```

# -----
# 3) Thread que percorre a sequência
# -----

```

```

class SequenceRunnerThread(QThread):
    progress      = pyqtSignal(int, int) # idx_atual, total
    imageCaptured = pyqtSignal(object)  # imagem capturada
    finished      = pyqtSignal()
    error         = pyqtSignal(str)

    def __init__(self,
        motion: MotionBackend,
        camera: CameraBackend | None,
        positions: List[InspectionPosition],
        *,
        apply_mode: bool,          # True = APPLY / False = VIEW
        feed_rate: float = 1000.0):
        super().__init__()
        self._motion = motion
        self._camera = camera
        self._positions = positions
        self._feed_rate = feed_rate
        self._apply_mode = apply_mode # guarda selecção do usuário
        self._stop_flag = False

    def request_stop(self):
        self._stop_flag = True

    def run(self):
        try:
            total = len(self._positions)
            if total == 0:
                self.error.emit("Sequência vazia")
                return
            # feed opcional
            if hasattr(self._motion, "set_feed_rate"):
                self._motion.set_feed_rate(self._feed_rate)

```

```

for idx, pos in enumerate(self._positions, 1):
    if self._stop_flag:
        self.error.emit("Execução interrompida pelo usuário")
        return

    # -----
    # DECIDE SE OFFSET É APLICADO NESTE PONTO
    # • VIEW → nunca aplica
    # • APPLY → depende do tipo da ação
    # -----
    if not self._apply_mode:      # modo VIEW
        apply_off = False
    else:                         # modo APPLY
        a = (pos.camera_params or {}).get("action")
        apply_off = a not in ("barcode", "fiducial", "inspect")
    if hasattr(self._motion, "set_offset_mode"):
        try:
            self._motion.set_offset_mode(apply_off)
        except Exception:
            print("[seq] Falha ao definir modo de offset")
            pass

    ok = self._motion.move_to_absolute_position(
        pos.x, pos.y2, pos.y1, pos.z, feed_rate=self._feed_rate)
    if not ok:
        self.error.emit(f"Falha ao enviar movimento para {pos.name}")
        return
    if not self._motion.wait_for_idle():
        self.error.emit("Timeout de movimento")
        return

    # ----- dwell 100 ms entre pontos -----
    # evita que o próximo comando seja enfileirado antes
    # do CLP realmente liberar todos os eixos.
    self.msleep(100)      # (≈ 0,1 s)
    # -----
    # AÇÃO "dot" → escreve freq/qty antes do pulso
    # -----
    if (pos.camera_params or {}).get("action") == "dot":
        # aplica configurações de dot (freq e qty)
        qty = int((pos.camera_params or {}).get("dot_qty", 1))
        if hasattr(self._motion, "apply_dot_qty"):
            try:
                self._motion.apply_dot_qty(qty)
            except Exception:
                print("[seq] Falha ao aplicar dot settings")
                pass

    # 1. envia PULSO em M5000
    if hasattr(self._motion, "trigger_dot"):
        ok = self._motion.trigger_dot()

```

```

        if not ok:
            self.error.emit("Falha ao acionar dot – M5000")
            return

        # 2. espera conclusão via M5001
        if hasattr(self._motion, "wait_dot_complete"):
            ok = self._motion.wait_dot_complete(timeout=10.0)
            if not ok:
                self.error.emit("Timeout aguardando fim do dot (M5001)")
                return
        if self._camera is not None:
            try:
                img = self._camera.capture(pos.camera_params or None)
                self.imageCaptured.emit(img)
            except Exception as exc:
                print(f"[seq] Erro ao capturar imagem: {exc}")
                self.error.emit(f"Erro na captura: {exc}")
                return

        self.progress.emit(idx, total)

        self.finished.emit()

    except Exception as exc:
        print(f"[seq] Erro na execução da sequência: {exc}")
        self.error.emit(str(exc))

```

```

# -----
# 4) Widget de controle da sequência
# -----

```

```

class SequenceControlWidget(QWidget):

```

```

    """

```

Widget genérico com:

- Botão Executar
- Botão Parar
- Barra de progresso
- Status

Sinais principais do runner são replicados para facilitar
ligação com outros componentes da GUI.

```

    """

```

```

imageCaptured = pyqtSignal(object)
sequenceFinished = pyqtSignal()
sequenceError = pyqtSignal(str)

```

```

def __init__(self,
              motion: MotionBackend,
              camera: CameraBackend | None = None,
              parent=None):
    super().__init__(parent)

```

```

self._motion = motion
self._camera = camera
self._positions: List[InspectionPosition] = []
self._runner: SequenceRunnerThread | None = None
self._build_ui()

# ----- API pública -----
def set_positions(self, positions: List[InspectionPosition]):
    """Define/atualiza a lista de posições da sequência."""
    self._positions = positions
    self._status.setText(f'{len(positions)} posições prontas')

def load_positions_from_callable(self, loader: Callable[[], List[InspectionPosition]]):
    """Permite ligar um botão 'Carregar...' externo a esta função."""
    try:
        self.set_positions(loader())
    except Exception as exc:
        print(f'[seq] Falha ao carregar posições: {exc}')
        QMessageBox.critical(self, "Erro", f'Falha ao carregar posições:\n{exc}')

# ----- UI -----
def _build_ui(self):
    grp = QGroupBox("Controle de Sequência")
    v = QVBoxLayout(grp)

    self._btn_execute = QPushButton("Executar Sequência")
    self._btn_stop = QPushButton("Parar")
    self._btn_stop.setEnabled(False)

    self._progress = QProgressBar()
    self._progress.setValue(0)
    self._progress.setAlignment(Qt.AlignmentFlag.AlignCenter)

    self._status = QLabel("Sem sequência")

    v.addWidget(self._btn_execute)
    v.addWidget(self._btn_stop)

# ----- NOVO SELETOR VIEW / APPLY -----
hmode = QHBoxLayout()
from PyQt6.QtWidgets import QRadioButton, QLabel as _QLabel
self.radio_view = QRadioButton("View")
self.radio_apply = QRadioButton("Apply")
self.radio_apply.setChecked(True) # default = aplicar offset
hmode.addWidget(_QLabel("Offset:"))
hmode.addWidget(self.radio_view)
hmode.addWidget(self.radio_apply)
hmode.addStretch()
v.addLayout(hmode)

v.addWidget(self._progress)

```

```

v.addWidget(self._status)

main = QVBoxLayout(self)
main.addWidget(grp)
main.addStretch()

# ligações
self._btn_execute.clicked.connect(self._start)
self._btn_stop.clicked.connect(self._stop)

# troca de modo view/apply
self.radio_view.toggled.connect(
    lambda checked: self._on_mode_changed(checked))

# ----- modo offset -----
def _on_mode_changed(self, view_checked: bool):
    """
    view_checked = True → modo VIEW (não aplica offset)
    False → radio_apply ativo (aplica offset)
    """
    apply_off = not view_checked
    if hasattr(self._motion, "set_offset_mode"):
        try:
            self._motion.set_offset_mode(apply_off)
        except Exception:
            print("[seq] Falha ao definir modo de offset")
            pass
    self._status.setText("Modo: VIEW (nozzle)" if view_checked
                        else "Modo: APPLY (camera)")

# ----- handlers -----
def _start(self):
    # ----- verifica posições -----
    if not self._positions:
        QMessageBox.warning(self, "Aviso", "Nenhuma posição definida")
        return
    # ----- encerra runner anterior -----
    if self._runner is not None:
        if self._runner.isRunning():
            QMessageBox.warning(self, "Execução",
                                "A sequência ainda está em andamento.")
            return
    # desconecta e destrói completamente
    try:
        self._runner.finished.disconnect()
        self._runner.error.disconnect()
    except Exception:
        pass
    self._runner.deleteLater()

```

```

self._runner = None

self._progress.setRange(0, len(self._positions))
self._progress.setValue(0)
self._status.setText("Executando...")
self._btn_execute.setEnabled(False)
self._btn_stop.setEnabled(True)

self._runner = SequenceRunnerThread(
    self._motion,
    self._camera,
    self._positions,
    apply_mode=self.radio_apply.isChecked() # passa modo global
)
self._runner.progress.connect(self._on_progress)
self._runner.imageCaptured.connect(self.imageCaptured)
self._runner.finished.connect(self._on_finished)
self._runner.error.connect(self._on_error)
self._runner.start()

def _stop(self):
    if self._runner and self._runner.isRunning():
        self._runner.request_stop()

def _on_progress(self, idx: int, total: int):
    self._progress.setValue(idx)
    self._status.setText(f'{idx}/{total} concluídos')

def _on_finished(self):
    self._status.setText("Concluída")
    self._btn_execute.setEnabled(True)
    self._btn_stop.setEnabled(False)
    self.sequenceFinished.emit()
    # ----- encerra thread e limpa ponteiro -----
    if self._runner:
        self._runner.wait(1000)
        self._runner.deleteLater()
        self._runner = None

def _on_error(self, msg: str):
    self._status.setText(f'Erro: {msg}')
    self._btn_execute.setEnabled(True)
    self._btn_stop.setEnabled(False)
    self.sequenceError.emit(msg)
    if self._runner:
        self._runner.wait(1000)
        self._runner.deleteLater()
        self._runner = None

```

```

# 5) Demonstração stand-alone
# -----
if __name__ == "__main__":
    import sys
    from random import random
    from PyQt6.QtWidgets import QApplication

    # --- Back-ends fictícios -----
    class _DummyMotion:
        def move_to_absolute_position(self, x, y2, y1, z, feed_rate=1000):
            print(f"[motion] X={x:.2f} Y2={y2:.2f} Y1={y1:.2f} Z={z:.2f} F{feed_rate}")
            return True
        def wait_for_idle(self): pass
        def set_feed_rate(self, fr): print("[motion] Feed =", fr)

    class _DummyCamera:
        def capture(self, params=None):
            import numpy as np
            # gera imagem RGB aleatória 320x240
            return (np.random.rand(240, 320, 3) * 255).astype("uint8")

    # --- dados de exemplo -----
    demo_positions = [
        InspectionPosition(f"P{i+1}", random()*100, random()*80, 0.0)
        for i in range(5)
    ]

    # --- aplica ao widget -----
    app = QApplication(sys.argv)
    widget = SequenceControlWidget(_DummyMotion(), _DummyCamera())
    widget.set_positions(demo_positions)
    widget.resize(280, 200)
    widget.show()

    widget.imageCaptured.connect(lambda img: print("[seq] imagem capturada", type(img)))

    sys.exit(app.exec())

```

File: table_program_tab.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/teste_clp

```

# table_program_tab.py

```

```

"""

```

Aba de programação de uma mesa (Mesa 1 ou Mesa 2).

Mostra controles para X, Y (lógico) e Z apenas.

```

"""

```

```

from PyQt6.QtWidgets import QWidget, QGridLayout, QVBoxLayout, QFrame, QSizePolicy, QLabel, QSpinBox
from PyQt6.QtCore import Qt, QTimer

```

```

from PyQt6.QtGui import QPixmap
import base64, cv2
import numpy as np
from PIL import Image
import time

from inspection_config_widget import InspectionConfigWidget
from movement_controls_widget import MovementControlsWidget
from position_status_widget import PositionStatusWidget
from inspection_positions_widget import InspectionPositionsWidget
from program_io_widget import ProgramIOWidget
from table_positions_backend import TablePositionsBackend
from sequence_control import SequenceControlWidget, InspectionPosition
from dot_patterns_widget import DotPatternsWidget
from dot_action_selector_widget import DotActionSelectorWidget
from action_config_placeholders import (
    BarcodeConfigWidget, FiducialConfigWidget, InspectionConfigWidget
)
from PyQt6.QtWidgets import QStackedWidget
from PyQt6.QtCore import pyqtSignal

# -----
# QLabel que emite sinal ao clicar – para mover a cabeça
# -----
class ClickableLabel(QLabel):
    # usa float porque ev.position().x()/y() devolvem qreal (float)
    clicked = pyqtSignal(float, float) # x,y em pixels dentro do label
    def mousePressEvent(self, ev):
        if ev.button() == Qt.MouseButton.LeftButton:
            self.clicked.emit(ev.position().x(), ev.position().y())
            super().mousePressEvent(ev)

class TableProgramTab(QWidget):
    """
    Mesa 1 → usa eixo físico Y1
    Mesa 2 → usa eixo físico Y2
    """
    def __init__(self, controller, mesa_id: int):
        super().__init__()
        assert mesa_id in (1, 2)
        self.ctrl = controller
        self.mesa = mesa_id # 1 ou 2
        self.y_axis = 'Y1' if mesa_id == 1 else 'Y2'
        # orientação do Y: +1 → clicar acima do centro move Y+
        # -1 → clicar abaixo move Y-
        # sua câmera está “invertida”, portanto usamos +1
        self._PIXEL_TO_Y_SIGN = +1
        self._build_ui()
        self._jog_active_axis: str | None = None
        QTimer.singleShot(0, self._fix_video_size)

```

```
self.ctrl.camera_manager.frameReady.connect(self._update_frame)
```

```
# -----
```

```
# TAMANHO FIXO DO VÍDEO
```

```
# -----
```

```
def _fix_video_size(self):
```

```
    """Define largura/altura fixas para o lbl_video em ≈70 % da
    janela e muda o SizePolicy para Fixed, impedindo que o layout
    redimensione esse label no futuro."""
```

```
    win = self.window()
```

```
    if not win:
```

```
        return
```

```
    w = int(win.width() * 1.0)
```

```
    h = int(win.height() * 1.0 * 1.0 )
```

```
    # mantém razão 4:3 aproximada
```

```
    if h * 4 < w * 3:
```

```
        w = int(h * 4 / 3)
```

```
    else:
```

```
        h = int(w * 3 / 4)
```

```
    self.lbl_video.setSizePolicy(QSizePolicy.Policy.Fixed,
                                QSizePolicy.Policy.Fixed)
```

```
    self.lbl_video.setFixedSize(w, h)
```

```
# ----- ATUALIZAÇÃO DO QUADRO -----
```

```
def _update_frame(self, img):
```

```
    # escala mantendo proporção
```

```
    # guarda tamanho original para coordenadas
```

```
    self._last_frame_sz = (img.width(), img.height())
```

```
    pix = QPixmap.fromImage(img).scaled(
        self.lbl_video.size(), Qt.AspectRatioMode.KeepAspectRatio,
        Qt.TransformationMode.SmoothTransformation)
```

```
# ---- desenha cruz -----
```

```
p = QPixmap(pix.size())
```

```
p.fill(Qt.GlobalColor.transparent)
```

```
from PyQt6.QtGui import QPainter, QPen
```

```
pa = QPainter(p)
```

```
pen = QPen(Qt.GlobalColor.red, 4, Qt.PenStyle.SolidLine)
```

```
pa.setPen(pen)
```

```
cx = p.width()//2; cy = p.height()//2
```

```
pa.drawLine(cx-100, cy, cx+100, cy)
```

```
pa.drawLine(cx, cy-100, cx, cy+100)
```

```
# ----- overlays se ação = fiducial -----
```

```
if self.action_selector.current_action() == 'fiducial':
```

```
    win = self._fid_window
```

```
    rad = self._fid_radius
```

```
    sx = pix.width() / self._last_frame_sz[0]
```

```
    sy = pix.height() / self._last_frame_sz[1]
```

```
    # ROI (amarelo cheio)
```

```
    pen_y = QPen(Qt.GlobalColor.yellow, 2, Qt.PenStyle.SolidLine)
```

```

pa.setPen(pen_y)
rw = int(win * sx); rh = int(win * sy)
pa.drawRect(cx - rw//2, cy - rh//2, rw, rh)
# ÁREA DE BUSCA (amarelo tracejado)
pen_d = QPen(Qt.GlobalColor.yellow, 1, Qt.PenStyle.DashLine)
pa.setPen(pen_d)
r2 = win + 2*rad
rw2 = int(r2 * sx); rh2 = int(r2 * sy)
pa.drawRect(cx - rw2//2, cy - rh2//2, rw2, rh2)
# Resultado do último teste (2 s)
if self._match_info:
    rect, color, t0 = self._match_info
    if time.time() - t0 < 2.0:
        pen_m = QPen(color, 2)
        pa.setPen(pen_m)
        x, y, w, h = rect
        pa.drawRect(int(x*sx), int(y*sy), int(w*sx), int(h*sy))
    else:
        self._match_info = None
elif self.action_selector.current_action() == 'barcode':
    bw, bh = self._bc_w, self._bc_h
    sx = pix.width() / self._last_frame_sz[0]
    sy = pix.height() / self._last_frame_sz[1]
    pen_b = QPen(Qt.GlobalColor.yellow, 2)
    pa.setPen(pen_b)
    # retângulo da ROI (amarelo) – converte para int
    rx = int(cx - bw*sx/2)
    ry = int(cy - bh*sy/2)
    rw = int(bw * sx)
    rh = int(bh * sy)
    pa.drawRect(rx, ry, rw, rh)
    if self._bc_match and time.time()-self._bc_match[2] < 2.0:
        rect, color, t0 = self._bc_match
        pen_m = QPen(color, 2)
        pa.setPen(pen_m)
        x,y,w,h = rect
        pa.drawRect(int(x*sx), int(y*sy), int(w*sx), int(h*sy))
    elif self._bc_match:
        self._bc_match = None
elif self.action_selector.current_action() == 'inspect':
    iw, ih = self._insp_w, self._insp_h
    sx = pix.width()/self._last_frame_sz[0]
    sy = pix.height()/self._last_frame_sz[1]
    pen_i = QPen(Qt.GlobalColor.yellow, 2)
    pa.setPen(pen_i)
    rx = int(cx - iw*sx/2)
    ry = int(cy - ih*sy/2)
    rw = int(iw*sx); rh = int(ih*sy)
    pa.drawRect(rx, ry, rw, rh)
    if self._insp_match and time.time()-self._insp_match[2]<2.0:
        rect,color,t0 = self._insp_match

```

```

        pen_m = QPen(color,2); pa.setPen(pen_m)
        x,y,w,h = rect
        pa.drawRect(int(x*sx), int(y*sy), int(w*sx), int(h*sy))
    elif self._insp_match:
        self._insp_match=None
    pa.end()
    qp = QPixmap(pix.size())
    qp.fill(Qt.GlobalColor.transparent)
    painter = QPainter(qp); painter.drawPixmap(0,0,pix); painter.drawPixmap(0,0,p); painter.end()
    self.lbl_video.setPixmap(qp)

# ----- overlay helpers -----
def _on_fid_params(self, win:int, rad:int):
    self._fid_window = win
    self._fid_radius = rad
def _on_fid_match(self, sim:float, ok:bool, x:int, y:int, w:int, h:int):
    from PyQt6.QtGui import QColor
    self._match_info = ((x, y, w, h),
                        QColor(Qt.GlobalColor.green if ok else Qt.GlobalColor.red),
                        time.time())

# ----- barcode helpers -----
def _set_barcode_roi(self, w:int, h:int):
    self._bc_w, self._bc_h = w, h
def _on_bc_test(self, ok:bool, x:int, y:int, w:int, h:int):
    from PyQt6.QtGui import QColor
    self._bc_match = ((x,y,w,h),
                    QColor(Qt.GlobalColor.green if ok else Qt.GlobalColor.red),
                    time.time())

# ----- inspeção helpers -----
def _set_insp_roi(self,w:int,h:int):
    self._insp_w=w; self._insp_h=h
def _on_insp_region(self,img):
    # último retângulo verde (sempre OK por definição)
    from PyQt6.QtGui import QColor
    self._insp_match=((0,0,self._insp_w,self._insp_h),
                    QColor(Qt.GlobalColor.green),
                    time.time())

def _build_ui(self):
    grid = QGridLayout(self)
    grid.setContentsMargins(0, 0, 0, 0)

    # proporções
    grid.setColumnStretch(0, 15)
    grid.setColumnStretch(1, 70)
    grid.setColumnStretch(2, 15)
    grid.setRowStretch(0, 7)
    grid.setRowStretch(1, 3)

```

```

# ----- COLUNA ESQUERDA (posições) -----
v_left = QVBoxLayout()

aux = self.ctrl.create_auxiliary_controls()
v_left.addWidget(aux)
v_left.addStretch()

# ----- SELETOR DE AÇÃO -----
self.action_selector = DotActionSelectorWidget()
v_left.addWidget(self.action_selector)
# ----- STACK DE CONFIGURAÇÕES -----
self._config_stack = QStackedWidget()
# página 0 – DOTS
self.dots_view = DotPatternsWidget()
self._config_stack.addWidget(self.dots_view)
# página 1 – BARCODE (widget real)
self.barcode_cfg = BarcodeConfigWidget(self.ctrl)
self._config_stack.addWidget(self.barcode_cfg)
# página 2 – INSPEÇÃO (widget real)
self.inspect_cfg = InspectionConfigWidget(self.ctrl)
self._config_stack.addWidget(self.inspect_cfg)
# página 3 – FIDUCIAL (widget real – precisa do controller)
self.fiducial_cfg = FiducialConfigWidget(self.ctrl)
self._config_stack.addWidget(self.fiducial_cfg) # index 3

# ----- overlay barcode -----
self._bc_w = 400; self._bc_h = 150
self._bc_match = None # (rect,color,t0)
self.barcode_cfg.parametersChanged.connect(
    lambda w,h: self._set_barcode_roi(w,h))
self.barcode_cfg.testFinished.connect(self._on_bc_test)

# ----- overlay fiducial -----
self._fid_window = 50
self._fid_radius = 80
self._match_info = None # (rect, color, t0)
self.fiducial_cfg.parametersChanged.connect(self._on_fid_params)
self.fiducial_cfg.matchTested.connect(self._on_fid_match)
# ----- overlay inspeção -----
self._insp_w=400; self._insp_h=300
self._insp_match=None
self.inspect_cfg.parametersChanged.connect(
    lambda w,h: self._set_insp_roi(w,h))
self.inspect_cfg.regionCaptured.connect(self._on_insp_region)

v_left.addWidget(self._config_stack, 1) # ocupa o restante

# conecta mudança de ação
self.action_selector.actionChanged.connect(self._show_action_config)
# seleção default: Dot
self.action_selector._group.buttons()[1].setChecked(True) # 'dot'

```

```

self._show_action_config('dot')

w_left = QWidget(); w_left.setLayout(v_left)
grid.addWidget(w_left, 0, 0)

# ----- COLUNA CENTRAL (preview câmera) -----

self.lbl_video = ClickableLabel("Sem vídeo")
self.lbl_video.setAlignment(Qt.AlignmentFlag.AlignCenter)
self.lbl_video.setStyleSheet(
    "QLabel { background:#000; color:#FFF; border:1px dashed #607D8B; }")
# não deixa o label pedir altura maior que a célula
self.lbl_video.setSizePolicy(
    QSizePolicy.Policy.Expanding,    # horizontal livre
    QSizePolicy.Policy.Preferred)    # vertical controlado
# mantém o label FIXO (já dimensionado por _fix_video_size) e
# o coloca sempre no centro da célula (h & v).
grid.addWidget(self.lbl_video, 0, 1,
               alignment=Qt.AlignmentFlag.AlignCenter)
# clique → mover cabeça
self.lbl_video.clicked.connect(self._on_video_click)

# ----- COLUNA DIREITA (controles) -----
v_right = QVBoxLayout()

self.mov_widget = MovementControlsWidget() # padrão completo
v_right.addWidget(self.mov_widget)

# limites provenientes do controlador
self._limits = self.ctrl.table_limits[self.mesa]

# Posições de inspeção (X,Y,Z apenas, com validação)
self.inspect_widget = InspectionPositionsWidget(
    get_current_position=self._get_current_position,
    validate_position=self._validate_position,
    get_action_context=self._current_action_context
)

# ■■■■■ NOVO: duplo-clique leva a cabeça até a posição ■■■■■
self.inspect_widget.list_widget.itemDoubleClicked.connect(
    self._on_position_double_clicked)

v_right.addWidget(self.inspect_widget)

# --- backend que salva 3 eixos + extensão dedicada -----
backend = TablePositionsBackend(self.inspect_widget, self.y_axis)
file_filter = f"Programa Mesa {self.mesa} (*.{self.mesa})"
self.prog_widget = ProgramIOWidget(
    backend,
    file_filter=file_filter,
    default_suffix=f".{self.mesa}",

```

```

        default_dir=self.ctrl.projects_dir
    )
    v_right.addWidget(self.prog_widget)
    # remove botões Salvar/Carregar da interface desta aba
    self.prog_widget.hide()

    self.seq_widget = SequenceControlWidget(
        motion=self.ctrl._plc_motion_backend, camera=None)
    v_right.addWidget(self.seq_widget)

    v_right.addStretch()
    w_right = QWidget(); w_right.setLayout(v_right)
    grid.addWidget(w_right, 0, 2)

    # ----- LINHA INFERIOR -----
    left_sp = QWidget(); right_sp = QWidget()
    for s in (left_sp, right_sp):
        s.setSizePolicy(QSizePolicy.Policy.Expanding,
            QSizePolicy.Policy.Preferred)

    bottom = QFrame()
    bottom.setStyleSheet(
        "QFrame { background:#ECEFF1; border:1px dashed #B0BEC5;}")
    # garante que a linha 1 tenha "peso" para competir com o vídeo
    bottom.setSizePolicy(QSizePolicy.Policy.Expanding,
        QSizePolicy.Policy.Expanding)

    grid.addWidget(left_sp, 1, 0)
    grid.addWidget(bottom, 1, 1)
    grid.addWidget(right_sp, 1, 2)

    # ----- SINAIS -----
    self.mov_widget.stepMoveRequested.connect(self._on_step_move)
    self.mov_widget.jogStart.connect(self._on_jog_start)
    self.mov_widget.jogStop.connect(self._on_jog_stop)
    self.mov_widget.goToZeroRequested.connect(self._go_center)

    self.inspect_widget.positionAdded.connect(
        lambda _: self.seq_widget.set_positions(self._to_model()))
    # NOVO: criação imediata das pastas / imagens
    self.inspect_widget.positionAdded.connect(self._on_position_added)
    self.inspect_widget.positionRemoved.connect(
        lambda _: self.seq_widget.set_positions(self._to_model()))

    self.prog_widget.fileLoaded.connect(
        lambda _: self.seq_widget.set_positions(self._to_model()))

    # logs
    self.prog_widget.fileLoaded.connect(
        lambda f: self.ctrl.log(f"[Mesa {self.mesa}] Programa carregado: {f}"))
    self.prog_widget.fileSaved.connect(

```

```

lambda f: self.ctrl.log(f"[Mesa {self.mesa}] Programa salvo: {f}")

# ----- GRÁFICO -----
# sempre que houver mudança nos pontos ou carregamento de um
# programa, envia nova lista ao MesaPlotWidget da mesa.
for sig in (self.inspect_widget.positionAdded,
            self.inspect_widget.positionRemoved,
            self.prog_widget.fileLoaded):
    sig.connect(lambda _=None: self._update_plot())

QTimer.singleShot(0, self._update_plot) # primeira vez

# ===== PERSONALIZAÇÃO UI =====
self._adapt_widgets_for_single_y()

# =====
# DUAS LINHAS → duplo-clique na lista "Posições de Inspeção"
# =====
def _on_position_double_clicked(self, item):
    """
    Move a cabeça para as coordenadas salvas no ponto
    que o usuário clicou duas vezes na Tree/List.
    """
    row = self.inspect_widget.list_widget.row(item)
    try:
        pos = self.inspect_widget.positions()[row]
    except IndexError:
        return

    # extrai destino absoluto
    tgt_x = int(pos.x)
    tgt_y = int(pos.y1 if self.y_axis == "Y1" else pos.y2)
    tgt_z = int(pos.z)
    y_axis = self.y_axis

    # escreve spinBoxes "fantasma" para que o backend PLC use
    self.ctrl.pulsos_spin_X.setValue(tgt_x)
    getattr(self.ctrl, f"pulsos_spin_{y_axis}").setValue(tgt_y)
    self.ctrl.pulsos_spin_Z.setValue(tgt_z)

    # desloca cada eixo (X e Y juntos; Z por último ou primeiro,
    # conforme preferir – aqui fazemos X/Y depois Z para segurança)
    self.ctrl.move_axis_absolute("X")
    self.ctrl.move_axis_absolute(y_axis)
    self.ctrl.move_axis_absolute("Z")

    self.ctrl.log(
        f"■ Duplo-clique: movendo para {pos.name} "
        f"(X={tgt_x}, {y_axis}={tgt_y}, Z={tgt_z})")

# -----

```

```

# NOVO – cria estrutura e salva imagens no ato da adição
# -----
def _on_position_added(self, pos):
    """
    Executado logo após o usuário clicar “Adicionar posição atual”.
    Cria as pastas da região / posição mecânica / w* e salva:
        – imagem da região completa (regioes/regiao_<n>/<...>.png)
        – ROI azul de cada janela (.../w*/referencia/referencia.png)
        – JSON mínimo (.../<posicao>_w*.json)
    """

    if not getattr(self.ctrl, "prog_mgr", None):
        # programa ainda não criado (deveria existir)
        return
    meta = pos.meta or {}
    if meta.get("action") != "inspect":
        return
    try:
        # ----- Nomes base -----
        regioao_idx = self.inspect_widget.positions().index(pos) + 1
        regioao_name = f"regiao_{regiao_idx}"
        # Posição mecânica usa o nome dado pelo ROI editor
        componentes = meta.get("componentes", {})
        region_png = meta.get("_region_png")
        if region_png:
            np_img = cv2.imdecode(np.frombuffer(region_png, np.uint8),
                                   cv2.IMREAD_COLOR)
            if np_img is not None and np_img.size:
                self.ctrl.prog_mgr.save_region(
                    regioao_name, Image.fromarray(
                        cv2.cvtColor(np_img, cv2.COLOR_BGR2RGB)))
        # ----- w* e janelas -----
        for posicao_nome, comp_data in componentes.items():
            inspecoes = comp_data.get("inspecoes", [])
            for insp in inspecoes:
                # crop ROI azul da imagem da região
                if region_png is None:
                    continue
                # As coordenadas já vêm em pixels absolutos
                x, y = insp.get("posicao", (0, 0))
                w, h = insp.get("tamanho", (0, 0))
                if w == 0 or h == 0:
                    continue
                roi_bgr = np_img[y:y+h, x:x+w].copy()
                if roi_bgr.size == 0:
                    continue
                roi_pil = Image.fromarray(
                    cv2.cvtColor(roi_bgr, cv2.COLOR_BGR2RGB))
                w_nome = insp.get("nome") or insp.get("window_name") \
                    or f"w{inspecoes.index(insp)+1}"
                # salva estrutura + arquivos
                self.ctrl.prog_mgr.save_blue_reference(

```

```

        regioao_name,
        posicao_nome,
        w_nome,
        roi_pil,
        meta=insp    # JSON simples
    )
except Exception as exc:
    self.ctrl.log(f"■ Erro ao criar estrutura da posição: {exc}")
# -----
# NOVO – limpa visores 'Região' e 'ROI' após adicionar INSPEÇÃO
# -----
try:
    if self.inspect_cfg:
        self.inspect_cfg.clear_views()
except Exception as exc:
    self.ctrl.log(f"■ Erro ao limpar visores de inspeção: {exc}")

# -----
# Clique no vídeo → mover cabeça
# -----
def _on_video_click(self, px: float, py: float):
    if not hasattr(self, "_last_frame_sz"):
        return
    # ----- 1. Conversão pixel → mm -----
    # -----
    # Determina retângulo efetivo da imagem dentro do QLabel
    # -----
    disp_w = self.lbl_video.width()
    disp_h = self.lbl_video.height()
    orig_w, orig_h = self._last_frame_sz
    # escala aplicada pelo QPixmap.scaled(KeepAspectRatio)
    scale = min(disp_w / orig_w, disp_h / orig_h)
    pix_w = orig_w * scale
    pix_h = orig_h * scale
    offset_x = (disp_w - pix_w) / 2
    offset_y = (disp_h - pix_h) / 2
    # ignora clique fora da imagem
    if not (offset_x <= px <= offset_x + pix_w and
            offset_y <= py <= offset_y + pix_h):
        return
    # coordenada relativa ao centro da imagem, em pixels
    dx_pix = (px - offset_x - pix_w/2) / scale
    dy_pix = (py - offset_y - pix_h/2) / scale
    # conserta orientação Y
    dy_pix *= self._PIXEL_TO_Y_SIGN
    # ----- 4. Escala MM/PX – modelo óptico -----
    z_cur = self.ctrl.current_positions['Z']
    c    = self.ctrl._fov_coeffs
    w_mm = c["aX"] * z_cur + c["bX"]
    h_mm = c["aY"] * z_cur + c["bY"]
    mmpp_x = w_mm / orig_w

```

```

mmpp_y = h_mm / orig_h

dx_mm = dx_pix * mmpp_x
dy_mm = dy_pix * mmpp_y
# ----- 2. Converte para pulsos ----
dx_pulses = self.ctrl.pulses_from_mm('X', dx_mm)
dy_pulses = self.ctrl.pulses_from_mm(self.y_axis, dy_mm)

# ----- 3. Calcula alvo absoluto ---
cur_x = self.ctrl.current_positions['X']
cur_y = self.ctrl.current_positions[self.y_axis]
tgt_x = cur_x + dx_pulses
tgt_y = cur_y + dy_pulses

# limita dentro da área de trabalho da mesa
x_min, x_max = self._limits['x']
y_min, y_max = self._limits['y']
tgt_x = max(x_min, min(x_max, tgt_x))
tgt_y = max(y_min, min(y_max, tgt_y))

# deslocamentos corrigidos
dx_corr = tgt_x - cur_x
dy_corr = tgt_y - cur_y

# evita comandos nulos / redundantes
if dx_corr:
    self.ctrl.move_relative('X', int(dx_corr))
if dy_corr:
    self.ctrl.move_relative(self.y_axis, int(dy_corr))

self.ctrl.log(f"■ Clique: ΔX={dx_corr} ΔY={dy_corr} pulsos "
              f"(destino X={tgt_x}, Y={tgt_y})")

# ----- helpers ação -----
def _current_action_context(self) -> dict | None:
    key = self.action_selector.current_action()
    if not key:
        return None
    if key == 'fiducial':
        # precisa haver template capturado
        tmpl = self.fiducial_cfg.template_image()
        if tmpl is None:
            return None
        # Usa diretamente os módulos importados no topo
        _, buf = cv2.imencode('.png', tmpl)
        b64 = base64.b64encode(buf).decode('ascii')
        return {
            'action': 'fiducial',
            'window': self.fiducial_cfg.spin_window.value(),
            'radius': self.fiducial_cfg.spin_radius.value(),
            'threshold': self.fiducial_cfg.spin_thresh.value(),

```

```

        'template_png_b64': b64
    }
elif key == 'barcode':
    w,h = self.barcode_cfg.roi_size()
    return {'action':'barcode', 'width':w, 'height':h}
elif key == 'inspect':
    # 1) Dados completos (posições mecânicas + janelas azuis)
    aux_data = self.inspect_cfg.get_cached_auxiliary_data()
    comp_dict = aux_data.get('componentes', {})

    # -----
    # SANITIZAÇÃO:
    #   • mantemos SOMENTE coordenadas e dimensões das janelas;
    #   • cada janela ('w1' ...) recebe sua própria “similaridade”;
    #   • campo 'nome' é salvo para uso futuro.
    # -----
    sim_min = float(self.inspect_cfg.spin_sim.value()) / 100.0

    def _clean_inspecao(idx: int, insp: dict) -> dict:
        return {
            'nome': f'w{idx+1}',
            'posicao': tuple(insp.get('posicao', (0, 0))),
            'tamanho': tuple(insp.get('tamanho', (0, 0))),
            'similaridade': sim_min
        }

    serializable_comp = {}

    for nome, dados in comp_dict.items():
        insp_list = dados.get('inspecoes', [])
        serializable_comp[nome] = dict(
            posicao = tuple(dados.get('posicao', (0, 0))),
            dimensoes = tuple(dados.get('dimensoes', (0, 0))),
            inspecoes = [_clean_inspecao(i, insp)
                          for i, insp in enumerate(insp_list)]
        )

    return {
        'action': 'inspect',
        'componentes': serializable_comp,
        # -----
        # ROI COMPLETA (PNG *bytes*, NÃO base64)
        #   • É apenas um payload temporário para o backend
        #   salvar os .png na árvore de pastas.
        #   • Será removido antes do JSON final, portanto não
        #   aparece no arquivo *.m1 / *.m2.
        # -----
        '_region_png': (lambda _img=self.inspect_cfg
                        ._cached_auxiliary_data
                        .get('region_image'):
                        (None if _img is None else

```

```

        cv2.imencode('.png', _img)[1].tobytes()))()
    }
    elif key == 'dot':
        pat = self.dots_view.current_pattern()
        if pat is None:
            return None    # impede adicionar sem seleção
        return {
            'action': 'dot',
            'dot_id': pat.id,
            'dot_name': pat.name,
            'dot_qty': pat.qty,
            # freq default será lida da aba Config Registradores
            'dot_freq_hz': self.ctrl.findChild(QSpinBox,
                "cfg_spin_D24000_FREQ").value()
        }
    else:
        return {'action': key}

# -----
# Exibe página conforme ação escolhida
# -----
def _show_action_config(self, action_key: str):
    index_map = {
        'dot': 0,
        'barcode': 1,
        'inspect': 2,
        'fiducial': 3
    }
    idx = index_map.get(action_key, 0)
    self._config_stack.setCurrentIndex(idx)

# -----
# Atualiza gráfico correspondente à mesa
# -----
def _update_plot(self):
    if not hasattr(self.ctrl, "mesa_plot_widgets"):
        return
    pts = []
    for p in self.inspect_widget.positions():
        y_val = p.y1 if self.y_axis == 'Y1' else p.y2
        pts.append((p.x, y_val))
    self.ctrl.mesa_plot_widgets[self.mesa].update_points(pts)

# ----- pode ser chamado externamente -----
def set_limits(self, limits: dict[str, tuple[int, int]]):
    """Atualiza área de trabalho desta aba."""
    self._limits = limits

# -----
# valida ponto em relação à área de trabalho desta mesa
# -----

```

```

def _validate_position(self, x, y2, y1, z) -> str | None:
    # escolhe o Y físico desta mesa
    y_val = y1 if self.y_axis == 'Y1' else y2

    x_ok = self._limits['x'][0] <= x <= self._limits['x'][1]
    y_ok = self._limits['y'][0] <= y_val <= self._limits['y'][1]

    if not x_ok or not y_ok:
        return (f"Coordenada fora dos limites da Mesa {self.mesa}.\n"
                f"Permitido X:{self._limits['x']} "
                f"Y:{self._limits['y']}\n"
                f"Recebido X={x:.0f} Y={y_val:.0f}")
    return None

```

```

# -----

```

```

def _adapt_widgets_for_single_y(self):
    """
    Oculta/renomeia elementos de Y1 ou Y2 para que cada aba
    mostre apenas um eixo 'Y'.
    """
    # ---- PositionStatusWidget -----
    # --- trata rótulos e valores de Y -----
    # ---- Oculta linha Y que não pertence à mesa -----
    if self.y_axis == 'Y1':
        self.mov_widget.lbl_pos_y2.hide()
        self.mov_widget.lbl_pos_y1.setText("0")      # rename
    else:
        self.mov_widget.lbl_pos_y1.hide()
        self.mov_widget.lbl_pos_y2.setText("0")

    # ---- MovementControlsWidget (oculta e realinha) -----
    g: QGridLayout = self.mov_widget.grid          # grid de botões

    if self.y_axis == 'Y1':
        # Oculta botões Y2 (btn_up / btn_down) e renomeia Y1
        for b in ('btn_up', 'btn_down'):
            getattr(self.mov_widget, b).hide()
        self.mov_widget.btn_y1_up.setText("Y+")
        self.mov_widget.btn_y1_down.setText("Y-")

        # Reposiciona Y+ / Y- da mesa na coluna do STOP (col 1)
        g.removeWidget(self.mov_widget.btn_y1_up)
        g.removeWidget(self.mov_widget.btn_y1_down)
        g.addWidget(self.mov_widget.btn_y1_up, 0, 1)
        g.addWidget(self.mov_widget.btn_y1_down, 2, 1)
    else:
        # Oculta botões Y1 e renomeia Y2
        self.mov_widget.btn_y1_up.hide()
        self.mov_widget.btn_y1_down.hide()
        self.mov_widget.btn_up.setText("Y+")

```

```

self.mov_widget.btn_down.setText("Y-")

# Reposiciona Y+ / Y- padrão para coluna do STOP
g.removeWidget(self.mov_widget.btn_up)
g.removeWidget(self.mov_widget.btn_down)
g.addWidget(self.mov_widget.btn_up, 0, 1)
g.addWidget(self.mov_widget.btn_down, 2, 1)

# -----
def _get_current_position(self):
    """Retorna posição atual só com o eixo Y físico desta mesa."""
    if self.y_axis == 'Y1':
        return {
            'x': self.ctrl.current_positions['X'],
            'y1': self.ctrl.current_positions['Y1'],
            'y2': 0,
            'z': self.ctrl.current_positions['Z']
        }
    else: # mesa 2 → usa Y2
        return {
            'x': self.ctrl.current_positions['X'],
            'y2': self.ctrl.current_positions['Y2'],
            'y1': 0,
            'z': self.ctrl.current_positions['Z']
        }

# ----- callbacks movimento -----
def _on_step_move(self, axis, dist, feed):
    if axis == 'Y1' or axis == 'Y2':
        axis = self.y_axis # força usar o eixo da mesa
        self.ctrl.move_relative(axis, int(dist))

def _on_jog_start(self, axis, direction, feed):
    # converte Y lógico → eixo físico da mesa
    if axis in ('Y1', 'Y2'):
        axis = self.y_axis

    self._jog_active_axis = axis # memoriza
    self.ctrl.jog_start(axis, '+' if direction > 0 else '-')

def _on_jog_stop(self):
    if self._jog_active_axis:
        self.ctrl.jog_stop(self._jog_active_axis)
        self._jog_active_axis = None

# ----- homing / center -----
def _go_center(self):
    """Leva o cabeçote ao homing virtual configurado p/ esta mesa."""
    self.ctrl.goto_virtual_home(self.mesa)

# -----

```

```

def _to_model(self):
    out = []
    for p in self.inspect_widget.positions():
        # -----
        # GUARDA META COMPLETA NO InspectionPosition
        #
        # • Precisamos do campo dot_qty no runner para escrever
        #   D24100, portanto não pode ser descartado aqui.
        # • camera_params agora recebe **todo** o dict meta
        #   (se existir) preservando "action", "dot_qty", etc.
        # -----
        cam_params = (p.meta.copy()          # dict completo
                      if isinstance(p.meta, dict)
                      else None)
        # Salva somente X, Y (desta mesa) e Z
        if self.y_axis == 'Y1':
            out.append(InspectionPosition(name=p.name,
                                           x=p.x,
                                           y1=p.y1, y2=None,
                                           z=p.z,
                                           camera_params=cam_params))
        else:
            out.append(InspectionPosition(name=p.name,
                                           x=p.x,
                                           y2=p.y2, y1=None,
                                           z=p.z,
                                           camera_params=cam_params))

    return out

```

File: inspection_positions_widget.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/teste_clp

```

# inspection_positions_widget.py
#
# Widget reutilizável para registrar posições de inspeção
# (c) 2025 – livre para uso sob MIT License

from __future__ import annotations
from dataclasses import dataclass
from typing import Callable, List, Dict, Optional

from PyQt6.QtCore import Qt, pyqtSignal
from PyQt6.QtWidgets import (
    QWidget, QListWidget, QListWidgetItem, QPushButton, QVBoxLayout,
    QHBoxLayout, QLabel, QMessageBox
)

# -----

```

```

# Camada de modelo
# -----
@dataclass
class InspectionPosition:
    name: str
    x: float
    y2: float
    y1: float
    z: float = 0.0
    meta: Dict[str, object] | None = None      # espaço para parâmetros extras

    def __str__(self) -> str:
        return f"{self.name} (X={self.x:.2f}, Y2={self.y2:.2f}, Y1={self.y1:.2f}, Z={self.z:.2f})"

class InspectionPositionListModel:
    """
    Classe mínima para guardar a lista de posições; não depende de Qt.
    """

    def __init__(self) -> None:
        self._positions: List[InspectionPosition] = []

    # ---- API pública -----
    def add(self, pos: InspectionPosition) -> None:
        self._positions.append(pos)

    def remove(self, index: int) -> Optional[InspectionPosition]:
        if 0 <= index < len(self._positions):
            return self._positions.pop(index)
        return None

    def clear(self) -> None:
        self._positions.clear()

    def __iter__(self):
        return iter(self._positions)

    def __len__(self):
        return len(self._positions)

    def __getitem__(self, idx):
        return self._positions[idx]

# -----
# Camada de interface
# -----
class InspectionPositionsWidget(QWidget):
    """
    Widget que exibe e gerencia as "Posições de Inspeção".

```

Parâmetros do construtor:

get_current_position: Callable[[], Dict[str,float]]

Função/callable que devolve

{'x':float,'y2':float,'y1':float,'z':float}.

Pode ser None; nesse caso somente add_position(...) externo

poderá alimentar a lista.

Sinais:

positionAdded(InspectionPosition)

positionRemoved(InspectionPosition)

"""

positionAdded = pyqtSignal(object) # InspectionPosition

positionRemoved = pyqtSignal(object) # InspectionPosition

```
def __init__(self,
    get_current_position: Callable[[], Dict[str, float]] | None = None,
    validate_position: Callable[[float,float,float,float], str | None] | None = None,
    get_action_context: Callable[[], Dict[str,object] | None] | None = None,
    parent=None) -> None:
    super().__init__(parent)
    self._get_current_position = get_current_position
    self._validate_position = validate_position
    self._get_action_context = get_action_context
    self._model = InspectionPositionListModel()
    self._build_ui()
```

API pública para outras partes do programa

def add_position(self, pos: InspectionPosition) -> None:

"""Adiciona já com objeto pronto (útil para carregamento de arquivo)."""

self._model.add(pos)

self._append_item_to_list(pos)

self.positionAdded.emit(pos)

def remove_selected(self) -> Optional[InspectionPosition]:

"""Remove a posição atualmente destacada."""

item = self.list_widget.currentItem()

if not item:

return None

row = self.list_widget.row(item)

removed = self._model.remove(row)

if removed:

self.list_widget.takeItem(row)

self.positionRemoved.emit(removed)

return removed

def clear(self) -> None:

self.list_widget.clear()

```

self._model.clear()

def positions(self) -> List[InspectionPosition]:
    """Devolve cópia da lista interna."""
    return list(self._model)

# -----
# Construção UI
# -----

def _build_ui(self) -> None:
    v = QVBoxLayout(self)

    title = QLabel("Posições de Inspeção")
    title.setStyleSheet("font-weight:bold;")
    v.addWidget(title)

    # listagem
    self.list_widget = QListWidget()
    v.addWidget(self.list_widget, 1)

    # botões
    h = QHBoxLayout()
    self.btn_add = QPushButton("Adicionar posição atual")
    self.btn_remove = QPushButton("Remover posição")
    h.addWidget(self.btn_add)
    h.addWidget(self.btn_remove)
    v.addLayout(h)

    # conexões
    self.btn_add.clicked.connect(self._on_add_clicked)
    self.btn_remove.clicked.connect(self.remove_selected)

# -----
# Slots privados
# -----

def _on_add_clicked(self) -> None:
    """
    Obtém a posição atual através do callable injetado e solicita
    ao usuário um nome. Gera o objeto InspectionPosition e adiciona
    ao modelo/lista.
    """
    if not callable(self._get_current_position):
        QMessageBox.warning(self, "Indisponível",
                            "Função get_current_position não foi fornecida.")
        return

    try:
        pos_dict = self._get_current_position() or {}
        x = float(pos_dict.get("x", 0.0))
        y2 = float(pos_dict.get("y2", 0.0))
        y1 = float(pos_dict.get("y1", 0.0))

```

```

        z = float(pos_dict.get("z", 0.0))
except Exception as exc:
    QMessageBox.critical(self, "Erro",
        f"Não foi possível obter a posição atual:\n{exc}")

return

# --- validação opcional -----
if callable(self._validate_position):
    err = self._validate_position(x, y2, y1, z)
    if err:
        QMessageBox.warning(self, "Fora dos limites", err)
    return

# Nome numérico sequencial: 1, 2, 3...
name = str(len(self._model) + 1)

# ----- anexa dados de ação -----
if callable(self._get_action_context):
    meta = self._get_action_context()
    if meta is None:
        QMessageBox.warning(self, "Ação não definida",
            "Selecione uma ação antes de adicionar o ponto.")
        return
    else:
        meta = None
new_pos = InspectionPosition(name, x, y2, y1, z, meta=meta)
self._model.add(new_pos)
self._append_item_to_list(new_pos)
self.positionAdded.emit(new_pos)

def _append_item_to_list(self, pos: InspectionPosition) -> None:
    txt = (f"{pos.name}. "
        f"X={pos.x:.2f} Y2={pos.y2:.2f} "
        f"Y1={pos.y1:.2f} Z={pos.z:.2f}")
    if pos.meta and pos.meta.get("action") == "dot":
        txt += f" [dot {pos.meta.get('dot_qty',1)}]"
    item = QListWidgetItem(txt)
    self.list_widget.addItem(item)

# -----
# Test-drive rápido do widget isolado
# -----
if __name__ == "__main__":
    import sys
    import random
    from PyQt6.QtWidgets import QApplication

    # Simula um motion controller retornando coordenadas random
    def _mock_current_position():
        return {"x": random.uniform(0, 100),

```

```

        "y": random.uniform(0, 100),
        "z": random.uniform(0, 50)}

app = QApplication(sys.argv)
w = InspectionPositionsWidget(get_current_position=_mock_current_position)
w.resize(400, 300)
w.show()

# Conecta sinais a prints
w.positionAdded.connect(lambda p: print("ADDED:", p))
w.positionRemoved.connect(lambda p: print("REMOVED:", p))

sys.exit(app.exec())

```

File: settings.json

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/teste_clp

```

{
  "table_limits": {
    "1": {
      "x": [
        25001,
        50000
      ],
      "y": [
        0,
        18000
      ]
    },
    "2": {
      "x": [
        0,
        25000
      ],
      "y": [
        0,
        18000
      ]
    }
  },
  "camera_index": 2,
  "focus_calibration": {
    "a": 0.4185714285714286,
    "b": 367.0
  },
  "axis_calibration": {
    "X": 100.0,
    "Y2": 50.0,
    "Y1": 50.0,

```

```

    "Z": 20.0
},
"virtual_home": {
    "1": {
        "x": 38744,
        "y": 5796
    },
    "2": {
        "x": 11321,
        "y": 5750
    }
},
"camera_mm_per_pixel": 0.05,
"camera_fov": {
    "z0_z_pulses": 0,
    "z0_width_mm": 68.5,
    "z0_height_mm": 52.5,
    "z1_z_pulses": 700,
    "z1_width_mm": 37.0,
    "z1_height_mm": 29.0
},
"camera_nozzle_offset": {
    "x": 24,
    "y": -4586
},
"projects_dir": "C:\\Users\\sense\\PycharmProjects\\cnc_aoi\\teste_clp\\Projetos"
}

```

File: alignment_dialog.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/teste_clp

```

from PyQt6.QtWidgets import (
    QDialog, QVBoxLayout, QLabel, QPushButton, QHBoxLayout, QMessageBox
)
from PyQt6.QtCore import Qt, pyqtSignal

```

```

class AlignmentDialog(QDialog):

```

```

    """

```

Passo-a-passo para medir o deslocamento entre
o centro da CÂMERA e o centro do NOZZLE.

1. Leve o NOZZLE manualmente até um ponto de referência e clique
"Capturar bico".
2. Leve a CÂMERA (centro da imagem) ao MESMO ponto e clique
"Capturar câmera".

O diálogo calcula ΔX , ΔY e salva.

```

    """

```

```

offsetSaved = pyqtSignal(dict)    # {'x':dx, 'y':dy}

```

```

def __init__(self, ctrl, parent=None):
    super().__init__(parent, Qt.WindowType.Window)
    self.setWindowTitle("Alinhamento Nozzle ↔ Câmera")
    self.setWindowModality(Qt.WindowModality.NonModal)
    self.setWindowFlag(Qt.WindowType.WindowStaysOnTopHint, True)

    self._c = ctrl
    self._noz_pos = None # (x,y)
    self._cam_pos = None

    self._build_ui()

# ----- UI -----
def _build_ui(self):
    v = QVBoxLayout(self)
    v.addWidget(QLabel(
        "1. Posicione o BICO sobre um ponto fixo e clique em "
        "<b>Capturar bico</b>.\n"
        "2. Mova a CÂMERA até o MESMO ponto (use o clique na imagem) "
        "e clique em <b>Capturar câmera</b>.\n"
        "3. Clique <b>Salvar</b> para gravar o offset."))

    # linha status
    self.lbl_status = QLabel("ΔX = 0  ΔY = 0")
    self.lbl_status.setAlignment(Qt.AlignmentFlag.AlignCenter)
    v.addWidget(self.lbl_status)

    h1 = QHBoxLayout()
    self.btn_noz = QPushButton("Capturar bico")
    self.btn_cam = QPushButton("Capturar câmera")
    h1.addWidget(self.btn_noz)
    h1.addWidget(self.btn_cam)
    v.addLayout(h1)

    h2 = QHBoxLayout()
    btn_save = QPushButton("Salvar")
    btn_cancel = QPushButton("Fechar")
    h2.addStretch()
    h2.addWidget(btn_save)
    h2.addWidget(btn_cancel)
    v.addLayout(h2)

    # conexões
    self.btn_noz.clicked.connect(self._capture_nozzle)
    self.btn_cam.clicked.connect(self._capture_camera)
    btn_save.clicked.connect(self._on_save)
    btn_cancel.clicked.connect(self.close)

# ----- capturas -----
def _capture_nozzle(self):

```

```

self._noz_pos = (self._c.current_positions['X'],
                self._c.current_positions['Y1' # Y física da Mesa 1
                if self._c.tab_widget.currentWidget() is self._c.mesa_tabs[1]
                else 'Y2'])
self._update_status()

def _capture_camera(self):
    """
    Posição da CÂMERA é simplesmente a posição atual dos eixos,
    pois o usuário deve mover o cabeçote de forma que o ponto de
    interesse fique no centro da imagem.
    """
    self._cam_pos = (self._c.current_positions['X'],
                    self._c.current_positions['Y1'
                    if self._c.tab_widget.currentWidget() is self._c.mesa_tabs[1]
                    else 'Y2'])
    self._update_status()

def _update_status(self):
    if self._noz_pos and self._cam_pos:
        dx = self._cam_pos[0] - self._noz_pos[0]
        dy = self._cam_pos[1] - self._noz_pos[1]
        self.lbl_status.setText(f"ΔX = {dx} ΔY = {dy}")
    else:
        self.lbl_status.setText("ΔX = ? ΔY = ?")

# ----- salvar -----
def _on_save(self):
    if not (self._noz_pos and self._cam_pos):
        QMessageBox.warning(self, "Incompleto",
                            "Capture as duas posições antes de salvar.")
        return
    dx = self._cam_pos[0] - self._noz_pos[0]
    dy = self._cam_pos[1] - self._noz_pos[1]
    self.offsetSaved.emit({"x": dx, "y": dy})
    self.close()

```

File: settings_manager.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/teste_clp

```

# settings_manager.py
import json, typing as _t
from pathlib import Path

class SettingsManager:
    _FILE = Path(__file__).with_name("settings.json")

    def __init__(self):
        self.data = {                # defaults

```

```

"table_limits": {
    "1": {"x": [0, 25000], "y": [0, 18000]},
    "2": {"x": [0, 25000], "y": [0, 18000]}
},
"camera_index": 0,
"focus_calibration": {"a": 0.0, "b": 0.0},
# ----- steps/mm por eixo -----
"axis_calibration": {
    "X": 1.0,
    "Y2": 1.0,
    "Y1": 1.0,
    "Z": 1.0
},
# ----- homing virtual de cada mesa -----
"virtual_home": {
    "1": {"x": 0, "y": 0},
    "2": {"x": 0, "y": 0}
},
# ----- escala câmera (mm por pixel @ Z-ref = 0) -----
"camera_mm_per_pixel": 0.05,
"camera_fov": { # NOVO – campo de visão
    "z0_z_pulses": 0,
    "z0_width_mm": 61.0,
    "z0_height_mm": 45.0,
    "z1_z_pulses": 700,
    "z1_width_mm": 30.5,
    "z1_height_mm": 22.5
},
# ----- OFFSET CÂMERA → NOZZLE -----
# Guardado em PULSOS (mesma unidade dos motores).
"camera_nozzle_offset": {"x": 0, "y": 0},

# ----- NOVO: pasta-raiz dos projetos -----
# será algo como C:\QualquerLugar\Projetos
"projects_dir": str((Path.home() / "Projetos").resolve())
}
self.load()

```

```

# -----
def load(self):
    if self._FILE.exists():
        try:
            self.data.update(json.loads(self._FILE.read_text("utf-8")))
        except Exception:
            pass # mantém defaults

def save(self):
    try:
        self._FILE.write_text(json.dumps(self.data, indent=2), encoding="utf-8")
    except Exception:
        pass

```

```

# conveniência
@property
def table_limits(self) -> dict[int, dict[str, _t.Any]]:
    return {int(k): v for k, v in self.data["table_limits"].items()}

@table_limits.setter
def table_limits(self, v):
    self.data["table_limits"] = {str(k): v[k] for k in v}

@property
def camera_index(self) -> int:
    return int(self.data.get("camera_index", 0))

@camera_index.setter
def camera_index(self, idx: int):
    self.data["camera_index"] = int(idx)

# ----- foco -----
@property
def focus_coefs(self):
    d = self.data.get("focus_calibration", {"a":0.0,"b":0.0})
    return d.get("a",0.0), d.get("b",0.0)

@focus_coefs.setter
def focus_coefs(self, ab):
    self.data["focus_calibration"] = {"a":ab[0], "b":ab[1]}

# ----- steps/mm -----
@property
def axis_steps(self) -> dict[str,float]:
    return {k: float(v) for k,v in self.data.get("axis_calibration", {}).items()}

@axis_steps.setter
def axis_steps(self, d: dict[str,float]):
    self.data["axis_calibration"] = {k: float(v) for k,v in d.items()}

# ----- homing virtual -----
@property
def virtual_home(self) -> dict[int, dict[str,int]]:
    vh = self.data.get("virtual_home", {"1":{"x":0,"y":0},"2":{"x":0,"y":0}})
    return {int(k): {"x":int(v["x"]), "y":int(v["y"])} for k,v in vh.items()}

@virtual_home.setter
def virtual_home(self, d: dict[int,dict[str,int]]):
    self.data["virtual_home"] = {str(k): {"x":int(v["x"]), "y":int(v["y"])}
                                for k,v in d.items()}

# ----- câmera -----
@property
def camera_mm_per_pixel(self) -> float:
    return float(self.data.get("camera_mm_per_pixel", 0.05))

@camera_mm_per_pixel.setter

```

```

def camera_mm_per_pixel(self, v: float):
    self.data["camera_mm_per_pixel"] = float(v)
# ----- campo-de-visão -----
@property
def camera_fov(self) -> dict:
    return self.data.get("camera_fov", {})
@camera_fov.setter
def camera_fov(self, d: dict):
    self.data["camera_fov"] = d

# ----- OFFSET CÂMERA↔NOZZLE -----
@property
def cam_noz_offset(self) -> dict[str, int]:
    off = self.data.get("camera_nozzle_offset", {"x": 0, "y": 0})
    return {"x": int(off.get("x", 0)), "y": int(off.get("y", 0))}

@cam_noz_offset.setter
def cam_noz_offset(self, off: dict[str, int]):
    self.data["camera_nozzle_offset"] = {"x": int(off["x"]), "y": int(off["y"])}

# ----- PASTA «PROJETOS» -----
@property
def projects_dir(self) -> Path:
    return Path(self.data.get("projects_dir",
                              str(Path.home() / "Projetos"))).resolve()

@projects_dir.setter
def projects_dir(self, p: Path | str):
    self.data["projects_dir"] = str(Path(p).resolve())

```

File: table_positions_backend.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/teste_clp

```
# table_positions_backend.py
```

```
"""
```

Backend de persistência específico das mesas.

Grava apenas X, Y (físico da mesa) e Z em JSON.

Extensões Mesa 1 → *.m1

Mesa 2 → *.m2

```
"""
```

```
from pathlib import Path
```

```
from typing import List
```

```
from program_io_widget import ProgramStorageBackend
```

```
from inspection_positions_widget import InspectionPositionsWidget, InspectionPosition
```

```
from adhesive_program_manager import AdhesiveProgramManager
```

```
import os, copy, json, cv2, numpy as np
```

```
from PIL import Image
```

[illegible]

```

mgr.save_region(regiao_name, region_pil)
# ----- janelas w* -----
componentes = meta.get("componentes", {})
for posicao_nome, comp_data in componentes.items():
    for insp in comp_data.get("inspecoes", []):
        # As coordenadas já estão em pixels absolutos
        x, y = map(int, insp.get("posicao", (0, 0)))
        w, h = map(int, insp.get("tamanho", (0, 0)))

        # Ignora valores vazios ou negativos
        if w <= 0 or h <= 0:
            continue

        # Garante que o recorte não ultrapasse a borda da imagem
        img_h, img_w = np_img.shape[:2]
        x = max(0, min(x, img_w - 1))
        y = max(0, min(y, img_h - 1))
        w = min(w, img_w - x)
        h = min(h, img_h - y)

        roi = np_img[y:y + h, x:x + w]
        if roi.size == 0:
            continue

        roi_pil = Image.fromarray(
            cv2.cvtColor(roi, cv2.COLOR_BGR2RGB))
        w_nome = insp.get("nome", "w?")
        meta_js = {
            "janela_azul": (x, y, w, h),
            "similaridade": insp.get("similaridade", 0.9)
        }
        mgr.save_blue_reference(
            regiao_name,
            posicao_nome,
            w_nome,
            roi_pil,
            meta_js)

except Exception as exc:
    print("[WARN] Falha ao salvar imagens de inspeção:", exc)

# -----
# EXTRA 2 ■ remove chaves temporárias antes de serializar JSON
# -----
json_ready: List[dict] = []
for p in self._w.positions():
    y = p.y1 if self._y_axis == 'Y1' else p.y2
    m = copy.deepcopy(p.meta or {})
    if m.get("action") == "inspect":
        # descarta payload binário antes do JSON
        m.pop("_region_png", None)

```

```

json_ready.append({
    "name": p.name,
    "x": p.x,
    "y": y,
    "z": p.z,
    "meta": m
})

```

```

Path(filename).write_text(json.dumps(json_ready, indent=2), 'utf-8')

```

```

# -----
# EXTRA 3 ■ apenas garante árvore vazia quando não havia w*
# -----
try:
    # Nome do programa = nome do arquivo sem extensão
    prog_name = Path(filename).stem
    # Raiz = mesma pasta do *.m? (mantém simples)
    base_dir = Path(filename).parent
    mgr = AdhesiveProgramManager(base_dir)
    # Se já existir não apaga – somente garante estrutura
    try:
        mgr.create_program(prog_name, overwrite=False)
    except FileExistsError:
        mgr.model_dir = Path(base_dir) / "modelos" / prog_name

    for idx, p in enumerate(self._w.positions(), 1):
        meta = p.meta or {}
        if meta.get("action") != "inspect":
            continue
        componentes : dict = meta.get("componentes", {})
        if not componentes:
            continue

        # Cada posição de inspeção recebe uma pasta "regiao_<n>"
        regiao = f"regiao_{idx}"
        for comp_name, comp_data in componentes.items():
            inspecoes = comp_data.get("inspecoes", [])
            # Cria sub-pastas w1, w2...
            for i in range(max(1, len(inspecoes))):
                w_name = f"w{i+1}"
                mgr.ensure_position_dirs(regiao, comp_name, w_name)
except Exception as exc:
    # não falha a gravação do programa se a estrutura de pastas
    # não puder ser criada (por exemplo, permissão)
    print("[WARN] Falha ao preparar pasta de inspeção:", exc)

return True

```

```

# ----- carregar -----
def load_from_file(self, filename: str) -> bool:
    raw = json.loads(Path(filename).read_text('utf-8'))

```

```

if not isinstance(raw, list):
    raise ValueError("formato inválido – esperada lista JSON")

self._w.clear()
for d in raw:
    y_val = float(d.get("y", 0))
    pos = InspectionPosition(
        name=str(d.get("name", "")),
        x=float(d.get("x", 0)),
        y1=y_val if self._y_axis == 'Y1' else 0.0,
        y2=y_val if self._y_axis == 'Y2' else 0.0,
        z=float(d.get("z", 0)),
        meta=d.get("meta")
    )
    self._w.add_position(pos)
return True

```

File: axes_control_tab.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/teste_clp

```
# axes_control_tab.py
```

```
"""
```

Widget completo de controle de eixos (quatro eixos + inspeção + salvar / carregar + sequência). Reaproveita toda a lógica já existente no MultiAxisMotorController; somente a parte visual fica aqui.

```
"""
```

```

from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QGridLayout, QFrame, QSizePolicy
)
from mesa_plot_widget import MesaPlotWidget
from PyQt6.QtCore import Qt, QTimer

```

```

from movement_controls_widget import MovementControlsWidget
from position_status_widget import PositionStatusWidget
from inspection_positions_widget import InspectionPositionsWidget
from program_io_widget import ProgramIOWidget
from positions_backend import InspectionPositionsBackend
from sequence_control import SequenceControlWidget
from sequence_control import InspectionPosition          # typing

```

```
class AxesControlTab(QWidget):
```

```
    """
```

Aba completa de controle. Recebe a instância do controlador principal (MultiAxisMotorController) para encaminhar sinais / slots.

Se `primary=True`, alguns atributos comuns (pos_widget, mov_widget ...) são expostos no controlador para que o código já existente continue funcionando sem alteração.

```
    """
```

```

def __init__(self, controller, *, primary: bool = False):
    super().__init__()
    self.ctrl = controller
    self.primary = primary
    self._build_ui()

# -----
# CONSTRUÇÃO
# -----

def _build_ui(self):
    """Copia exatamente a UI usada anteriormente."""
    grid = QGridLayout(self)
    grid.setContentsMargins(0, 0, 0, 0)
    grid.setHorizontalSpacing(0)
    grid.setVerticalSpacing(0)

    # ----- COLUNA ESQUERDA (15 %) -----
    left_col = QVBoxLayout()

    aux_group = self.ctrl.create_auxiliary_controls()
    left_col.addWidget(aux_group)
    left_col.addStretch()

    left_wrap = QWidget(); left_wrap.setLayout(left_col)
    grid.addWidget(left_wrap, 0, 0)

    # ----- COLUNA CENTRAL (70 %) -----
    # ----- VISUALIZAÇÃO DAS MESAS -----
    center_top = QFrame(objectName="centerTop")
    grid.addWidget(center_top, 0, 1)
    hplot = QHBoxLayout(center_top); hplot.setContentsMargins(2,2,2,2)
    # cria widgets gráficos para cada mesa
    self.plot_m1 = MesaPlotWidget(self.ctrl.table_limits[1], title="Mesa 1")
    self.plot_m2 = MesaPlotWidget(self.ctrl.table_limits[2], title="Mesa 2")
    hplot.addWidget(self.plot_m1); hplot.addWidget(self.plot_m2)

    # ----- COLUNA DIREITA (15 %) -----
    right_col = QVBoxLayout()

    self.mov_widget = MovementControlsWidget()
    right_col.addWidget(self.mov_widget)

    # Posições de inspeção
    self.inspect_widget = InspectionPositionsWidget(
        get_current_position=self.ctrl._get_current_position_dict)
    self.inspect_widget.setMaximumHeight(260)
    right_col.addWidget(self.inspect_widget)

    # Aba de controle global → utiliza extensão genérica .json
    backend = InspectionPositionsBackend(self.inspect_widget)
    file_filter = "Programas (*.json)"

```

```

self.prog_widget = ProgramIOWidget(
    backend,
    file_filter=file_filter,
    default_suffix=".json",
    default_dir=self.ctrl.projects_dir
)
right_col.addWidget(self.prog_widget)
# Esconde grupo de botões (passará a ser acessado via menu)
self.prog_widget.hide()

self.seq_widget = SequenceControlWidget(
    motion=self.ctrl._plc_motion_backend, # criado no controlador
    camera=None
)
right_col.addWidget(self.seq_widget)

right_col.addStretch()
right_wrap = QWidget(); right_wrap.setLayout(right_col)
grid.addWidget(right_wrap, 0, 2)

# ----- LINHA INFERIOR -----
left_spacer = QWidget()
right_spacer = QWidget()
for s in (left_spacer, right_spacer):
    s.setSizePolicy(QSizePolicy.Policy.Expanding,
                   QSizePolicy.Policy.Preferred)

bottom_placeholder = QFrame(objectName="centerBottom")
bottom_placeholder.setStyleSheet(
    "QFrame { background-color:#ECEFF1; border: 1px dashed #B0BEC5; }")
bottom_placeholder.setSizePolicy(
    QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Expanding)

grid.addWidget(left_spacer, 1, 0)
grid.addWidget(bottom_placeholder, 1, 1)
grid.addWidget(right_spacer, 1, 2)

# ----- proporções (colunas 15|70|15, linhas 70|30) -----
grid.setColumnStretch(0, 15)
grid.setColumnStretch(1, 70)
grid.setColumnStretch(2, 15)
grid.setRowStretch(0, 7)
grid.setRowStretch(1, 3)

# ----- CONEXÕES DE SINAIS -----
self.mov_widget.stepMoveRequested.connect(
    self.ctrl._on_step_move_requested)
self.mov_widget.jogStart.connect(self.ctrl._on_widget_jog_start)
self.mov_widget.jogStop.connect(self.ctrl._on_widget_jog_stop)
self.mov_widget.goToZeroRequested.connect(self.ctrl._on_go_to_zero)

```

```

self.mov_widget.emergencyStopToggled.connect(
    lambda e: self.ctrl.emergency_stop() if e else None)

self.inspect_widget.positionAdded.connect(
    lambda _: self.seq_widget.set_positions(self._to_model()))
self.inspect_widget.positionRemoved.connect(
    lambda _: self.seq_widget.set_positions(self._to_model()))
self.prog_widget.fileLoaded.connect(
    lambda _: self.seq_widget.set_positions(self._to_model()))

# logs salvar / carregar
self.prog_widget.fileLoaded.connect(
    lambda f: self.ctrl.log(f"Programa carregado de: {f}"))
self.prog_widget.fileSaved.connect(
    lambda f: self.ctrl.log(f"Programa salvo em: {f}"))

# ----- expõe widgets principais no controlador -----
if self.primary:
    self.ctrl.mov_widget = self.mov_widget
    self.ctrl.inspect_widget = self.inspect_widget
    self.ctrl.prog_io_widget = self.prog_widget
    self.ctrl.seq_widget = self.seq_widget
    # expõe gráficos
    self.ctrl.mesa_plot_widgets = {1: self.plot_m1,
                                   2: self.plot_m2}

# ----- helper: converte lista p/ SequenceControl -----
def _to_model(self):
    lst = []
    for p in self.inspect_widget.positions():
        lst.append(
            InspectionPosition(
                name=p.name,
                x=p.x, y2=p.y2, y1=p.y1, z=p.z
            )
        )
    return lst

```

File: program_io_widget.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/teste_clp

```

from __future__ import annotations
from pathlib import Path
from typing import Protocol, runtime_checkable

```

```

from PyQt6.QtCore import pyqtSignal
from PyQt6.QtWidgets import (
    QWidget, QGroupBox, QVBoxLayout, QPushButton,
    QFileDialog, QMessageBox
)

# -----
# 1) Protocolo (interface) para o backend que realmente salva/carrega
# -----
@runtime_checkable
class ProgramStorageBackend(Protocol):
    """
    Interface mínima que o backend de persistência deve expor.
    – InspectionPositionManager do projeto antigo já satisfaz.
    """

    def save_to_file(self, filename: str) -> bool: ...
    def load_from_file(self, filename: str) -> bool: ...

# -----
# 2) Widget de UI
# -----
class ProgramIOWidget(QWidget):
    """
    Painel “Salvar / Carregar” pronto para ser acoplado a qualquer
    aplicação. Usa um backend que implemente ProgramStorageBackend.
    """

    fileSaved = pyqtSignal(str) # Caminho salvo
    fileLoaded = pyqtSignal(str) # Caminho carregado

    def __init__(self,
        backend: ProgramStorageBackend,
        *,
        file_filter: str = "Arquivos JSON (*.json)",
        default_suffix: str = ".json",
        default_dir: str | Path = "",
        parent=None) -> None:
        super().__init__(parent)
        if not isinstance(backend, ProgramStorageBackend):
            raise TypeError(
                "backend não implementa ProgramStorageBackend (save_to_file / load_from_file)"
            )
        self._backend = backend
        self._file_filter = file_filter
        self._default_suf = default_suffix
        self._default_dir = str(default_dir) if default_dir else ""
        self._build_ui()

# -----
# construção da interface

```

```

# -----
def _build_ui(self):
    grp = QGroupBox("Salvar / Carregar")
    v = QVBoxLayout(grp)

    self.btn_save = QPushButton("Salvar Programa")
    self.btn_load = QPushButton("Carregar Programa")

    v.addWidget(self.btn_save)
    v.addWidget(self.btn_load)

    main = QVBoxLayout(self)
    main.addWidget(grp)
    main.addStretch()

    # conexões
    self.btn_save.clicked.connect(self._on_save_clicked)
    self.btn_load.clicked.connect(self._on_load_clicked)

# -----
# slots privados
# -----
def _on_save_clicked(self):
    fname, _ = QFileDialog.getSaveFileName(
        self, "Salvar Programa", "", self._file_filter
    )
    if not fname:
        return
    # garante extensão correta
    if self._default_suf and not fname.lower().endswith(self._default_suf):
        fname += self._default_suf

    ok = False
    try:
        ok = self._backend.save_to_file(fname)
    except Exception as exc:
        QMessageBox.critical(self, "Erro", f"Falha ao salvar:\n{exc}")
        return

    if ok:
        self.fileSaved.emit(fname)
    else:
        QMessageBox.critical(self, "Erro", "Falha ao salvar o programa")

def _on_load_clicked(self):
    fname, _ = QFileDialog.getOpenFileName(
        self, "Carregar Programa",
        self._default_dir,
        self._file_filter
    )
    if not fname:

```

```
return
```

```
ok = False
```

```
try:
```

```
    ok = self._backend.load_from_file(fname)
```

```
except Exception as exc:
```

```
    QMessageBox.critical(self, "Erro", f"Falha ao carregar:\n{exc}")
```

```
return
```

```
if ok:
```

```
    self.fileLoaded.emit(fname)
```

```
else:
```

```
    QMessageBox.critical(self, "Erro", "Falha ao carregar o programa")
```

```
# -----
```

```
# 3) Demonstração rápida
```

```
# -----
```

```
if __name__ == "__main__":
```

```
    import sys, json, tempfile
```

```
    from dataclasses import dataclass, asdict
```

```
    from PyQt6.QtWidgets import QApplication, QLabel
```

```
# Backend de exemplo -----
```

```
class _DummyBackend:
```

```
    """Exemplo simples que salva/carrega um dict em JSON."""
```

```
    def __init__(self):
```

```
        self.data = {"positions": [], "sequences": []}
```

```
    def save_to_file(self, filename: str) -> bool:
```

```
        Path(filename).write_text(json.dumps(self.data, indent=2),  
                                   encoding="utf-8")
```

```
        return True
```

```
    def load_from_file(self, filename: str) -> bool:
```

```
        self.data = json.loads(Path(filename).read_text(encoding="utf-8"))  
        return True
```

```
# Qt demo -----
```

```
app = QApplication(sys.argv)
```

```
backend = _DummyBackend()
```

```
widget = ProgramIOWidget(backend)
```

```
widget.resize(220, 120)
```

```
# Mostra em label onde salvou/carregou
```

```
info = QLabel()
```

```
def _saved(f): info.setText(f"Salvo: {Path(f).name}")
```

```
def _loaded(f): info.setText(f"Carregado: {Path(f).name}")
```

```
widget.fileSaved.connect(_saved)
```

```
widget.fileLoaded.connect(_loaded)
```

```

wrapper = QWidget()
lay = QVBoxLayout(wrapper); lay.addWidget(widget); lay.addWidget(info)
wrapper.show()
sys.exit(app.exec())

```

File: adhesive_program_manager.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/teste_clp

```
# adhesive_program_manager.py
```

```
"""
```

```
AdhesiveProgramManager
```

```
-----
```

Cria e gerencia a estrutura de diretórios/arquivos de um programa de inspeção da adesivadora.

Estrutura-alvo (ver txt original):

```
modelos/<nome_prog>/
```

```
    ■■ logs/<datahora>_<codigo>/          (# sessões de produção)
```

```
    ■    ■■ <datahora>_<codigo>.json      (log consolidado)
```

```
    ■    ■■ <datahora>_SFCS.json          (opcional)
```

```
    ■    ■■ regiao_<n>.png                (snapshots)
```

```
    ■■ regioes/
```

```
        ■■ <regiao_X>/posicoes_mecanicas/
```

```
            ■■ <posicao_mec_Y>/inspecoes/
```

```
                ■■ w1/ ... w4/
```

```
                    ■■ correct/          (imagens OK)
```

```
                    ■■ incorrect/        (imagens NOK)
```

```
                    ■■ <posicao_wi>.json (meta)
```

```
                    ■■ <posicao_wi>.png (ROI azul)
```

```
                    ■■ referencia/referencia.png
```

```
"""
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass, asdict
```

```
from pathlib import Path
```

```
from datetime import datetime
```

```
import json, shutil
```

```
from typing import List, Tuple, Dict, Any, Optional
```

```
from PIL import Image
```

```
# -----
```

```
# 1) Tipos de dados — opcionais, mas ajudam a manter contrato
```

```
# -----
```

```
@dataclass
```

```
class InspectionWindowInfo:
```

```
    posicao: Tuple[int, int]    # x,y absolutos na ROI mecânica
```

```
    tamanho: Tuple[int, int]  # w,h absolutos
```

```
    threshold: int            = 128
```

```
    cor_pixel: str            = 'branco' # 'branco'|'preto'
```

```
percentual_minimo: float    = .80    # 0-1
# caminho para imagens corret/incorrect ficarão no disco
```

```
@dataclass
```

```
class MechanicalPositionInfo:
```

```
    """Informações persistidas no JSON '<posicao>_w?.json'."""
```

```
janela_azul: Tuple[int,int,int,int]    # x,y,w,h
```

```
similaridade: float                # 0-1
```

```
inspecoes: List[InspectionWindowInfo]  # w1...w4
```

```
# -----
```

```
# 2) Classe principal
```

```
# -----
```

```
class AdhesiveProgramManager:
```

```
    def __init__(self, base_dir: str | Path = ".") -> None:
```

```
        self.base_dir = Path(base_dir).resolve()
```

```
        self.model_dir: Path | None = None
```

```
# -----
```

```
# Propriedade de compatibilidade
```

```
# Várias partes do código legado ainda acessam `prog_mgr.path`.
```

```
# Para não precisar alterar todos os módulos, expomos `.path`
```

```
# como alias somente-leitura para `model_dir`.
```

```
# -----
```

```
@property
```

```
def path(self) -> Path | None:
```

```
    """
```

```
    Alias para `model_dir`.
```

```
    Retorna None até `create_program()` ser chamado.
```

```
    """
```

```
    return self.model_dir
```

```
# ----- criação do PROGRAMA -----
```

```
def create_program(self, name: str, overwrite: bool = True) -> Path:
```

```
    """
```

```
    Cria ou abre um programa:
```

```
    modelos/<name> + sub-pastas vazias.
```

```
    """
```

```
self.model_dir = self.base_dir / "modelos" / name
```

```
if self.model_dir.exists() and not overwrite:
```

```
    raise FileExistsError(f"Programa \"{name}\" já existe.")
```

```
# limpa só se overwrite=True
```

```
if self.model_dir.exists() and overwrite:
```

```
    shutil.rmtree(self.model_dir)
```

```
# cria tronco principal
```

```
# sub-pastas fixas
```

```
for sub in ("logs", "regioes", "arquivos_auxiliares"):
```

```
    (self.model_dir / sub).mkdir(parents=True, exist_ok=True)
```

```

# grava config.txt minimal (mesma linha de raciocínio do PM original)
cfg_txt = self.model_dir / "config.txt"
cfg_txt.write_text(f"model_name={name}\n"
                  f"created_at={datetime.now()}\n"
                  f"created_by=adhesive_program_manager\n",
                  encoding="utf-8")

return self.model_dir

# ----- regiões / posições -----
def ensure_position_dirs(self,
                        regioao: str,
                        posicao: str,
                        w: str) -> Path:
    """
    Garante que .../regioes/<regiao>/posicoes_mecanicas/<posicao>/inspecoes/<w>/
    exista e devolve esse Path.
    """
    if not self.model_dir:
        raise RuntimeError("create_program() ainda não foi chamado.")

    target = (self.model_dir / "regioes" / regioao /
              "posicoes_mecanicas" / posicao /
              "inspecoes" / w)

    # sub-pastas fixas
    for p in ("correct", "incorrect", "referencia"):
        (target / p).mkdir(parents=True, exist_ok=True)
    return target

# ----- salvar imagens e JSONs -----
def save_blue_reference(self,
                        regioao: str,
                        posicao: str,
                        w: str,
                        roi_pil: Image.Image,
                        meta: MechanicalPositionInfo | Dict[str,Any]):
    """
    Salva:
    • ROI azul (.../<posicao>_<w>.png)
    • JSON (.../<posicao>_<w>.json)
    Ambos dentro de .../inspecoes/<w>/
    """
    dest = self.ensure_position_dirs(regiao, posicao, w)

    png_path = dest / f"{posicao}_{w}.png"
    json_path = dest / f"{posicao}_{w}.json"

    # imagem
    roi_pil.save(png_path, format="PNG", compress_level=0)

```

```

# json
if isinstance(meta, MechanicalPositionInfo):
    meta_to_dump = asdict(meta)
else:
    meta_to_dump = meta
json_path.write_text(json.dumps(meta_to_dump, indent=2), "utf-8")

# referência global (janela azul) — sobrescreve
ref_dir = dest / "referencia"
roi_pil.save(ref_dir / "referencia.png", format="PNG", compress_level=0)

# -----
# NOVO: salva imagem completa da região logo que é criada
# -----
def save_region(self, regioao: str, img_pil: "Image.Image"):
    """
    Salva regioes/<regiao>/<regiao>.png (sobre-escreve).
    """
    if not self.model_dir:
        raise RuntimeError("create_program() ainda não foi chamado.")
    target_dir = self.model_dir / "regioes" / regioao
    target_dir.mkdir(parents=True, exist_ok=True)
    img_pil.save(target_dir / f"{regiao}.png",
                 format="PNG", compress_level=0)

# ----- LOG de execução -----
def new_log_session(self, codigo_lote: str) -> Path:
    """
    Cria pasta logs/<datahora>_<codigo>/ e devolve o Path.
    """
    if not self.model_dir:
        raise RuntimeError("create_program() ainda não foi chamado.")

    ts = datetime.now().strftime("%Y%m%d_%H%M%S")
    dest = self.model_dir / "logs" / f"{ts}_{codigo_lote}"
    dest.mkdir(parents=True, exist_ok=True)
    return dest

def append_json_log(self, session_dir: Path, data: Dict[str, Any]):
    """
    Acrescenta entrada a <session>/*.json (cria se não existir).
    """
    main_json = session_dir / f"{session_dir.name}.json"
    entries : List[Dict] = []
    if main_json.exists():
        entries = json.loads(main_json.read_text("utf-8"))
        if not isinstance(entries, list):
            entries = [entries]
    entries.append(data)
    main_json.write_text(json.dumps(entries, indent=2), "utf-8")

```

```

def save_snapshot(self, session_dir: Path,
                  regio_idx: int,
                  img: Image.Image):
    """
    Guarda regio_{n}.png dentro da pasta da sessão.
    """
    snap_path = session_dir / f"regiao_{regio_idx}.png"
    img.save(snap_path, format="PNG", compress_level=0)

# -----
# 3) Exemplo de uso rápido
# -----
if __name__ == "__main__":          # test-drive
    from PIL import Image, ImageDraw

    base = AdhesiveProgramManager(".")
    base.create_program("demo_prog", overwrite=True)

    # cria ROI fictícia só para teste
    roi = Image.new("RGB", (400, 300), "gray")
    d = ImageDraw.Draw(roi); d.rectangle([50,50,350,250], outline="blue", width=4)

    insp_info = MechanicalPositionInfo(
        janela_azul=(50, 50, 300, 200),
        similaridade=.97,
        inspecoes=[
            InspectionWindowInfo((60,60),(80,60),128,'preto',.9),
            InspectionWindowInfo((200,80),(60,60),100,'branco',.8)
        ]
    )
    base.save_blue_reference("regiao_1", "posicao_mecanica_A", "w1",
                           roi_pil=roi, meta=insp_info)

    ses = base.new_log_session("L123")
    base.append_json_log(ses, {"evento": "start", "t": datetime.now().isoformat()})
    base.save_snapshot(ses, 1, roi)
    print("Estrutura criada em:", base.model_dir)

```

File: roi_window_editor.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/teste_clp

```

# roi_window_editor.py – v3 (22/Jul/2025)
#
# Novidades v3 – Janelas-dependentes (“filhos”)
# -----
# • Qualquer janela pode ser marcada como PAI (tipicamente a posição mecânica)
# • add_window(..., parent=item_pai) → cria filho
# • editor.focus_on_parent(item|uid|None) → mostra só os filhos do pai ativo
# • serialização inclui:

```

```
# { "id": <int>, "parent": <int|null>, ... }
# • Métodos utilitários:
#     editor.current_parent()    → item ou None
#     editor.all_children(parent) → lista[ResizableRectItem]
#
# Compatível com v2 – APIs antigas não mudaram.
# -----
```

```
from __future__ import annotations
import json, os, itertools
from typing import Callable, Dict, Optional, List, Any, Union
from PyQt6.QtCore import Qt, QRectF, QPointF, QObject, QEvent, pyqtSignal
from PyQt6.QtGui import QPen, QBrush
from PyQt6.QtWidgets import (
    QGraphicsRectItem, QGraphicsItem, QGraphicsView, QGraphicsScene,
    QInputDialog
)
```

```
HANDLE = 6 # px
```

```
# =====
# Item individual (janela)
# =====
```

```
class ResizableRectItem(QGraphicsRectItem):
```

```
    """
```

```
    Retângulo arrastável/redimensionável.
```

```
    Novo v3:
```

- id → identificador único inteiro
- parent_uid → id do item-pai (ou None)
- is_child() → True se pertence a um pai

```
    """
```

```
    _id_counter = itertools.count(1) # gerador global
```

```
    pen_normal = QPen(Qt.GlobalColor.green, 2)
```

```
    pen_selected = QPen(Qt.GlobalColor.blue, 2, Qt.PenStyle.DashLine)
```

```
    handle_brush = QBrush(Qt.GlobalColor.white)
```

```
    handle_pen = QPen(Qt.GlobalColor.black, 1)
```

```
    def __init__(
```

```
        self,
```

```
        x: float, y: float, w: float, h: float,
```

```
        *,
```

```
        pen: Optional[QPen] = None,
```

```
        change_callback: Optional[Callable[['ResizableRectItem'], None]] = None,
```

```
        deletable: bool = True,
```

```
        name: Optional[str] = None,
```

```
        meta: Optional[Dict[str, Any]] = None,
```

```
        parent_uid: Optional[int] = None,
```

```
        parent: Optional[QGraphicsItem] = None,
```

```

    editable: bool = True          # NOVO: janelas "somente-seleção" no visor principal
):
    super().__init__(x, y, w, h, parent)
    self.editable: bool = editable
    self.id: int = next(self._id_counter)
    self.parent_uid: Optional[int] = parent_uid
    # -----
    # ItemsMovable + redimensionamento só se editable=True
    flags = (
        QGraphicsItem.GraphicsItemFlag.ItemsSelectable |
        QGraphicsItem.GraphicsItemFlag.ItemSendsGeometryChanges
    )
    if self.editable:
        flags |= QGraphicsItem.GraphicsItemFlag.ItemsMovable
    self.setFlags(flags)
    self.setAcceptHoverEvents(self.editable)
    self._active_handle: Optional[str] = None
    self._pen_custom = pen
    self.change_callback = change_callback
    self.deletable = deletable
    # ----- metadados livres -----
    self.name: Optional[str] = name
    self.meta: Dict[str, Any] = meta or {}
    if self.name:
        self.setToolTip(self.name)
    # callback inicial
    if callable(self.change_callback):
        self.change_callback(self)

# -----
# Conveniências
# -----
def is_child(self) -> bool:
    return self.parent_uid is not None

# -----
# Helpers internos (redimensionamento)
# -----
def _all_handles(self) -> Dict[str, QRectF]:
    r = self.rect()
    x, y, w, h = r.x(), r.y(), r.width(), r.height()
    s = HANDLE
    cx, cy = x + w/2.0, y + h/2.0
    return {
        'tl': QRectF(x - s/2, y - s/2, s, s),
        'tm': QRectF(cx - s/2, y - s/2, s, s),
        'tr': QRectF(x + w - s/2, y - s/2, s, s),
        'mr': QRectF(x + w - s/2, cy - s/2, s, s),
        'br': QRectF(x + w - s/2, y + h - s/2, s, s),
        'bm': QRectF(cx - s/2, y + h - s/2, s, s),
        'bl': QRectF(x - s/2, y + h - s/2, s, s),
    }

```

```
        'ml': QRectF(x - s/2, cy - s/2, s, s),
    }
}
```

```
@staticmethod
```

```
def _cursor_for(handle: Optional[str]) -> Qt.CursorShape:
```

```
    return {
        'tl': Qt.CursorShape.SizeFDiagCursor,
        'br': Qt.CursorShape.SizeFDiagCursor,
        'tr': Qt.CursorShape.SizeBDiagCursor,
        'bl': Qt.CursorShape.SizeBDiagCursor,
        'ml': Qt.CursorShape.SizeHorCursor,
        'mr': Qt.CursorShape.SizeHorCursor,
        'tm': Qt.CursorShape.SizeVerCursor,
        'bm': Qt.CursorShape.SizeVerCursor,
    }.get(handle, Qt.CursorShape.ArrowCursor)
```

```
def _handle_at(self, pos: QPointF) -> Optional[str]:
```

```
    for name, rect in self._all_handles().items():
        if rect.contains(pos):
            return name
    return None
```

```
# ----- eventos -----
```

```
def hoverMoveEvent(self, ev):
```

```
    if self.editable:
        self.setCursor(self._cursor_for(self._handle_at(ev.pos())))
    super().hoverMoveEvent(ev)
```

```
def mousePressEvent(self, ev):
```

```
    if self.editable and ev.button() == Qt.MouseButton.LeftButton:
        self._active_handle = self._handle_at(ev.pos())
    super().mousePressEvent(ev)
```

```
def mouseMoveEvent(self, ev):
```

```
    if self.editable and self._active_handle:
        self._resize(ev.pos())
        if callable(self.change_callback):
            self.change_callback(self)
    else:
        super().mouseMoveEvent(ev)
```

```
def mouseReleaseEvent(self, ev):
```

```
    self._active_handle = None
    if callable(self.change_callback):
        self.change_callback(self)
    super().mouseReleaseEvent(ev)
```

```
# ----- mudanças de posição -----
```

```
def itemChange(self, change, value):
```

```
    if (change == QGraphicsItem.GraphicsItemChange.ItemPositionHasChanged
        and callable(self.change_callback)):
```

```
        self.change_callback(self)
    return super().itemChange(change, value)
```

```
# ----- redimensionamento -----
```

```
def _resize(self, pos: QPointF):
```

```
    r = self.rect()
    x, y, w, h = r.x(), r.y(), r.width(), r.height()
    px, py = pos.x(), pos.y()
```

```
    if 'l' in self._active_handle:
        new_x = min(px, x + w - 1)
        w += x - new_x
        x = new_x
```

```
    if 'r' in self._active_handle:
        w = max(1, px - x)
```

```
    if 't' in self._active_handle:
        new_y = min(py, y + h - 1)
        h += y - new_y
        y = new_y
```

```
    if 'b' in self._active_handle:
        h = max(1, py - y)
```

```
    self.prepareGeometryChange()
    self.setRect(QRectF(x, y, w, h))
```

```
# ----- desenho -----
```

```
def paint(self, painter, option, widget=None):
```

```
    pen = self._pen_custom or self.pen_normal
    if self.isSelected():
```

```
        pen = self.pen_selected
    painter.setPen(pen)
    painter.drawRect(self.rect())
```

```
# Alças de redimensionamento apenas se editável
```

```
if self.isSelected() and self.editable:
    painter.setBrush(self.handle_brush)
    painter.setPen(self.handle_pen)
    for rect in self._all_handles().values():
        painter.drawRect(rect)
```

```
# ----- serialização -----
```

```
def to_dict(self) -> Dict[str, Any]:
```

```
    r = self.rect()
    return dict(
        id=self.id,
        parent=self.parent_uid,
        x=r.x(), y=r.y(), w=r.width(), h=r.height(),
        name=self.name,
        meta=self.meta
    )
```

```

# =====
# Editor / Gerenciador
# =====
class ROIWindowEditor(QObject):
    """
    Gerenciador universal de janelas redimensionáveis.

    Novidades v3 (janelas-filhas):
    -----
    • add_window(..., parent=None) → cria dependência hierárquica.
    • focus_on_parent(pai|uid|None)
      – None → mostra todas as janelas
      – ResizableItem → mostra apenas seus filhos
    • current_parent() → retorna pai ativo
    • all_children(pai) → lista de filhos
    """

    windowAdded = pyqtSignal(object) # ResizableRectItem
    windowRemoved = pyqtSignal(object)
    windowChanged = pyqtSignal(object)

    # ----- init -----
    def __init__(self,
        view: QGraphicsView,
        *,
        ask_name: bool = False,
        autosave_path: Optional[str] = None):

        super().__init__(view)
        self.view = view
        self._ask_name = ask_name
        self.autosave_path = autosave_path
        if self.view.scene() is None:
            self.view.setScene(QGraphicsScene(self.view))

        self._drawing = False
        self._drawing_parent: Optional[ResizableRectItem] = None
        self._tmp_rect: Optional[ResizableRectItem] = None
        self._start_pos: QPointF | None = None
        self.windows: List[ResizableRectItem] = []

        # parent currently focused (None = todos visíveis)
        self._parent_focus_uid: Optional[int] = None

        # event filter (viewport para mouse, scene para teclado)
        view.viewport().installEventFilter(self)
        self.view.scene().installEventFilter(self)

    # ----- API PÚBLICA -----
    # ----- modo desenho -----
    def start_drawing(self, *,
        pen: QPen = QPen(Qt.GlobalColor.red, 2),

```

```

        parent: Optional[ResizableRectItem] = None):
        """Habilita desenho; janelas receberão `parent` (opcional)."""
        self._drawing_pen = pen
        self._drawing_parent = parent
        self._drawing = True
        self.view.setCursor(Qt.CursorShape.CrossCursor)

def stop_drawing(self):
    self._drawing = False
    self._drawing_parent = None
    self.view.setCursor(Qt.CursorShape.ArrowCursor)
    if self._tmp_rect and self._tmp_rect.scene():
        self.view.scene().removeItem(self._tmp_rect)
    self._tmp_rect = None

# ---- criação programática -----
def add_window(self,
    x: float, y: float, w: float, h: float,
    *,
    pen: Optional[QPen] = None,
    deletable: bool = True,
    name: Optional[str] = None,
    meta: Optional[Dict[str, Any]] = None,
    change_cb: Optional[Callable[[ResizableRectItem], None]] = None,
    parent: Optional[ResizableRectItem] = None,
    editable: bool = True      # NOVO
) -> ResizableRectItem:

    item_parent_uid = parent.id if parent else None
    item = ResizableRectItem(
        x, y, w, h,
        pen=pen,
        deletable=deletable,
        name=name,
        meta=meta,
        parent_uid=item_parent_uid,
        parent=parent,          # hierarquia Qt
        change_callback=self._wrap_change_cb(change_cb),
        editable=editable
    )
    # CORREÇÃO: Sempre adiciona à cena se não há pai Qt Graphics
    # (mesmo que tenha parent_uid para associação lógica)
    if parent is None:
        self.view.scene().addItem(item)
    # Se tem pai Qt Graphics, não precisa addItem (já vinculado automaticamente)
    self.windows.append(item)
    self.windowAdded.emit(item)
    self._autosave()
    return item

# ----- foco / filtragem -----

```

```

def focus_on_parent(self,
    parent: Optional[Union[ResizableRectItem, int]]):
    """
    Define a janela-pai "ativa".
    • None → todas as janelas visíveis
    • item / id → exibe APENAS os filhos desse pai
    Janelas sem pai (top-level) permanecem sempre visíveis.
    """

    if isinstance(parent, ResizableRectItem):
        uid = parent.id
    else:
        uid = parent
    self._parent_focus_uid = uid
    # PROTEÇÃO: Remove itens inválidos da lista antes de usar
    valid_windows = []
    # DEBUGGING: Log da operação de foco
    if uid is None:
        print(f"■ Focando em: TODOS os itens")
    else:
        print(f"■ Focando em pai UID: {uid}")
    for w in self.windows:
        try:
            # Testa se objeto Qt ainda existe
            _ = w.scene()
            valid_windows.append(w)
            # Se chegou aqui, objeto é válido - aplica visibilidade
            if w.parent_uid is None:
                w.setVisible(True)      # sempre visível
                print(f" ■ Item sem pai sempre visível: {getattr(w, 'name', w.id)}")
            else:
                should_be_visible = (w.parent_uid == uid)
                w.setVisible(should_be_visible)
                visibility_status = "VISÍVEL" if should_be_visible else "OCULTO"
                print(f" {'■' if should_be_visible else '■'} Item {getattr(w, 'name', w.id)} (pai: {w.parent_uid}): {visibility_status}")
        except RuntimeError:
            print(f" ■■ Item inválido removido da lista")
            # Objeto foi deletado - ignora silenciosamente
            pass
    self.windows = valid_windows

def show_all_comparison_windows(self):
    """
    Mostra todas as janelas de comparação (que têm parent_uid),
    mantendo posições mecânicas (sem parent_uid) sempre visíveis.
    """
    for w in self.windows:
        # Posições mecânicas (sem pai) sempre visíveis
        # Janelas de comparação (com pai) sempre visíveis também
        w.setVisible(True)

def current_parent(self) -> Optional[ResizableRectItem]:

```

```
"""Retorna o item-pai atualmente em foco (ou None)."""
```

```
if self._parent_focus_uid is None:
```

```
    return None
```

```
for w in self.windows:
```

```
    if w.id == self._parent_focus_uid:
```

```
        return w
```

```
return None
```

```
def all_children(self, parent: ResizableRectItem) -> List[ResizableRectItem]:
```

```
    """Lista todos os filhos imediatos do item informado."""
```

```
    return [w for w in self.windows if w.parent_uid == parent.id]
```

```
def get_window_by_id(self, window_id: int) -> Optional[ResizableRectItem]:
```

```
    """Retorna janela pelo ID"""
```

```
for w in self.windows:
```

```
    if hasattr(w, 'id') and w.id == window_id:
```

```
        return w
```

```
return None
```

```
# ----- EVENT FILTER -----
```

```
def eventFilter(self, obj, ev):
```

```
    # ---- tecla Delete (escopo da cena) -----
```

```
    if ev.type() == QEvent.Type.KeyPress and ev.key() == Qt.Key.Key_Delete:
```

```
        self._delete_selected()
```

```
        return True
```

```
# ---- desenho (viewport) -----
```

```
if obj is self.view.viewport() and self._drawing:
```

```
    from PyQt6.QtCore import QPoint
```

```
    if ev.type() == QEvent.Type.MouseButtonPress and ev.button() == Qt.MouseButton.LeftButton:
```

```
        self._start_pos = self.view.mapToScene(QPoint(int(ev.position().x()),
                                                         int(ev.position().y())))
```

```
        self._tmp_rect = ResizableRectItem(
```

```
            self._start_pos.x(), self._start_pos.y(), 0, 0,
```

```
            pen=self._drawing_pen, deletable=True,
```

```
            parent=self._drawing_parent, # pai Qt Graphics (pode ser None)
```

```
            parent_uid=self._drawing_parent.id if self._drawing_parent else None
```

```
        )
```

```
        # CORREÇÃO: Sempre adiciona à cena durante desenho
```

```
        if self._drawing_parent is None:
```

```
            self.view.scene().addItem(self._tmp_rect)
```

```
        else:
```

```
            # Mesmo com pai lógico, se pai Qt Graphics for de outra cena, adiciona à cena atual
```

```
            if self._drawing_parent.scene() != self.view.scene():
```

```
                self.view.scene().addItem(self._tmp_rect)
```

```
                self._tmp_rect.parent = None # Remove pai Qt Graphics para evitar conflitos
```

```
        self.windows.append(self._tmp_rect)
```

```
        return True
```

```
if ev.type() == QEvent.Type.MouseMove and self._tmp_rect and self._start_pos:
```

```
    cur = self.view.mapToScene(QPoint(int(ev.position().x()),
```

```

        int(ev.position().y()))
x = min(self._start_pos.x(), cur.x())
y = min(self._start_pos.y(), cur.y())
w = abs(cur.x() - self._start_pos.x())
h = abs(cur.y() - self._start_pos.y())
self._tmp_rect.setRect(x, y, w, h)
return True

```

```

if ev.type() == QEvent.Type.MouseButtonRelease and self._tmp_rect and self._start_pos:
    valid = (self._tmp_rect.rect().width() > 2
            and self._tmp_rect.rect().height() > 2)

```

```

# nome opcional somente para janelas TOP-LEVEL
if valid and self._ask_name and self._drawing_parent is None:
    name, ok = QInputDialog.getText(
        self.view, "Nome da Janela", "Digite um nome (opcional):"
    )
    if ok:
        name = name.strip()
        if name:
            self._tmp_rect.name = name
            self._tmp_rect.setToolTip(name)

```

```

if not valid:
    # descarta
    if self._tmp_rect in self.windows:
        self.windows.remove(self._tmp_rect)
    if self._tmp_rect.scene():
        self._tmp_rect.scene().removeItem(self._tmp_rect)
else:
    # finaliza
    self._tmp_rect.change_callback = self._wrap_change_cb(
        self._tmp_rect.change_callback)
    self.windowAdded.emit(self._tmp_rect)
    self._autosave()
    self._tmp_rect = None
    self._start_pos = None
    if not self._drawing:
        self.view.setCursor(Qt.CursorShape.ArrowCursor)
    return True
return False

```

----- SERIALIZAÇÃO -----

```

def to_dict_list(self) -> List[Dict[str, Any]]:
    return [w.to_dict() for w in self.windows]

```

```

def from_dict_list(self, data: List[Dict[str, Any]], *, clear=True):

```

```

    """

```

Restaura hierarquia em 2 passagens para garantir que
itens-pai existam antes dos filhos.

```

    """

```

```

if clear:
    self._clear_all()
# 1ª passagem – cria todos os TOP LEVEL
id_map: Dict[int, ResizableRectItem] = {}
for d in data:
    if d.get("parent") is not None:
        continue
    it = self.add_window(
        d["x"], d["y"], d["w"], d["h"],
        name=d.get("name"), meta=d.get("meta", {}),
        parent=None
    )
    it.id = d["id"]          # preserva UID
    id_map[it.id] = it

```

```

# 2ª passagem – cria filhos
for d in data:
    if d.get("parent") is None:
        continue
    parent_it = id_map.get(d["parent"])
    if parent_it is None:      # órfão → ignora
        continue
    it = self.add_window(
        d["x"], d["y"], d["w"], d["h"],
        name=d.get("name"), meta=d.get("meta", {}),
        parent=parent_it
    )
    it.id = d["id"]
    id_map[it.id] = it

```

```

self.view.viewport().update()
self._autosave()

```

```

def save_json(self, path: str):
    os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
    with open(path, "w", encoding="utf-8") as f:
        json.dump(self.to_dict_list(), f, indent=2)

```

```

def load_json(self, path: str, *, clear=True):
    if not os.path.isfile(path):
        return False
    try:
        with open(path, "r", encoding="utf-8") as f:
            data = json.load(f)
        if isinstance(data, list):
            self.from_dict_list(data, clear=clear)
            return True
    except Exception:
        pass
    return False

```

```

# ----- INTERNOS -----
def _wrap_change_cb(self, user_cb):
    """
    Garante callback + autosave + sinal.
    """
    def _cb(item):
        if callable(user_cb):
            user_cb(item)
        self.windowChanged.emit(item)
        self._autosave()
    return _cb

def _autosave(self):
    if self.autosave_path:
        try:
            self.save_json(self.autosave_path)
        except Exception:
            pass

def _delete_selected(self):
    scene = self.view.scene()
    for it in scene.selectedItems():
        if isinstance(it, ResizableRectItem) and it.deletable:
            # remove também os FILHOS ligados a este item
            for child in list(self.all_children(it)):
                if child.scene():
                    child.scene().removeItem(child)
            if child in self.windows:
                self.windows.remove(child)
            self.windowRemoved.emit(child)
        scene.removeItem(it)
    if it in self.windows:
        self.windows.remove(it)
    self.windowRemoved.emit(it)
    scene.clearSelection()
    self._autosave()

def _clear_all(self):
    scene = self.view.scene()
    for w in list(self.windows):
        if w.scene() is scene:
            scene.removeItem(w)
    self.windows.clear()

```

File: selectable_rect_item.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/teste_clp

```

# selectable_rect_item.py
from PyQt6.QtWidgets import QGraphicsRectItem, QGraphicsItem

```

```

from PyQt6.QtGui    import QPen, QBrush, QColor, QCursor
from PyQt6.QtCore    import Qt, QRectF, QPointF

```

```

HANDLE_SIZE = 6      # px
HANDLE_BRUSH = QBrush(QColor(255, 255, 255))
HANDLE_PEN   = QPen(Qt.GlobalColor.black, 1)

```

```

class SelectableResizableRectItem(QGraphicsRectItem):

```

```

    """

```

- Seleccionável (clique) – borda fica tracejada
- Arrastável – ItemIsMovable
- Redimensionável – 8 alças nos cantos/lados

```

    """

```

```

def __init__(self, x, y, w, h, pen=None, parent=None,
              change_callback=None, deletavel: bool = True):
    super().__init__(x, y, w, h, parent)
    self.setFlags(
        QGraphicsItem.GraphicsItemFlag.ItemIsSelectable |
        QGraphicsItem.GraphicsItemFlag.ItemIsMovable |
        QGraphicsItem.GraphicsItemFlag.ItemSendsGeometryChanges
    )
    self.setAcceptHoverEvents(True)
    self.pen_default = pen or QPen(Qt.GlobalColor.green, 2)
    self.pen_selected = QPen(Qt.GlobalColor.blue, 2, Qt.PenStyle.DashLine)
    self.setPen(self.pen_default)
    self.change_callback = change_callback
    # --- nova propriedade comum -----
    # True → pode ser removido com a tecla Delete
    # False → protegido contra deleção pelo atalho
    self.deletavel = deletavel

```

```

    # alça atualmente ativa (None se não está redimensionando)

```

```

    if callable(self.change_callback):
        self.change_callback(self)

```

```

# -----

```

```

# HELPERS DE ALÇAS

```

```

# -----

```

```

def _handles(self):

```

```

    """

```

```

    Retorna dicionário {nome: QRectF} em coordenadas da cena
    nomes = 'tl', 'tm', 'tr', 'mr', 'br', 'bm', 'bl', 'ml' (top-left etc.)

```

```

    """

```

```

    r = self.rect()
    x, y, w, h = r.x(), r.y(), r.width(), r.height()
    s = HANDLE_SIZE
    cx, cy = x + w/2, y + h/2
    return {
        'tl': QRectF(x-s/2, y-s/2, s, s),
        'tm': QRectF(cx-s/2, y-s/2, s, s),
        'tr': QRectF(x+w-s/2, y-s/2, s, s),

```

```

        'mr': QRectF(x+w-s/2, cy-s/2, s, s),
        'br': QRectF(x+w-s/2, y+h-s/2, s, s),
        'bm': QRectF(cx-s/2, y+h-s/2, s, s),
        'bl': QRectF(x-s/2, y+h-s/2, s, s),
        'ml': QRectF(x-s/2, cy-s/2, s, s)
    }

```

```

def _handle_at(self, pos: QPointF):
    for name, rect in self._handles().items():
        if rect.contains(pos):
            return name
    return None

```

```

# -----
# EVENTOS mouse / hover
# -----

```

```

def hoverMoveEvent(self, ev):
    handle = self._handle_at(ev.pos())
    cursors = {
        'tl': Qt.CursorShape.SizeFDiagCursor, 'br': Qt.CursorShape.SizeFDiagCursor,
        'tr': Qt.CursorShape.SizeBDiagCursor, 'bl': Qt.CursorShape.SizeBDiagCursor,
        'ml': Qt.CursorShape.SizeHorCursor, 'mr': Qt.CursorShape.SizeHorCursor,
        'tm': Qt.CursorShape.SizeVerCursor, 'bm': Qt.CursorShape.SizeVerCursor,
    }
    self.setCursor(QCursor(cursors.get(handle, Qt.CursorShape.ArrowCursor)))
    super().hoverMoveEvent(ev)

```

```

def mousePressEvent(self, ev):
    self._active_handle = self._handle_at(ev.pos())
    super().mousePressEvent(ev)

```

```

def mouseMoveEvent(self, ev):
    if self._active_handle:
        self._resize(ev.pos())
    else:
        super().mouseMoveEvent(ev)

```

```

def mouseReleaseEvent(self, ev):
    self._active_handle = None
    super().mouseReleaseEvent(ev)

```

```

def itemChange(self, change, value):
    if (change == QGraphicsItem.GraphicsItemChange.ItemPositionHasChanged and
        callable(self.change_callback)):
        self.change_callback(self)
    return super().itemChange(change, value)

```

```

# -----
def _resize(self, pos: QPointF):
    rect = self.rect()
    x, y, w, h = rect.x(), rect.y(), rect.width(), rect.height()

```

```

px, py = pos.x(), pos.y()

if 'l' in self._active_handle:
    new_x = min(px, x+w-1)
    w += x - new_x
    x = new_x
if 'r' in self._active_handle:
    w = max(1, px - x)
if 't' in self._active_handle:
    new_y = min(py, y+h-1)
    h += y - new_y
    y = new_y
if 'b' in self._active_handle:
    h = max(1, py - y)

self.prepareGeometryChange()
self.setRect(QRectF(x, y, w, h))
# notifica alteração durante o redimensionamento
if callable(self.change_callback):
    self.change_callback(self)

# -----
# DESENHO
# -----
def paint(self, painter, option, widget=None):
    # borda
    painter.setPen(self.pen_selected if self.isSelected() else self.pen_default)
    super().paint(painter, option, widget)
    # alças
    if self.isSelected():
        painter.setBrush(HANDLE_BRUSH)
        painter.setPen(HANDLE_PEN)
        for rect in self._handles().values():
            painter.drawRect(rect)

```

File: dot_patterns.json

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/teste_clp

```

[
  {
    "id": 1,
    "name": "memoria",
    "qty": 50
  },
  {
    "id": 2,
    "name": "processador",
    "qty": 100
  },

```

```
{
    "id": 3,
    "name": "chipset",
    "qty": 75
},
{
    "id": 4,
    "name": "diodo",
    "qty": 15
}
]
```

File: testes_comunicacao_e_comandos.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/teste_clp

```
import sys
import time
import json
from pathlib import Path
from virtual_home_dialog import ConfigHomeDialog
from axis_calibration_dialog import AxisCalibrationDialog
from settings_manager import SettingsManager
from camera_manager import CameraManager
from alignment_dialog import AlignmentDialog
# widgets externos importados
from table_program_tab import TableProgramTab
from axes_control_tab import AxesControlTab
from movement_controls_widget import MovementControlsWidget
from position_status_widget import PositionStatusWidget
from inspection_positions_widget import InspectionPositionsWidget
from program_io_widget import ProgramIOWidget
from positions_backend import InspectionPositionsBackend
from sequence_control import (
    SequenceControlWidget, InspectionPosition, MotionBackend
)

from PyQt6.QtWidgets import (QApplication, QMainWindow, QWidget, QVBoxLayout,
                             QHBoxLayout, QLabel, QSpinBox, QPushButton,
                             QGroupBox, QGridLayout, QTabWidget, QTextEdit,
                             QFrame, QSizePolicy, QCheckBox, QScrollArea,
                             QMessageBox)
from PyQt6.QtCore import Qt, QTimer, pyqtSignal, QObject, QThread
from PyQt6.QtGui import QFont
from pymodbus.client import ModbusTcpClient

class MultiAxisMotorController(QMainWindow):
    # -----
    # S I G N A L (garante acesso aos widgets só no thread GUI)
    # -----
    logRequested = pyqtSignal(str) # emitido por qualquer thread
```

```

def __init__(self):
    super().__init__()
    self.client = None
    self.connected = False

# Endereços Modbus CORRIGIDOS
self.addresses = {
    # Memórias M - Endereços corrigidos conforme ladder real
    # EIXO Y2
    'M0_Y2': 0,    # ATRIBUI ZERO A POSIÇÃO ATUAL (Y2)
    'M50_Y2': 50,  # INICIA O MOVIMENTO ABSOLUTO (Y2)
    'M100_Y2': 100, # ACIONA LÓGICA DE COMPARAÇÃO (Y2)
    # EIXO Y1
    'M500_Y1': 500, # ATRIBUI ZERO A POSIÇÃO ATUAL (Y1)
    'M550_Y1': 550, # INICIA O MOVIMENTO ABSOLUTO (Y1)
    'M600_Y1': 600, # ACIONA LÓGICA DE COMPARAÇÃO (Y1)
    # EIXO Z
    'M1500_Z': 1500, # ATRIBUI ZERO A POSIÇÃO ATUAL (Z)
    'M1550_Z': 1550, # INICIA O MOVIMENTO ABSOLUTO (Z)
    'M1600_Z': 1600, # ACIONA LÓGICA DE COMPARAÇÃO (Z)
    # EIXO X
    'M1000_X': 1000, # ATRIBUI ZERO A POSIÇÃO ATUAL (X)
    'M1050_X': 1050, # INICIA O MOVIMENTO ABSOLUTO (X)
    'M1100_X': 1100, # ACIONA LÓGICA DE COMPARAÇÃO (X)
    # APLICADORA DE ADESIVO
    'M5000': 5000,  # Trigger aplicadora de adesivo
    'M5001': 5001,  # Auxiliar aplicadora

    # HOMING - Busca pelo HOME
    'M350': 350,    # GO TO HOME Y2
    'M850': 850,    # GO TO HOME Y1
    'M1350': 1350,  # GO TO HOME X
    'M1850': 1850,  # GO TO HOME Z

    # HOMING REALIZADO - Status do homing
    'M300': 300,    # HOMING REALIZADO Y2
    'M800': 800,    # HOMING REALIZADO Y1
    'M1300': 1300,  # HOMING REALIZADO X
    'M1800': 1800,  # HOMING REALIZADO Z

    # SISTEMA JOG TODOS OS EIXOS (conforme nova estrutura)
    # Y2
    'M70_Y2': 70,   # JOG POSITIVO Y2
    'M80_Y2': 80,   # JOG NEGATIVO Y2
    'M10_Y2': 10,   # INTERROMPE JOG LIMITE POSITIVO Y2
    'M11_Y2': 11,   # INTERROMPE JOG LIMITE NEGATIVO Y2
    # Y1
    'M570_Y1': 570, # JOG POSITIVO Y1
    'M580_Y1': 580, # JOG NEGATIVO Y1
    'M510_Y1': 510, # INTERROMPE JOG LIMITE POSITIVO Y1
    'M511_Y1': 511, # INTERROMPE JOG LIMITE NEGATIVO Y1

```

X

'M1070_X': 1070, # JOG POSITIVO X

'M1080_X': 1080, # JOG NEGATIVO X

'M1010_X': 1010, # INTERROMPE JOG LIMITE POSITIVO X

'M1011_X': 1011, # INTERROMPE JOG LIMITE NEGATIVO X

Z

'M1570_Z': 1570, # JOG POSITIVO Z

'M1580_Z': 1580, # JOG NEGATIVO Z

'M1510_Z': 1510, # INTERROMPE JOG LIMITE POSITIVO Z

'M1511_Z': 1511, # INTERROMPE JOG LIMITE NEGATIVO Z

MEMÓRIAS AUXILIARES HOMING (conforme ladder real)

Y2

'M311': 311, 'M312': 312, 'M313': 313, 'M314': 314, 'M315': 315,

'M360': 360,

Y1

'M811': 811, 'M812': 812, 'M813': 813, 'M814': 814, 'M815': 815,

'M860': 860,

X

'M1311': 1311, 'M1312': 1312, 'M1313': 1313, 'M1314': 1314, 'M1315': 1315,

'M1360': 1360,

Z

'M1811': 1811, 'M1812': 1812, 'M1813': 1813, 'M1814': 1814, 'M1815': 1815,

'M1860': 1860,

TEMPORIZADORES HOMING

'T0': 0, 'T1': 1, 'T2': 2, 'T3': 3, 'T4': 4, 'T5': 5, 'T6': 6, 'T7': 7,

VELOCIDADES JOG (nova estrutura D22000 série)

'D22000': 22000, # VELOCIDADE JOG POSITIVO Y2

'D22010': 22010, # VELOCIDADE JOG POSITIVO Y1

'D22020': 22020, # VELOCIDADE JOG POSITIVO X

'D22030': 22030, # VELOCIDADE JOG POSITIVO Z

'D22050': 22050, # VELOCIDADE JOG NEGATIVO Y2

'D22060': 22060, # VELOCIDADE JOG NEGATIVO Y1

'D22070': 22070, # VELOCIDADE JOG NEGATIVO X

'D22080': 22080, # VELOCIDADE JOG NEGATIVO Z

'D22100': 22100, # TEMPO DE ACELERAÇÃO JOG

'D22110': 22110, # TEMPO DE DESACELERAÇÃO JOG

LIMITES SALVOS EM ROM (novos conforme planilha)

'D23050': 23050, # LIMITE NEGATIVO Y2 (ROM)

'D23060': 23060, # LIMITE NEGATIVO Y1 (ROM)

'D23070': 23070, # LIMITE NEGATIVO X (ROM)

'D23080': 23080, # LIMITE NEGATIVO Z (ROM)

'D23090': 23090, # LIMITE POSITIVO Y2 (ROM)

'D23100': 23100, # LIMITE POSITIVO Y1 (ROM)

'D23110': 23110, # LIMITE POSITIVO X (ROM)

'D23120': 23120, # LIMITE POSITIVO Z (ROM)

VELOCIDADES HOMING (conforme planilha)

```

'D23000': 23000, # VELOCIDADE HOMING Y2 (ROM)
'D23010': 23010, # VELOCIDADE HOMING Y1 (ROM)
'D23020': 23020, # VELOCIDADE HOMING X (ROM)
'D23030': 23030, # VELOCIDADE HOMING Z (ROM)
'D450': 450, # VELOCIDADE HOMING Y2 (aplicada)
'D950': 950, # VELOCIDADE HOMING Y1 (aplicada)
'D1450': 1450, # VELOCIDADE HOMING X (aplicada)
'D1950': 1950, # VELOCIDADE HOMING Z (aplicada)

# Registradores D - Corrigidos conforme ladder real
# EIXO Y2
'D0_Y2': 0, # MOVE TO ABS (Y2)
'D50_Y2': 50, # LIMITE NEGATIVO Y2 (trabalho)
'D23050_Y2': 23050, # LIMITE NEGATIVO Y2 (ROM)
'D100_Y2': 100, # POSIÇÃO ABSOLUTA DEFINIDA PELO USUÁRIO (Y2) - ENTRADA
'D150_Y2': 150, # POSIÇÃO ABSOLUTA DEFINIDA (Y2)
'D20000_Y2': 20000, # VELOCIDADE DE DESLOCAMENTO (Y2)
# EIXO Y1 - NOVO
'D500_Y1': 500, # MOVE TO ABS (Y1)
'D550_Y1': 550, # LIMITE NEGATIVO Y1 (trabalho)
'D23060_Y1': 23060, # LIMITE NEGATIVO Y1 (ROM)
'D600_Y1': 600, # POSIÇÃO ABSOLUTA DEFINIDA PELO USUÁRIO (Y1) - ENTRADA
'D650_Y1': 650, # POSIÇÃO ABSOLUTA DEFINIDA (Y1)
'D20500_Y1': 20500, # VELOCIDADE DE DESLOCAMENTO (Y1)
# EIXO Z
'D1500_Z': 1500, # MOVE TO ABS (Z)
'D1550_Z': 1550, # LIMITE NEGATIVO Z (trabalho)
'D23080_Z': 23080, # LIMITE NEGATIVO Z (ROM)
'D1600_Z': 1600, # POSIÇÃO ABSOLUTA DEFINIDA PELO USUÁRIO (Z) - ENTRADA
'D1650_Z': 1650, # POSIÇÃO ABSOLUTA DEFINIDA (Z)
'D21500_Z': 21500, # VELOCIDADE DE DESLOCAMENTO (Z)
# EIXO X
'D1000_X': 1000, # MOVE TO ABS (X)
'D1050_X': 1050, # LIMITE NEGATIVO X (trabalho)
'D23070_X': 23070, # LIMITE NEGATIVO X (ROM)
'D1100_X': 1100, # POSIÇÃO ABSOLUTA DEFINIDA PELO USUÁRIO (X) - ENTRADA
'D1150_X': 1150, # POSIÇÃO ABSOLUTA DEFINIDA (X)
'D21000_X': 21000, # VELOCIDADE DE DESLOCAMENTO (X)

# ----- DOT CONTROL -----
# D-registradores para frequência e quantidade de aplicação
# (mudaram de D4000 / D4100 → D24000 / D24100 no ladder)
'D24000_FREQ': 24000, # Frequência (Hz) dos dots
'D24100_QTY': 24100, # Quantidade de dots

# Registradores de status (leitura)
'D3100_Y2': 3100, # POSIÇÃO ATUAL MOTOR Y2 (cópia do SR460)
'D3400_Y1': 3400, # POSIÇÃO ATUAL MOTOR Y1 (cópia do SR500)
'D3200_Z': 3200, # POSIÇÃO ATUAL MOTOR Z (cópia do SR480)
'D3000_X': 3000, # POSIÇÃO ATUAL MOTOR X (cópia do SR520)

```

```

# Saídas Y (coils) – endereços Modbus oficiais (AS-Series Manual, Tabela 7-3)
'Y00': 40960, 'Y01': 40961,    # Y0.0 / Y0.1 eixo-Y2
'Y02': 40962, 'Y03': 40963,    # Y0.2 / Y0.3 eixo-Z
'Y04': 40964, 'Y05': 40965,    # Y0.4 / Y0.5 eixo-Y1
'Y06': 40966, 'Y07': 40967,    # Y0.6 / Y0.7 eixo-X
'Y010': 40970,                # Y0.10 (0xA00A) aplicadora de adesivo <<< FIX

# Entradas X (sensores)
'X04_Y2': 8196,    # X0.4 sensor homing Y2
'X06_Y1': 8198,    # X0.6 sensor homing Y1
'X08_X': 8200,     # X0.8 sensor homing X
'X09_Z': 8201,     # X0.9 sensor homing Z
}

# Posição atual dos eixos
self.current_positions = {
    'Y2': 0,
    'Y1': 0,
    'Z': 0,
    'X': 0
}

# ----- CONTEXTO DO PROJETO -----
self.prog_mgr      = None # AdhesiveProgramManager ou None
self.current_proj_name = None # string ou None

# -----
# SPINBOX “FANTASMA” PARA CADA EIXO (necessário para o backend PLC)
# -----
from PyQt6.QtWidgets import QSpinBox
for _axis in ('Y2', 'Y1', 'X', 'Z'):
    if not hasattr(self, f'pulsos_spin_{_axis}'):
        sb = QSpinBox()
        sb.setRange(-2_147_483_648, 2_147_483_647)
        sb.setValue(0)
        setattr(self, f'pulsos_spin_{_axis}', sb)

# ----- NOVO: lista de registradores configuráveis -----
# (qualquer outro pode ser acrescentado facilmente depois)
self.configurable_registers = [
    # ---- LIMITES SALVOS EM ROM ----
    ('D23050', 'Limite- Y2 (ROM)'), ('D23090', 'Limite+ Y2 (ROM)'),
    ('D23060', 'Limite- Y1 (ROM)'), ('D23100', 'Limite+ Y1 (ROM)'),
    ('D23070', 'Limite- X (ROM)'), ('D23110', 'Limite+ X (ROM)'),
    ('D23080', 'Limite- Z (ROM)'), ('D23120', 'Limite+ Z (ROM)'),
    # ---- VELOCIDADE DE DESLOCAMENTO ----
    ('D20000_Y2', 'Vel. Desloc. Y2'),
    ('D20500_Y1', 'Vel. Desloc. Y1'),
    ('D21000_X', 'Vel. Desloc. X'),
    ('D21500_Z', 'Vel. Desloc. Z'),
    # ---- VELOCIDADE DE HOMING (ROM) ----
    ('D23000', 'Vel. Homing Y2'),

```

```

('D23010', 'Vel. Homing Y1'),
('D23020', 'Vel. Homing X'),
('D23030', 'Vel. Homing Z'),
# ---- VELOCIDADE JOG ----
('D22000', 'Jog+ Y2'), ('D22050', 'Jog- Y2'),
('D22010', 'Jog+ Y1'), ('D22060', 'Jog- Y1'),
('D22020', 'Jog+ X'), ('D22070', 'Jog- X'),
('D22030', 'Jog+ Z'), ('D22080', 'Jog- Z'),
('D22100', 'Tempo Acel. Jog'),
('D22110', 'Tempo Desac. Jog'),
# ----- DOT -----
('D24000_FREQ', 'Frequência dot (Hz)'),
('D24100_QTY', 'Quantidade dot')
]

# ----- limites de trabalho (padrão) + persistência ---
# ----- carrega configurações persistentes -----
self.settings = SettingsManager()
self.table_limits = self.settings.table_limits

# -----
# OFFSET CÂMERA ↔ NOZZLE deve existir ANTES do backend PLC
# -----
self.camera_nozzle_offset = self.settings.cam_noz_offset

# -----
# Diretório-base dos projetos (...\\Projetos\\modelos\\...)
# -----
self.projects_dir = self.settings.projects_dir
self.projects_dir.mkdir(parents=True, exist_ok=True)

# ----- câmera -----
self.camera_manager = CameraManager(self.settings.camera_index)

# aplica calibração se existir
a,b = self.settings.focus_coeffs
self.camera_manager.load_calibration(a,b)

# backend de movimento criado uma única vez e reutilizado
self._plc_motion_backend = PLCMotionBackend(self)
# ----- STEPS/MM -----
self.steps_per_mm: dict[str,float] = self.settings.axis_steps
# ----- escala da câmera (mm/pixel) -----
self.mm_per_pixel: float = self.settings.camera_mm_per_pixel

# ----- campo-de-visão (dois pontos) -----
self.camera_fov: dict = self.settings.camera_fov
# coeficientes lineares largura = aX*z + bX (idem altura)
self._fov_coeffs = self._calc_fov_coeffs()

# ---- homing virtual -----

```

```

self.virtual_home: dict[int,dict[str,int]] = self.settings.virtual_home

self.init_ui()
# Ao iniciar, criação/edição de programa fica BLOQUEADA
QTimer.singleShot(0, lambda: self._set_creation_controls_enabled(False))
self.connect_plc()

# Lê velocidades salvas na ROM após conectar
QTimer.singleShot(1500, self.read_initial_jog_velocities)
QTimer.singleShot(2500, self.read_initial_limits)
QTimer.singleShot(3000, self.read_current_positions)
QTimer.singleShot(3500, self.read_initial_homing_status)

# Controle de teclas do teclado
self.keyboard_jog_active = {
    'X': False,
    'Y2': False,
    'Y1': False,
    'Z': False
} # Flags para controlar se JOG está ativo via teclado para cada eixo
self.setFocusPolicy(Qt.FocusPolicy.StrongFocus) # Permite capturar teclas

# Timer para atualização
# ----- logger seguro -----
self.logRequested.connect(self._append_log) # slot GUI

self.timer = QTimer()
self.timer.timeout.connect(self.update_status)
self.timer.start(500)

# ===== LOG thread-safe =====
def log(self, msg: str):
    """
    Pode ser chamado de QUALQUER thread.
    No thread GUI a mensagem é escrita imediatamente;
    nos demais threads emite-se logRequested.
    """
    ts = time.strftime("%H:%M:%S")
    full = f"[{ts}] {msg}"
    print(full, flush=True)
    if QThread.currentThread() is self.thread(): # GUI?
        self._append_log(full)
    else:
        self.logRequested.emit(full)

def _append_log(self, text: str):
    """Slot executado SEMPRE no thread GUI"""
    if not hasattr(self, "log_text"):
        return
    self.log_text.append(text)
    # mantém no máx. 100 linhas

```

```

if self.log_text.document().lineCount() > 100:
    cursor = self.log_text.textCursor()
    cursor.movePosition(cursor.MoveOperation.Start)
    cursor.select(cursor.SelectionType.LineUnderCursor)
    cursor.removeSelectedText()

```

```

def init_ui(self):

```

```

    self.setWindowTitle("Controle Multi-Eixos - Delta AS (ENDEREÇOS LADDER REAIS)")
    self.setMinimumSize(900, 700)

```

```

    central_widget = QWidget()
    self.setCentralWidget(central_widget)
    layout = QVBoxLayout(central_widget)

```

```

# ----- MENU -----

```

```

menubar = self.menuBar()
# ----- MENU "ARQUIVO" -----
menu_file = menubar.addMenu("Arquivo")
act_new = menu_file.addAction("Novo Projeto")
act_open = menu_file.addAction("Abrir Projeto...")
act_save = menu_file.addAction("Salvar Projeto")
act_saveas = menu_file.addAction("Salvar Como...")
menu_file.addSeparator()
act_exit = menu_file.addAction("Sair")

```

```

act_new.triggered.connect(self._new_project)
act_open.triggered.connect(self._open_project)
act_save.triggered.connect(self._save_project)
act_saveas.triggered.connect(self._save_as_project)
act_exit.triggered.connect(self.close)
menu_cfg = menubar.addMenu("Configurações")

```

```

# ===== NOVO ■ PREFERÊNCIAS =====

```

```

menu_pref = menubar.addMenu("Preferências")
act_setdir = menu_pref.addAction("Definir caminho Modelos...")
act_setdir.triggered.connect(self._set_models_path)
menu_prog = menubar.addMenu("Programa")
act_limits = menu_cfg.addAction("Limites de mesa...")
act_limits.triggered.connect(self._open_limits_dialog)

```

```

# ---- submenu câmera -----

```

```

menu_cam = menu_cfg.addMenu("Câmera")
act_select = menu_cam.addAction("Selecionar câmera...")
act_reopen = menu_cam.addAction("Reconectar")
act_focus = menu_cam.addAction("Calibrar foco...")
act_fov = menu_cam.addAction("Calibração campo de visão...")
act_select.triggered.connect(self._select_camera_dialog)
act_reopen.triggered.connect(lambda: self.camera_manager.open(self.settings.camera_index))
act_focus.triggered.connect(self._open_focus_dialog)
act_fov.triggered.connect(self._open_fov_dialog)

```

```

# ----- CALIBRAÇÃO EIXOS -----
act_calib = menu_cfg.addAction("Calibração dos Eixos...")
act_calib.triggered.connect(self._open_axis_calib_dialog)

# ----- HOMING VIRTUAL -----
act_homecfg = menu_cfg.addAction("Homing Virtual...")
act_homecfg.triggered.connect(self._open_virtual_home_dialog)

# ----- NOVO – ALINHAMENTO NOZZLE/CÂMERA -----
act_align = menu_cfg.addAction("Alinhamento Nozzle↔Câmera...")
act_align.triggered.connect(self._open_alignment_dialog)

# ----- DOT PATTERNS -----
act_dots = menu_prog.addAction("Padrões de Dots...")
act_dots.triggered.connect(self._open_dot_dialog)

# ----- BANNER DE CONEXÃO (rodapé) -----
# Usa o status-bar nativo do QMainWindow (aparece no rodapé).
self.status_bar = self.statusBar()      # QStatusBar
self.status_bar.setSizeGripEnabled(False) # oculta "grip" de redimensionar
self.status_bar.showMessage("Desconectado")
self.status_bar.setStyleSheet(
    "QStatusBar { background-color:red; color:white; font-weight:bold; }")

# Abas para organizar
# -----
# TAB WIDGET principal fica em self.tab_widget para acesso global
# -----
self.tab_widget = QTabWidget()

# === ABA 1: CONTROLE DOS EIXOS (principal) ===
self.control_tab = AxesControlTab(self, primary=True)
self.tab_widget.addTab(self.control_tab, "Controle de Eixos")

# === ABA 2: STATUS ===
status_tab = QWidget()
status_layout = QVBoxLayout(status_tab)

# Status das saídas
outputs_group = self.create_outputs_status()
status_layout.addWidget(outputs_group)

# Status das memórias
memories_group = self.create_memories_status()
status_layout.addWidget(memories_group)

# Sensores de HOMING (entradas X0.4 / X0.6 / X0.8 / X0.9)
sensors_group = self.create_homing_sensors_status()
status_layout.addWidget(sensors_group)

# Status dos registradores

```

```

registers_group = self.create_registers_status()
status_layout.addWidget(registers_group)

self.tab_widget.addTab(status_tab, "Status do Sistema")

# === ABA 3: CONFIGURAÇÃO DE REGISTRADORES (NOVA) ===
config_tab = self.create_config_tab()
self.tab_widget.addTab(config_tab, "Config. Registradores")

# === ABA 4: LOG ===
log_tab = QWidget()
log_layout = QVBoxLayout(log_tab)

log_label = QLabel("Log de Eventos:")
log_label.setFont(QFont("Arial", 10, QFont.Weight.Bold))
log_layout.addWidget(log_label)

self.log_text = QTextEdit()
self.log_text.setReadOnly(True)
self.log_text.setFont(QFont("Consolas", 9))
log_layout.addWidget(self.log_text)

clear_log_btn = QPushButton("Limpar Log")
clear_log_btn.clicked.connect(self.log_text.clear)
log_layout.addWidget(clear_log_btn)

self.tab_widget.addTab(log_tab, "Log")

# === ABA 5: MESA 1 =====
self.mesa_tabs = {}
mesa1_tab = TableProgramTab(self, mesa_id=1)
self.mesa_tabs[1] = mesa1_tab
self.tab_widget.addTab(mesa1_tab, "Mesa 1")

# === ABA 6: MESA 2 =====
mesa2_tab = TableProgramTab(self, mesa_id=2)
self.mesa_tabs[2] = mesa2_tab
self.tab_widget.addTab(mesa2_tab, "Mesa 2")

layout.addWidget(self.tab_widget)

# ----- rastreia arquivo atual do projeto -----
self._current_project_file = None
# ----- registra todos os ProgramIOWidgets -----
self._register_prog_widget(self.prog_io_widget)      # controle de eixos
self._register_prog_widget(mesa1_tab.prog_widget)
self._register_prog_widget(mesa2_tab.prog_widget)

# -----
# Helpers para ProgramIOWidget por aba
# -----

```

```

def _register_prog_widget(self, widget):
    """Acopla sinais de load/save para lembrar o último arquivo usado."""
    widget._last_file = None
    widget.fileSaved.connect(lambda f, w=widget: setattr(w, "_last_file", f))
    # mantém lista para futura atualização do default_dir
    if not hasattr(self, "_prog_widgets"):
        self._prog_widgets = []
    self._prog_widgets.append(widget)

# =====
#   ALINHAMENTO NOZZLE ↔ CÂMERA   (offset X,Y)
# =====

def _open_alignment_dialog(self):

    dlg = AlignmentDialog(self, self)      # não modal
    dlg.offsetSaved.connect(self._save_nozzle_offset)
    dlg.show()

def _save_nozzle_offset(self, off: dict[str, int]):
    """Salvo em memória e disco."""
    self.camera_nozzle_offset = off
    self.settings.cam_noz_offset = off
    self.settings.save()
    self.log(f"■ Offset câmera↔nozzle salvo: ΔX={off['x']} ΔY={off['y']}")

    # informa backend caso já exista
    if hasattr(self, "_plc_motion_backend"):
        self._plc_motion_backend._off_x = off.get("x", 0)
        self._plc_motion_backend._off_y = off.get("y", 0)

def _current_prog_widget(self):
    """Devolve o ProgramIOWidget associado à aba visível."""
    w = self.tab_widget.currentWidget()
    if w is self.control_tab:
        return self.prog_io_widget
    for mesa_id, tab in self.mesa_tabs.items():
        if w is tab:
            return tab.prog_widget
    # fallback
    return self.prog_io_widget

# =====
#   Conversões pulsos ↔ milímetros
# =====

def pulses_from_mm(self, axis: str, mm: float) -> int:
    """Converte mm → pulsos utilizando a calibração do eixo."""
    return int(round(mm * self.steps_per_mm.get(axis, 1.0)))

def _set_axis_steps(self, axis: str, steps: float):
    """Atualiza fator steps/mm de um eixo e persiste em settings."""
    self.steps_per_mm[axis] = float(steps)

```

```

self.settings.axis_steps = self.steps_per_mm
self.settings.save()
self.log(f"■ Calibração {axis}: {steps:.3f} steps/mm salva")

# =====
# HOMING VIRTUAL
# =====
def _open_virtual_home_dialog(self):
    # cria uma única instância reutilizável
    if not hasattr(self, '_vh_dialog'):
        from virtual_home_dialog import ConfigHomeDialog
        self._vh_dialog = ConfigHomeDialog(self, self)
        self._vh_dialog.acceptedAndSaved.connect(self._save_virtual_home)
    # mostra janela não modal, mantendo-a na frente
    self._vh_dialog.show()
    self._vh_dialog.raise_()
    self._vh_dialog.activateWindow()

# ----- DOTS -----
def _open_dot_dialog(self):
    if not hasattr(self, "_dot_dialog"):
        from dot_patterns_dialog import DotPatternsDialog
        self._dot_dialog = DotPatternsDialog(self)
    self._dot_dialog.show()
    self._dot_dialog.raise_(); self._dot_dialog.activateWindow()

def _save_virtual_home(self):
    self.virtual_home = self._vh_dialog.get_home_dict()
    self.settings.virtual_home = self.virtual_home
    self.settings.save()
    self.log("■ Homing virtual atualizado")

def goto_virtual_home(self, mesa: int):
    """Usado pelas abas Mesa 1/Mesa 2."""
    home = self.virtual_home.get(mesa, {"x":0,"y":0})
    x_tgt = home["x"]
    y_tgt = home["y"]
    # calcula diferenças
    dx = x_tgt - self.current_positions["X"]
    y_axis = 'Y1' if mesa == 1 else 'Y2'
    dy = y_tgt - self.current_positions[y_axis]
    if abs(dx) > 0:
        self.move_relative('X', dx)
    if abs(dy) > 0:
        self.move_relative(y_axis, dy)
    self.log(f"■ Mesa {mesa}: cabeça movida para Home virtual")

# =====
# AÇÕES DO MENU "ARQUIVO"
# =====
def _new_project(self):

```

"""

Inicia um NOVO programa.

Passos:

1. usuário já está na aba Mesa 1 ou Mesa 2;
2. pergunta nome do programa;
3. cria imediatamente modelos/<nome>/... (estrutura base);
4. limpa listas e libera botões de criação.

"""

```
from PyQt6.QtWidgets import QDialog
mesa_tab = self.tab_widget.currentWidget()
if mesa_tab not in self.mesa_tabs.values():
    QMessageBox.warning(self, "Novo projeto",
        "Antes de criar um projeto, selecione a aba Mesa 1 ou Mesa 2.")
    return
```

```
proj_name, ok = QDialog.getText(
    self, "Novo Projeto",
    "Nome do novo programa:",
    text=""
)
```

```
if not ok or not proj_name.strip():
    return # cancelado
proj_name = proj_name.strip()
```

Cria a pasta-raiz imediatamente

try:

```
from adhesive_program_manager import AdhesiveProgramManager
# garante .../<Projetos>/ existente
self.projects_dir.mkdir(parents=True, exist_ok=True)
self.proj_mgr = AdhesiveProgramManager(self.projects_dir)
# overwrite=True → recria estrutura completa sempre
self.proj_root = self.proj_mgr.create_program(proj_name, overwrite=True)
self.current_proj_name = proj_name
self.log(f"■■■ Estrutura base criada: {self.proj_root}")
```

except FileExistsError:

```
    QMessageBox.warning(self, "Projeto existente",
        f"O programa \"{proj_name}\" já existe.")
    return
```

except Exception as exc:

```
    self.log(f"■ Erro ao criar estrutura do projeto: {exc}")
    QMessageBox.critical(self, "Erro", str(exc))
    return
```

Limpa listas das abas

```
for tab in self.mesa_tabs.values():
    tab.inspect_widget.clear()
```

Libera botões de criação / ação

```
self._set_creation_controls_enabled(True)
self._current_project_file = None
self.log(f"■■■ Novo projeto \"{proj_name}\" iniciado")
```

```

# =====
# DEFINIR DIRETÓRIO BASE (Preferências ■ Definir caminho Modelos)
# =====

def _set_models_path(self):
    from PyQt6.QtWidgets import QFileDialog
    sel = QFileDialog.getExistingDirectory(
        self, "Escolha a pasta onde será criado "Projetos""
    )
    if not sel:
        return
    base = Path(sel).resolve()
    projetos_path = base / "Projetos"
    try:
        projetos_path.mkdir(parents=True, exist_ok=True)
    except Exception as exc:
        QMessageBox.critical(self, "Erro",
            f"Não foi possível criar "Projetos":\n{exc}")
        return
    # persiste
    self.projects_dir = projetos_path
    self.settings.projects_dir = projetos_path
    self.settings.save()
    # atualiza diretório-padrão nos ProgramIOWidgets já existentes
    for w in getattr(self, "_prog_widgets", []):
        if hasattr(w, "_default_dir"):
            w._default_dir = str(projetos_path)
    self.log(f"■ Diretório de projetos definido para: {projetos_path}")
    QMessageBox.information(self, "Preferências",
        f"Caminho configurado:\n{projetos_path}")

# --- salvar / abrir utilizando ProgramIOWidget já existente -----
def _save_project(self):
    """Salva arquivo referente à aba ativa."""
    if not self.current_proj_name or not self.projects_dir:
        QMessageBox.warning(self, "Salvar",
            "Nenhum projeto em edição ou pasta padrão indefinida.")
        return
    widget = self._current_prog_widget()
    # sufixo .m1 / .m2 conforme backend
    suffix = getattr(widget, "_default_suf", ".json")
    mesa_id = 1 if suffix.endswith("1") else 2
    dest = self.projects_dir / f"{self.current_proj_name}{suffix}"
    try:
        ok = widget._backend.save_to_file(str(dest))
        if ok:
            widget._last_file = str(dest)
            self.log(f"Projeto salvo em {dest}")
            QMessageBox.information(self, "Salvar",
                f"Projeto salvo com sucesso em:\n{dest}")
        else:
            QMessageBox.critical(self, "Erro", "Falha ao salvar o projeto.")

```

```
except Exception as exc:
    QMessageBox.critical(self, "Erro", str(exc))
```

```
def _save_as_project(self):
    """Abre diálogo de 'Salvar Como...' reaproveitando ProgramIOWidget."""
    widget = self._current_prog_widget()
    widget._on_save_clicked()
```

```
def _open_project(self):
    """Abre projeto (diálogo de arquivo) via ProgramIOWidget oculto."""
    widget = self._current_prog_widget()
    old_enabled = widget.isEnabled()
    widget._on_load_clicked()
    # se carregou com sucesso (_last_file setado), habilita botões
    if getattr(widget, "_last_file", None):
        # garante ponteiro do gerente de programa
        try:
            from adhesive_program_manager import AdhesiveProgramManager
            proj_path = Path(widget._last_file).resolve()
            proj_name = proj_path.stem
            self.proj_mgr = AdhesiveProgramManager(proj_path.parent)
            self.proj_mgr.create_program(proj_name, overwrite=False)
            self.current_proj_name = proj_name
        except Exception:
            QMessageBox.critical(self, "Erro",
                                "Não foi possível abrir o programa selecionado.")
            pass
    self._set_creation_controls_enabled(True)
```

```
# -----
# Habilita / desabilita botões de criação (ActionSelector + Add)
# -----
```

```
def _set_creation_controls_enabled(self, enabled: bool):
    """
    Ativa ou bloqueia os botões que só devem ser usados quando um
    projeto está em edição (novo ou aberto).
    """
```

```
for mesa_tab in self.mesa_tabs.values():
    # selector de ação
    mesa_tab.action_selector.setEnabled(enabled)
    # botão "Adicionar posição atual"
    mesa_tab.inspect_widget.btn_add.setEnabled(enabled)
# movimento / salvar posição da aba principal
if hasattr(self, 'inspect_widget'):
    self.inspect_widget.btn_add.setEnabled(enabled)
```

```
def _open_focus_dialog(self):
    from focus_calibration_dialog import FocusCalibrationDialog
    dlg = FocusCalibrationDialog(self, self)
    if dlg.exec():
```

```

# pega coeficientes
z0,f0,z1,f1 = dlg._z_top, dlg._f_top, dlg._z_bot, dlg._f_bot
a = (f1 - f0) / (z1 - z0)
b = f0 - a*z0
self.camera_manager.load_calibration(a,b)
self.settings.focus_coeffs = (a,b)
self.settings.save()
self.log("Calibração de foco salva")
# agora que existe calibração, liga auto-focus
self.camera_manager.enable_auto_focus(True)

```

```

# ----- diálogo FOV -----
def _open_fov_dialog(self):
    from fov_calibration_dialog import FOVCalibrationDialog
    dlg = FOVCalibrationDialog(self.settings, self)
    if dlg.exec():
        # recarrega as constantes na instância
        self.camera_fov = self.settings.camera_fov
        self._fov_coeffs = self._calc_fov_coeffs()
        self.log("Calibração de campo-de-visão salva")

```

```

# -----
# Calcula coeficientes lineares campo_mm(z)=a*z+b
# -----
def _calc_fov_coeffs(self) -> dict[str,float]:
    f = self.camera_fov
    z0 = float(f.get("z0_z_pulses", 0))
    z1 = float(f.get("z1_z_pulses", 700))
    w0 = float(f.get("z0_width_mm", 61.0))
    w1 = float(f.get("z1_width_mm", 30.5))
    h0 = float(f.get("z0_height_mm", 45.0))
    h1 = float(f.get("z1_height_mm", 22.5))
    if z1 == z0:      # garante denom.
        z1 += 1
    aX = (w1 - w0) / (z1 - z0)
    bX = w0 - aX * z0
    aY = (h1 - h0) / (z1 - z0)
    bY = h0 - aY * z0
    return {"aX":aX, "bX":bX, "aY":aY, "bY":bY}

```

```

# ----- diálogo calibração eixos -----
def _open_axis_calib_dialog(self):

    dlg = AxisCalibrationDialog(self, self)
    dlg.exec()

```

```

# ----- diálogo de limites -----
def _open_limits_dialog(self):
    from config_dialogs import WorkAreaConfigDialog
    dlg = WorkAreaConfigDialog(self.table_limits, self)
    if dlg.exec():

```

```

self.table_limits = dlg.get_limits()
# envia às abas
for mesa, tab in self.mesa_tabs.items():
    tab.set_limits(self.table_limits[mesa])
self.settings.table_limits = self.table_limits
self.settings.save()
self.log("Limites de mesa atualizados pelo usuário")

# ----- câmera -----
def _select_camera_dialog(self):
    from PyQt6.QtWidgets import QInputDialog
    idx, ok = QInputDialog.getInt(self, "Selecionar câmera",
                                  "Índice da câmera (0 = padrão):",
                                  value=self.settings.camera_index, min=0, max=10)

    if ok:
        self.settings.camera_index = idx
        self.settings.save()
        self.camera_manager.open(idx)
        self.log(f"Câmera {idx} selecionada")

# ----- persistência em JSON -----
def _load_limits_from_disk(self):
    try:
        if self._limits_file.exists():
            data = json.loads(self._limits_file.read_text(encoding="utf-8"))
            # valida estrutura mínima
            if all(str(k) in ("1", "2") for k in data):
                self.table_limits = {int(k): v for k, v in data.items()}
                self.log("Limites de mesa carregados do disco")
    except Exception as exc:
        self.log(f"Erro ao ler limites salvos: {exc}")

def _save_limits_to_disk(self):
    try:
        self._limits_file.write_text(
            json.dumps(self.table_limits, indent=2), encoding="utf-8")
        self.log("Limites de mesa gravados em table_limits.json")
    except Exception as exc:
        self.log(f"Erro ao salvar limites: {exc}")

def create_homing_sensors_status(self):
    """Exibe o estado ON/OFF dos sensores físicos de HOME"""
    group = QGroupBox("SENSORES DE HOME (Entradas X)")
    layout = QGridLayout()

    sensors = [
        ('X04_Y2', 'Sensor HOME Y2 (X0.4)'),
        ('X06_Y1', 'Sensor HOME Y1 (X0.6)'),
        ('X08_X', 'Sensor HOME X (X0.8)'),
        ('X09_Z', 'Sensor HOME Z (X0.9)')
    ]

```

```

for i, (addr_key, desc) in enumerate(sensors):
    layout.addWidget(QLabel(f'{desc}:'), i, 0)
    lbl = QLabel("OFF")
    lbl.setStyleSheet("QLabel { background-color: gray; color: white; padding: 5px; }")
    layout.addWidget(lbl, i, 1)
    setattr(self, f'{addr_key}_status', lbl)

```

```

group.setLayout(layout)
return group

```

```

def create_config_tab(self):
    """Aba que exibe TODOS os registradores configuráveis em formato editável"""
    tab = QWidget()
    vbox = QVBoxLayout(tab)

    # Scroll para caber em qualquer resolução
    scroll = QScrollArea()
    scroll.setWidgetResizable(True)
    scroll_content = QWidget()
    grid = QGridLayout(scroll_content)

    # Cria um spinBox 32 bits para cada registrador listado em self.configurable_registers
    for row, (reg_key, desc) in enumerate(self.configurable_registers):
        addr = self.addresses.get(reg_key)
        if addr is None:
            continue # pula se endereço ainda não mapeado

        # Coluna 0 – label descritivo
        grid.addWidget(QLabel(f'{reg_key} ({desc})'), row, 0)

        # Coluna 1 – spinBox (-2 147 483 648 a 2 147 483 647)
        spin = QSpinBox()
        spin.setRange(-2_147_483_648, 2_147_483_647)
        spin.setObjectName(f'cfg_spin_{reg_key}')
        grid.addWidget(spin, row, 1)

        # Coluna 2 – botão “Gravar”
        btn = QPushButton("Gravar")
        btn.clicked.connect(lambda _=False, rk=reg_key: self.write_single_config(rk))
        grid.addWidget(btn, row, 2)

    scroll_content.setLayout(grid)
    scroll.setWidget(scroll_content)
    vbox.addWidget(scroll)

    # Botões globais
    h = QHBoxLayout()
    ler_btn = QPushButton("Ler Todos")
    gravar_btn = QPushButton("Gravar Todos")
    ler_btn.clicked.connect(self.read_all_config)
    gravar_btn.clicked.connect(self.write_all_config)

```

```
h.addStretch()
h.addWidget(ler_btn)
h.addWidget(gravar_btn)
vbox.addLayout(h)
```

```
# Leitura inicial automática quando a aba é criada
QTimer.singleShot(200, self.read_all_config)
return tab
```

```
# -----
# Funções auxiliares da aba de configuração
# -----
def read_all_config(self):
    """Lê todos os registradores configuráveis e atualiza os spinBoxes"""
    if not self.connected:
        return
    for reg_key, _ in self.configurable_registers:
        addr = self.addresses.get(reg_key)
        if addr is None:
            continue
        try:
            value = self.read_dword(addr)
            spin: QSpinBox = self.findChild(QSpinBox, f"cfg_spin_{reg_key}")
            if spin:
                spin.blockSignals(True)
                spin.setValue(value)
                spin.blockSignals(False)
        except Exception as exc:
            self.log(f"⚠ Falha ao ler {reg_key}: {exc}")
            continue
    self.log("✅ Configuração – leitura concluída")

def write_single_config(self, reg_key):
    """Grava somente o registrador indicado"""
    if not self.connected:
        self.log("⚠ CLP não conectado")
        return
    addr = self.addresses.get(reg_key)
    if addr is None:
        return
    spin: QSpinBox = self.findChild(QSpinBox, f"cfg_spin_{reg_key}")
    if spin is None:
        return
    value = spin.value()
    try:
        res = self.write_dword(addr, value)
        if not res.isError():
            self.log(f"✅ {reg_key} gravado: {value}")
        else:
            self.log(f"⚠ Falha ao gravar {reg_key}")
    except Exception as e:
```

```
self.log(f"⚠ Erro ao gravar {reg_key}: {e}")
```

```
def write_all_config(self):
```

```
    """Grava TODOS os registradores configuráveis"""
```

```
    for reg_key, _ in self.configurable_registers:
```

```
        self.write_single_config(reg_key)
```

```
    self.log("✅ Configuração – gravação concluída")
```

```
def create_axis_control(self, title, axis_name):
```

```
    """Cria controle para um eixo - CORRIGIDO para valores absolutos"""
```

```
    group = QGroupBox(title)
```

```
    group.setStyleSheet("QGroupBox { font-weight: bold; font-size: 12px; }")
```

```
    layout = QVBoxLayout()
```

```
    # Parâmetros
```

```
    params_frame = QFrame()
```

```
    params_layout = QGridLayout(params_frame)
```

```
    # Mapeamento de endereços corrigido
```

```
    addr_map = {
```

```
        'Y2': (f'D100_{axis_name}', f'D20000_{axis_name}'), # Entrada usuário, Velocidade
```

```
        'Y1': (f'D600_{axis_name}', f'D20500_{axis_name}'), # Entrada usuário, Velocidade
```

```
        'Z': (f'D1600_{axis_name}', f'D21500_{axis_name}'), # Entrada usuário, Velocidade
```

```
        'X': (f'D1100_{axis_name}', f'D21000_{axis_name}') # Entrada usuário, Velocidade
```

```
    }
```

```
    pulsos_addr, vel_addr = addr_map[axis_name]
```

```
    # Apenas posição absoluta (velocidade passou para aba de configuração)
```

```
    pulsos_label = f'{pulsos_addr} - Posição Absoluta:"
```

```
    pulsos_tooltip = "Coordenada absoluta de destino"
```

```
    default_pulsos = 0
```

```
    # Posição absoluta – 32 bits (aceita valores negativos)
```

```
    params_layout.addWidget(QLabel(pulsos_label), 0, 0)
```

```
    pulsos_spin = QSpinBox()
```

```
    pulsos_spin.setRange(-2_000_000_000, 2_000_000_000)
```

```
    pulsos_spin.setValue(default_pulsos)
```

```
    pulsos_spin.setToolTip(pulsos_tooltip)
```

```
    params_layout.addWidget(pulsos_spin, 0, 1)
```

```
    setattr(self, f'pulsos_spin_{axis_name}', pulsos_spin)
```

```
    # Display da posição atual
```

```
    sr_addr_map = {'Y2': 'D3100_Y2', 'Y1': 'D3400_Y1', 'Z': 'D3200_Z', 'X': 'D3000_X'}
```

```
    params_layout.addWidget(QLabel(f'{sr_addr_map[axis_name]} - Posição Atual:"), 1, 0)
```

```
    current_pos_label = QLabel("0")
```

```
    current_pos_label.setStyleSheet("QLabel { background-color: lightblue; padding: 2px; font-weight: bold; max-height: 20px; }")
```

```
    params_layout.addWidget(current_pos_label, 1, 1)
```

```
    setattr(self, f'current_pos_label_{axis_name}', current_pos_label)
```

```

# Indicador LED de HOMING REALIZADO
params_layout.addWidget(QLabel("Status Homing:"), 2, 0)
homing_led = QLabel("●")
homing_led.setStyleSheet("QLabel { background-color: gray; color: gray; padding: 2px; font-size: 16px; font-weight: bold; }")
homing_led.setAlignment(Qt.AlignmentFlag.AlignCenter)
homing_led.setToolTip("Cinza: Homing não realizado | Verde: Homing realizado")
params_layout.addWidget(homing_led, 2, 1)
setattr(self, f'homing_led_{axis_name}', homing_led)

layout.addWidget(params_frame)

# Botões de escrita
write_btn = QPushButton("Escrever Parâmetros")
write_btn.clicked.connect(lambda: self.write_axis_parameters(axis_name))
layout.addWidget(write_btn)

# Botões de movimento
move_frame = QFrame()
move_layout = QGridLayout(move_frame)

# Movimento absoluto principal
move_abs_btn = QPushButton("■ MOVER PARA POSIÇÃO ABSOLUTA")
move_abs_btn.setStyleSheet("QPushButton { background-color: #2196F3; color: white; padding: 10px; font-weight: bold; }")
move_abs_btn.clicked.connect(lambda: self.move_axis_absolute(axis_name))
move_layout.addWidget(move_abs_btn, 0, 0, 1, 2)

# Controles JOG para TODOS OS EIXOS (novo sistema)
if axis_name in ['Y2', 'Y1', 'X', 'Z']: # Todos os eixos têm JOG agora
    # Separador visual
    separator = QFrame()
    separator.setFrameStyle(QFrame.Shape.HLine | QFrame.Shadow.Sunken)
    move_layout.addWidget(separator, 1, 0, 1, 2)

    # Frame JOG
    jog_frame = QFrame()
    jog_frame.setStyleSheet("QFrame { border: 2px solid #2196F3; border-radius: 5px; background-color: #E3F2FD; }")
    jog_layout = QVBoxLayout(jog_frame)

    # Botões JOG com pressionar/soltar
    jog_buttons_layout = QHBoxLayout()

    # Botão JOG Negativo (à esquerda)
    jog_minus_btn = QPushButton(f"■ JOG {axis_name}-")
    jog_minus_btn.setStyleSheet("""
        QPushButton {
            background-color: #FF5722;
            color: white;
            font-weight: bold;
            padding: 10px;
            border-radius: 5px;
        }
    """)

```

```

        QPushButton:pressed {
            background-color: #e64a19;
        }
    """)
    jog_minus_btn.pressed.connect(lambda: self.jog_start(axis_name, '-'))
    jog_minus_btn.released.connect(lambda: self.jog_stop(axis_name))
    jog_buttons_layout.addWidget(jog_minus_btn)
    setattr(self, f'jog_minus_btn_{axis_name}', jog_minus_btn)

```

Botão JOG Positivo (à direita)

```
jog_plus_btn = QPushButton(f"■ JOG {axis_name}+")
```

```
jog_plus_btn.setStyleSheet("""
```

```

    QPushButton {
        background-color: #4CAF50;
        color: white;
        font-weight: bold;
        padding: 10px;
        border-radius: 5px;
    }

```

```

    QPushButton:pressed {
        background-color: #45a049;
    }

```

```
""")
```

```

jog_plus_btn.pressed.connect(lambda: self.jog_start(axis_name, '+'))
jog_plus_btn.released.connect(lambda: self.jog_stop(axis_name))
jog_buttons_layout.addWidget(jog_plus_btn)
setattr(self, f'jog_plus_btn_{axis_name}', jog_plus_btn)

```

```
jog_layout.addLayout(jog_buttons_layout)
```

```
move_layout.addWidget(jog_frame, 2, 0, 1, 2)
```

```
layout.addWidget(move_frame)
```

```
group.setLayout(layout)
```

```
return group
```

```
def create_auxiliary_controls(self):
```

```
    """Cria controles auxiliares - Y0.10 CORRIGIDO"""
```

```
    group = QGroupBox("CONTROLES AUXILIARES")
```

```
    group.setStyleSheet("QGroupBox { font-weight: bold; font-size: 12px; }")
```

```
    layout = QVBoxLayout()
```

```
# Saída Y0.10 ■ comando tipo "falling-edge trigger"
```

```
y010_btn = QPushButton("SHOT (M5000)")
```

```
y010_btn.setCheckable(True)
```

```
y010_btn.clicked.connect(self.pulse_y010)
```

```
y010_btn.setStyleSheet("QPushButton:checked { background-color: orange; }")
```

```
layout.addWidget(y010_btn)
```

```
setattr(self, 'y010_btn', y010_btn)
```

```
group.setLayout(layout)
return group
```

```
# =====
#          SEQUÊNCIA DE HOMING GERAL (Z → (Y2,Y1,X))
# =====
```

```
def home_all_axes(self):
```

```
    """
```

```
    1) Aciona homing do eixo Z (M1850) e aguarda 3 s.
    2) Depois aciona simultaneamente os homings de Y2 (M350), Y1 (M850) e X (M1350).
    Todos os coils são soltos 200 ms após o disparo (pulso momentâneo).
    """
```

```
    if not self.connected:
```

```
        self.log("■ CLP não conectado")
        return
```

```
    # Etapa 1 – homing apenas do Z
```

```
    self.log("■ HOMING GERAL: acionando homing do eixo Z (M1850)...")
    self._pulse_coil('M1850')
```

```
    # Agenda etapa 2 para daqui a 3 s
```

```
    QTimer.singleShot(3000, self._home_remaining_axes)
```

```
def _home_remaining_axes(self):
```

```
    """Dispara homing de Y2, Y1 e X em paralelo e solta todos os coils."""
```

```
    try:
```

```
        self.log("■ Acionando homing simultâneo de Y2, Y1 e X...")
        for mem in ['M350', 'M850', 'M1350']:
            self._pulse_coil(mem)
```

```
    except Exception as e:
```

```
        self.log(f"■ Erro na etapa 2 do homing geral: {e}")
```

```
# -----
```

```
# Helper interno – envia pulso de 20 ms em uma memória M
```

```
# -----
```

```
def _pulse_coil(self, mem_key):
```

```
    """Liga o coil por 20 ms e depois desliga (edge-trigger)."""
```

```
    try:
```

```
        addr = self.addresses[mem_key]
        self.client.write_coil(addr, True)
        QTimer.singleShot(100, lambda a=addr: self.client.write_coil(a, False))
        self.log(f"■ Pulso {mem_key} enviado")
```

```
    except Exception as e:
```

```
        self.log(f"■ Falha ao pulsar {mem_key}: {e}")
```

```
def create_outputs_status(self):
```

```
    """Cria status das saídas"""
```

```
    group = QGroupBox("STATUS DAS SAÍDAS")
```

```
    layout = QGridLayout()
```

```

outputs = [
    ('Y0.0', 'Pulso Eixo 1'),
    ('Y0.1', 'Dir Eixo 1'),
    ('Y0.2', 'Pulso Eixo 2'),
    ('Y0.3', 'Dir Eixo 2'),
    ('Y0.4', 'Pulso Eixo 3'),
    ('Y0.5', 'Dir Eixo 3'),
    ('Y0.6', 'Pulso Eixo 4'),
    ('Y0.7', 'Dir Eixo 4'),
    ('Y0.10', 'Y0.10 (SHOT)')
]

```

```

for i, (name, desc) in enumerate(outputs):
    layout.addWidget(QLabel(f"{name}:"), i//2, (i%2)*2)
    status_label = QLabel("OFF")
    status_label.setStyleSheet("QLabel { background-color: gray; color: white; padding: 5px; }")
    layout.addWidget(status_label, i//2, (i%2)*2 + 1)
    setattr(self, f'{name.replace(".", "_")}__status', status_label)

```

```

layout.addWidget(QLabel(f"{desc}"), i//2, (i%2)*2 + 2)

```

```

group.setLayout(layout)
return group

```

```

def create_memories_status(self):

```

```

    """Cria status das memórias"""

```

```

    group = QGroupBox("STATUS DAS MEMÓRIAS")
    layout = QGridLayout()

```

```

    memories = ['M0_Y2', 'M50_Y2', 'M100_Y2', 'M500_Y1', 'M550_Y1', 'M600_Y1',
                'M1500_Z', 'M1550_Z', 'M1600_Z', 'M1000_X', 'M1050_X', 'M1100_X',
                'M5000', 'M5001',
                'M350', 'M850', 'M1350', 'M1850',
                'M300', 'M800', 'M1300', 'M1800',
                'M70_Y2', 'M570_Y1', 'M1070_X', 'M1570_Z']

```

```

    for i, mem in enumerate(memories):
        layout.addWidget(QLabel(f"{mem}:"), i//4, (i%4)*2)
        status_label = QLabel("OFF")
        status_label.setStyleSheet("QLabel { background-color: gray; color: white; padding: 5px; }")
        layout.addWidget(status_label, i//4, (i%4)*2 + 1)
        setattr(self, f'{mem}__status', status_label)

```

```

    group.setLayout(layout)
    return group

```

```

def create_registers_status(self):

```

```

    """Cria status dos registradores"""

```

```

    group = QGroupBox("STATUS DOS REGISTRADORES")
    layout = QGridLayout()

```

```

registers = [
    'D0_Y2', 'D100_Y2', 'D20000_Y2',    # Y2 (entrada usuário)
    'D500_Y1', 'D600_Y1', 'D20500_Y1',  # Y1 (entrada usuário)
    'D1500_Z', 'D1600_Z', 'D21500_Z',    # Z (entrada usuário)
    'D1000_X', 'D1100_X', 'D21000_X',    # X (entrada usuário)
    'D3100_Y2', 'D3400_Y1', 'D3200_Z', 'D3000_X', # Posições atuais - CORRIGIDO
    'D22000', 'D22010', 'D22020', 'D22030',    # Velocidades JOG
    'D23000', 'D23010', 'D23020', 'D23030',    # Velocidades Homing ROM
    'D23050', 'D23060', 'D23070', 'D23080',    # Limites Negativos ROM
    'D23090', 'D23100', 'D23110', 'D23120'    # Limites Positivos ROM
]

```

```

for i, reg in enumerate(registers):
    layout.addWidget(QLabel(f"{reg}:"), i//4, (i%4)*2)
    status_label = QLabel("0")
    status_label.setStyleSheet("QLabel { background-color: lightgray; padding: 5px; }")
    layout.addWidget(status_label, i//4, (i%4)*2 + 1)
    setattr(self, f'{reg}_status', status_label)

```

```

group.setLayout(layout)
return group

```

```

def connect_plc(self):
    """Conecta ao CLP"""
    try:
        self.log("Conectando ao CLP...")
        self.client = ModbusTcpClient('192.168.1.5', port=502)

        if self.client.connect():
            self.connected = True
            self.status_bar.showMessage("■ CONECTADO: 192.168.1.5:502")
            self.status_bar.setStyleSheet(
                "QStatusBar { background-color:green; color:white; font-weight:bold; }")
            self.log("■ Conectado com sucesso!")
            self.log("■ Endereços corrigidos conforme ladder real")
        else:
            self.connected = False
            self.status_bar.showMessage("Desconectado")
            self.status_bar.setStyleSheet(
                "QStatusBar { background-color:red; color:white; font-weight:bold; }")
            self.log("■ Falha na conexão")

    except Exception as e:
        self.connected = False
        self.status_bar.showMessage("Desconectado")
        self.status_bar.setStyleSheet(
            "QStatusBar { background-color:red; color:white; font-weight:bold; }")
        self.log(f"■ Erro na conexão: {e}")

```

```

def read_initial_jog_velocities(self):

```

```
"""Lê velocidades JOG atuais dos registradores D22000 série"""
```

```
if not self.connected:
```

```
    return
```

```
try:
```

```
    self.log("■ Lendo velocidades JOG atuais...")
```

```
    # Mapeamento dos registradores JOG
```

```
    jog_velocity_map = {
```

```
        'Y2': ('D22000', 'jog_velocity_spin_Y2'),
```

```
        'Y1': ('D22010', 'jog_velocity_spin_Y1'),
```

```
        'X': ('D22020', 'jog_velocity_spin_X'),
```

```
        'Z': ('D22030', 'jog_velocity_spin_Z')
```

```
    }
```

```
    for axis, (reg_key, spin_attr) in jog_velocity_map.items():
```

```
        try:
```

```
            jog_velocity = self.read_dword(self.addresses[reg_key])
```

```
    # Atualiza interface se spinbox existir
```

```
        if hasattr(self, spin_attr):
```

```
            jog_spin = getattr(self, spin_attr)
```

```
            jog_spin.setValue(abs(jog_velocity))
```

```
            self.log(f"■ Velocidade JOG {axis}: {abs(jog_velocity)} Hz")
```

```
    except Exception as e:
```

```
        self.log(f"■ Erro ao ler velocidade JOG {axis}: {e}")
```

```
        continue
```

```
except Exception as e:
```

```
    self.log(f"■ Erro geral na leitura das velocidades JOG: {e}")
```

```
def read_initial_limits(self):
```

```
    """Lê limites salvos na ROM"""
```

```
    if not self.connected:
```

```
        return
```

```
try:
```

```
    self.log("■ Lendo limites da ROM...")
```

```
    limits_map = {
```

```
        'Y2': ('D23050', 'D23090'), # Negativo, Positivo
```

```
        'Y1': ('D23060', 'D23100'), # Negativo, Positivo
```

```
        'X': ('D23070', 'D23110'), # Negativo, Positivo
```

```
        'Z': ('D23080', 'D23120') # Negativo, Positivo
```

```
    }
```

```
    for axis, (neg_reg, pos_reg) in limits_map.items():
```

```
        try:
```

```
            neg_limit = self.read_dword(self.addresses[neg_reg])
```

```
            pos_limit = self.read_dword(self.addresses[pos_reg])
```

```
            self.log(f"■ Limites {axis}: {neg_limit} a {pos_limit}")
```

```
except Exception as e:
    self.log(f"■■■ Erro ao ler limites {axis}: {e}")
    continue
```

```
except Exception as e:
    self.log(f"■ Erro geral na leitura dos limites: {e}")
```

```
def read_initial_homing_status(self):
    """Lê status inicial das memórias de homing na inicialização"""
    if not self.connected:
        return

    try:
        self.log("■ Lendo status inicial das memórias de homing...")

        homing_memories = {
            'M300': 'Y2',
            'M800': 'Y1',
            'M1300': 'X',
            'M1800': 'Z'
        }

        for mem_name, axis in homing_memories.items():
            try:
                addr = self.addresses[mem_name]
                result = self.client.read_coils(addr, count=1)

                if not result.isError():
                    state = result.bits[0]

                    # Atualiza LED diretamente
                    homing_led = getattr(self, f'homing_led_{axis}', None)
                    if homing_led:
                        self.update_homing_led(homing_led, state)

                    # Log do status inicial
                    status_text = "REALIZADO" if state else "NÃO REALIZADO"
                    self.log(f"■ Status inicial Homing {axis}: {status_text}")

                    # Inicializa variável de controle de mudança
                    setattr(self, f'_last_homing_status_M{mem_name[1:]}', state)

                    # FORÇA atualização do LED imediatamente
                    homing_led = getattr(self, f'homing_led_{axis}', None)
                    if homing_led:
                        self.update_homing_led(homing_led, state)

            except Exception as e:
                self.log(f"■■■ Erro ao ler status inicial {mem_name}: {e}")
                continue
```

```

self.log("■ Leitura inicial de homing concluída")

except Exception as e:
    self.log(f"■ Erro geral na leitura inicial de homing: {e}")

# =====
# SISTEMA DE CACHE PARA PRESERVAR JANELAS DE COMPARAÇÃO
# =====

def _save_current_comparison_windows(self, parent_item):
    """
    Salva o estado atual das janelas de comparação no cache
    associado à posição mecânica especificada
    """
    if parent_item is None or not self.roi_editor:
        return

    try:
        # Coleta todas as janelas atualmente no visor ROI
        current_windows = []
        for item in self.roi_editor.windows:
            if hasattr(item, 'comparison_name'):
                # Salva dados essenciais da janela
                window_data = {
                    'name': item.comparison_name,
                    'rect': {
                        'x': item.rect().x(),
                        'y': item.rect().y(),
                        'width': item.rect().width(),
                        'height': item.rect().height()
                    },
                    'scene_pos': {
                        'x': item.scenePos().x(),
                        'y': item.scenePos().y()
                    },
                    'tooltip': item.toolTip()
                }
                current_windows.append(window_data)

        # Salva no cache
        self._comparison_windows_cache[parent_item] = current_windows

        if current_windows:
            self.log(f"■ {len(current_windows)} janelas de comparação salvas no cache para posição mecânica")

    except Exception as e:
        self.log(f"■ Erro ao salvar janelas no cache: {e}")

def _load_comparison_windows_from_cache(self, parent_item):
    """

```

Carrega as janelas de comparação do cache para a posição mecânica especificada

"""

if parent_item is None or not self.roi_editor:

return

try:

Verifica se há janelas salvas para esta posição

cached_windows = self._comparison_windows_cache.get(parent_item, [])

if not cached_windows:

return

Limpa janelas atuais do ROI editor (sem emitir sinais)

self._clear_roi_editor_silently()

Recria cada janela salva

for window_data in cached_windows:

try:

Cria nova janela com os mesmos parâmetros

rect_data = window_data['rect']

new_item = self.roi_editor.add_window(

x=rect_data['x'],

y=rect_data['y'],

w=rect_data['width'],

h=rect_data['height'],

deletable=True

)

Restaura propriedades

new_item.comparison_name = window_data['name']

new_item.setToolTip(window_data.get('tooltip', ''))

Adiciona à TreeView

self._add_comparison_window_to_tree(parent_item, new_item)

except Exception as e:

self.log(f"■■■ Erro ao restaurar janela {window_data.get('name', '?')}: {e}")

continue

self.log(f"■ {len(cached_windows)} janelas de comparação restauradas do cache")

except Exception as e:

self.log(f"■■■ Erro ao carregar janelas do cache: {e}")

def _clear_roi_editor_silently(self):

"""Remove todas as janelas do ROI editor sem emitir sinais"""

if not self.roi_editor:

return

try:

Remove itens da cena silenciosamente

```

for item in self.roi_editor.windows[:]: # cópia para evitar modificação durante iteração
    try:
        if item.scene():
            self.roi_editor.view.scene().removeItem(item)
            self.roi_editor.windows.remove(item)
        except (RuntimeError, ValueError):
            # Item já foi removido ou não existe mais
            if item in self.roi_editor.windows:
                self.roi_editor.windows.remove(item)

except Exception as e:
    self.log(f"■■■ Erro ao limpar ROI editor: {e}")

def _clear_cache_for_position(self, parent_item):
    """Remove do cache todas as janelas associadas a uma posição mecânica"""
    if parent_item in self._comparison_windows_cache:
        del self._comparison_windows_cache[parent_item]
        self.log(f"■■■ Cache de janelas limpo para posição mecânica removida")

def read_current_positions(self):
    """Lê posições atuais dos motores (função dedicada)"""
    if not self.connected:
        return

    try:
        position_registers = {
            'D3100_Y2': 'Y2',
            'D3400_Y1': 'Y1',
            'D3200_Z': 'Z',
            'D3000_X': 'X'
        }

        for reg_name, axis in position_registers.items():
            try:
                addr = self.addresses[reg_name]
                value = self.read_dword(addr)

                # Atualiza posição e interface
                self.current_positions[axis] = value
                label = getattr(self, f'current_pos_label_{axis}', None)
                if label:
                    label.setText(str(value))

            except Exception as e:
                self.log(f"■■■ Erro ao ler posição {axis} ({reg_name}): {e}")
                continue

    except Exception as e:
        # Falha silenciosa para não afetar outras operações
        self.log(f"■■■ Erro ao ler posições: {e}")

```

```

def monitor_homing_status(self):
    """Função dedicada APENAS para monitorar status de homing em tempo real"""
    if not self.connected:
        return

    try:
        homing_memories = {
            'M300': 'Y2',
            'M800': 'Y1',
            'M1300': 'X',
            'M1800': 'Z'
        }

        for mem_name, axis in homing_memories.items():
            try:
                addr = self.addresses[mem_name]
                result = self.client.read_coils(addr, count=1)

                if not result.isError():
                    state = result.bits[0]

                    # Atualiza LED SEMPRE (sem verificação de mudança)
                    homing_led = getattr(self, f'homing_led_{axis}', None)
                    if homing_led:
                        current_led_state = "lime" in homing_led.styleSheet()

                        # Só atualiza se estado for diferente do LED atual
                        if (state and not current_led_state) or (not state and current_led_state):
                            self.update_homing_led(homing_led, state)
                            self.log(f"■ LED Homing {axis} atualizado: {'VERDE' if state else 'CINZA'}")

            except Exception as e:
                self.log(f"■ Erro ao ler status de homing {axis}: {e}")
                continue

        except Exception as e:
            self.log(f"■ Erro ao monitorar status de homing: {e}")
            pass

def test_homing_status(self):
    """Função de teste específica para verificar status das memórias de homing"""
    if not self.connected:
        return

    self.log("■ Testando status das memórias de homing...")

    homing_memories = {
        'M300': 'Y2',
        'M800': 'Y1',
        'M1300': 'X',
        'M1800': 'Z'
    }

```

```
}
```

```
for mem_name, axis in homing_memories.items():
```

```
    try:
```

```
        addr = self.addresses[mem_name]
```

```
        result = self.client.read_coils(addr, count=1)
```

```
    if not result.isError():
```

```
        state = result.bits[0]
```

```
        status_text = "REALIZADO" if state else "NÃO REALIZADO"
```

```
        self.log(f"■ Homing {axis} ({mem_name}): {status_text}")
```

```
        # Força atualização do LED
```

```
        homing_led = getattr(self, f'homing_led_{axis}', None)
```

```
        if homing_led:
```

```
            self.update_homing_led(homing_led, state)
```

```
            self.log(f"■ LED {axis} forçado para: {'VERDE' if state else 'CINZA'}")
```

```
    else:
```

```
        self.log(f"■ Erro ao ler {mem_name}: {result}")
```

```
        # Força uma atualização completa dos LEDs após o teste
```

```
        QTimer.singleShot(100, self.force_homing_leds_update)
```

```
        # Também força verificação dedicada
```

```
        QTimer.singleShot(200, self.force_homing_status_check)
```

```
except Exception as e:
```

```
    self.log(f"■ Erro ao testar {mem_name}: {e}")
```

```
def force_homing_leds_update(self):
```

```
    """Força atualização imediata de todos os LEDs de homing"""
```

```
    try:
```

```
        # Reset das variáveis de controle para forçar atualização
```

```
        for axis in ['Y2', 'Y1', 'X', 'Z']:
```

```
            if hasattr(self, f'_last_led_status_{axis}'):

```

```
                delattr(self, f'_last_led_status_{axis}')

```

```
            if hasattr(self, f'_last_homing_status_M{axis}'):

```

```
                delattr(self, f'_last_homing_status_M{axis}')
```

```
        self.log("■ Forçando atualização completa dos LEDs...")
```

```
        # Força leitura imediata
```

```
        QTimer.singleShot(50, self.monitor_homing_status)
```

```
except Exception as e:
```

```
    self.log(f"■ Erro ao forçar atualização: {e}")
```

```
def force_update_homing_leds(self):
```

```
    """Força atualização de todos os LEDs de homing"""
```

```
    self.log("■ Forçando atualização dos LEDs de homing...")
```

```
    self.test_homing_status()
```

```

# ----- helpers de 32 bits -----
def write_dword(self, address, value):
    """Escreve INT32 em dois registradores Modbus (little-endian)."""
    u32 = value & 0xFFFFFFFF
    lo = u32 & 0xFFFF
    hi = (u32 >> 16) & 0xFFFF
    return self.client.write_registers(address, [lo, hi])

def read_dword(self, address):
    """Lê INT32 assinado de dois registradores."""
    res = self.client.read_holding_registers(address, count=2)
    if res.isError():
        raise ValueError(res)
    lo, hi = res.registers
    u32 = (hi << 16) | lo
    return u32 if u32 < 0x8000_0000 else u32 - 0x1_0000_0000

# -----
def write_axis_parameters(self, axis_name):
    """Escreve parâmetros de um eixo"""
    if not self.connected:
        self.log("■ CLP não conectado")
        return

    try:
        # Obtém valores da interface
        p_spin = getattr(self, f'pulsos_spin_{axis_name}')
        pulsos_value = p_spin.value()

        # Mapeamento de endereços
        addr_map = {
            'Y2': (f'D100_{axis_name}', f'D20000_{axis_name}'),
            'Y1': (f'D600_{axis_name}', f'D20500_{axis_name}'),
            'Z': (f'D1600_{axis_name}', f'D21500_{axis_name}'),
            'X': (f'D1100_{axis_name}', f'D21000_{axis_name}')
        }
        p_addr_key, v_addr_key = addr_map[axis_name]
        pulsos_addr = self.addresses[p_addr_key]
        vel_addr = self.addresses[v_addr_key]

        self.log(f"■ Escrevendo parâmetros Eixo {axis_name}: Posição={pulsos_value}")

    # -----
    # 1) POSIÇÃO ABSOLUTA (sempre grava)
    # -----
    if self.write_dword(pulsos_addr, pulsos_value).isError():
        self.log(f"■ Erro ao escrever posição {axis_name}")
        return

    # -----

```

```

# 2) VELOCIDADE      (novo) – pega do spinBox da aba
#   Config. Registradores ou usa default 20000 Hz
# -----
try:
    vel_spin = self.findChild(QSpinBox, f"cfg_spin_{v_addr_key}")
    vel_value = vel_spin.value() if vel_spin else 20000
except Exception:
    vel_value = 20000

if self.write_dword(vel_addr, vel_value).isError():
    self.log(f"■ Erro ao escrever velocidade {axis_name}")
    return

# LOG
self.log(f"■ Eixo {axis_name}: pos={pulsos_value} vel={vel_value}")

# Verifica se posição ficou realmente gravada
self.verify_write(pulsos_addr, pulsos_value, p_addr_key)

except Exception as e:
    self.log(f"■ Erro ao escrever parâmetros eixo {axis_name}: {e}")

def verify_write(self, address, expected_value, name):
    """Verifica DWORD"""
    try:
        actual_value = self.read_dword(address)
        if actual_value == expected_value:
            self.log(f"✓ {name} verificado: {actual_value}")
        else:
            self.log(f"■ {name} divergente: esperado {expected_value}, lido {actual_value}")
    except Exception as e:
        self.log(f"■ Erro verificação {name}: {e}")

def move_axis_absolute(self, axis_name):
    """Move eixo para posição absoluta"""
    if not self.connected:
        self.log("■ CLP não conectado")
        return

    try:
        # Escreve parâmetros primeiro
        self.write_axis_parameters(axis_name)

        # Comando de movimento
        cmd_map = {
            'Y2': f'M50_{axis_name}', # INICIA MOVIMENTO ABSOLUTO
            'Y1': f'M550_{axis_name}', # INICIA MOVIMENTO ABSOLUTO
            'Z': f'M1550_{axis_name}', # INICIA MOVIMENTO ABSOLUTO
            'X': f'M1050_{axis_name}' # INICIA MOVIMENTO ABSOLUTO
        }
        cmd_addr = self.addresses[cmd_map[axis_name]]

```

```
target_position = getattr(self, f'pulsos_spin_{axis_name}').value()
self.log(f"■ Movendo eixo {axis_name} para posição {target_position}")
```

```
# envia pulso de 100 ms no coil → garante borda de subida
self._pulse_coil(cmd_map[axis_name])
self.log(f"■ Pulso {cmd_map[axis_name]} enviado – alvo {target_position}")
```

```
except Exception as e:
    self.log(f"■ Erro ao mover eixo {axis_name}: {e}")
```

```
def move_axis_home(self, axis_name):
    """Move eixo para HOME (zera posição)"""
```

```
if not self.connected:
```

```
    self.log("■ CLP não conectado")
    return
```

```
try:
    self.log(f"■ Zerando posição eixo {axis_name}")
```

```
# Comando de busca pelo HOME - endereços corrigidos conforme planilha
```

```
cmd_map = {
    'Y2': 'M350', # GO TO HOME Y2
    'Y1': 'M850', # GO TO HOME Y1
    'Z': 'M1850', # GO TO HOME Z
    'X': 'M1350' # GO TO HOME X
}
```

```
cmd_addr = self.addresses[cmd_map[axis_name]]
```

```
result = self.client.write_coil(cmd_addr, True)
```

```
if not result.isError():
```

```
    self.log(f"■ Comando HOME enviado para eixo {axis_name}")
```

```
except Exception as e:
```

```
    self.log(f"■ Erro movimento HOME eixo {axis_name}: {e}")
```

```
def move_relative(self, axis_name, offset):
```

```
    """Movimento relativo a partir da posição atual"""
```

```
if not self.connected:
```

```
    self.log("■ CLP não conectado")
    return
```

```
try:
```

```
    current_pos = self.current_positions[axis_name]
    new_position = current_pos + offset
```

```
# Atualiza interface e executa movimento
```

```
p_spin = getattr(self, f'pulsos_spin_{axis_name}')
```

```
p_spin.setValue(new_position)
```

```
self.move_axis_absolute(axis_name)
```

```
self.log(f"■ Movimento relativo {axis_name}: {current_pos} + {offset} = {new_position}")
```

except Exception as e:

```
self.log(f"■ Erro movimento relativo {axis_name}: {e}")
```

SISTEMA JOG TODOS OS EIXOS

```
def set_jog_velocity(self, axis_name):
```

```
    """Define velocidade do JOG para qualquer eixo"""
```

```
    if not self.connected:
```

```
        self.log("■ CLP não conectado")
```

```
        return
```

```
    try:
```

```
        # Obtém velocidade: se spinbox existir usa-o; caso contrário
```

```
        # lê o valor já programado no CLP (mantém UI enxuta).
```

```
        if hasattr(self, f'jog_velocity_spin_{axis_name}')
```

```
            velocity = getattr(self, f'jog_velocity_spin_{axis_name}').value()
```

```
        else:
```

```
            pos_addr_key = {
```

```
                'Y2': 'D22000', 'Y1': 'D22010', 'X': 'D22020', 'Z': 'D22030'
```

```
            }[axis_name]
```

```
            velocity = abs(self.read_dword(self.addresses[pos_addr_key]))
```

```
    # Mapeamento dos registradores JOG por eixo
```

```
    jog_addr_map = {
```

```
        'Y2': ('D22000', 'D22050'), # Positivo, Negativo
```

```
        'Y1': ('D22010', 'D22060'), # Positivo, Negativo
```

```
        'X': ('D22020', 'D22070'), # Positivo, Negativo
```

```
        'Z': ('D22030', 'D22080') # Positivo, Negativo
```

```
    }
```

```
    pos_addr_key, neg_addr_key = jog_addr_map[axis_name]
```

```
    # Verifica se mudou antes de escrever
```

```
    current_velocity = self.read_dword(self.addresses[pos_addr_key])
```

```
    if current_velocity == velocity:
```

```
        self.log(f"■ Velocidade JOG {axis_name} já está em {velocity} Hz - não alterada")
```

```
        return
```

```
    # Escreve velocidades positiva e negativa
```

```
    result1 = self.write_dword(self.addresses[pos_addr_key], velocity) # Positiva
```

```
    result2 = self.write_dword(self.addresses[neg_addr_key], -velocity) # Negativa
```

```
    # Escreve tempos de aceleração/desaceleração (comuns para todos os eixos)
```

```
    result3 = self.write_dword(self.addresses['D22100'], 1) # Tempo aceleração
```

```
    result4 = self.write_dword(self.addresses['D22110'], 1) # Tempo desaceleração
```

```
    if not result1.isError() and not result2.isError() and not result3.isError() and not result4.isError():
```

```
        self.log(f"■ Velocidade JOG {axis_name} definida: ±{velocity} Hz")
```

```
        # Verifica se foi escrito corretamente
```

```
        QTimer.singleShot(200, lambda: self.verify_jog_velocity(axis_name, velocity))
```

else:

```
self.log(f"■ Erro ao definir velocidade JOG {axis_name}")
```

except Exception as e:

```
self.log(f"■ Erro ao configurar velocidade JOG {axis_name}: {e}")
```

```
def verify_jog_velocity(self, axis_name, expected_velocity):
```

```
    """Verifica se a velocidade JOG foi escrita corretamente"""
```

```
    try:
```

```
        jog_addr_map = {
```

```
            'Y2': ('D22000', 'D22050'),
```

```
            'Y1': ('D22010', 'D22060'),
```

```
            'X': ('D22020', 'D22070'),
```

```
            'Z': ('D22030', 'D22080')
```

```
        }
```

```
        pos_addr_key, neg_addr_key = jog_addr_map[axis_name]
```

```
        actual_positive = self.read_dword(self.addresses[pos_addr_key])
```

```
        actual_negative = self.read_dword(self.addresses[neg_addr_key])
```

```
        if actual_positive == expected_velocity and actual_negative == -expected_velocity:
```

```
            self.log(f"✓ Velocidade JOG {axis_name} verificada: {pos_addr_key}={actual_positive}, {neg_addr_key}={actual_negative}")
```

```
        else:
```

```
            self.log(f"■ Velocidade JOG {axis_name} divergente: Esperado ±{expected_velocity}, Lido {actual_positive}/{actual_negative}")
```

except Exception as e:

```
self.log(f"■ Erro verificação velocidade JOG {axis_name}: {e}")
```

```
def jog_start(self, axis_name, direction):
```

```
    """Inicia movimento JOG contínuo para qualquer eixo"""
```

```
    if not self.connected:
```

```
        self.log("■ CLP não conectado")
```

```
        return
```

```
    try:
```

```
        # Configura velocidade automaticamente
```

```
        self.set_jog_velocity(axis_name)
```

```
        # Mapeamento de comandos JOG por eixo (conforme ladder real)
```

```
        jog_cmd_map = {
```

```
            'Y2': ('M70_Y2', 'M80_Y2'), # JOG+, JOG-
```

```
            'Y1': ('M570_Y1', 'M580_Y1'), # JOG+, JOG-
```

```
            'X': ('M1070_X', 'M1080_X'), # JOG+, JOG-
```

```
            'Z': ('M1570_Z', 'M1580_Z') # JOG+, JOG-
```

```
        }
```

```
        pos_cmd, neg_cmd = jog_cmd_map[axis_name]
```

```
        if direction == '+':
```

```
            result = self.client.write_coil(self.addresses[pos_cmd], True)
```

```
            direction_text = f"POSITIVO ({pos_cmd})"
```

```
            self.log(f"■ Acionando {pos_cmd} para JOG {axis_name}+")
```

```

else:
    result = self.client.write_coil(self.addresses[neg_cmd], True)
    direction_text = f"NEGATIVO ({neg_cmd})"
    self.log(f"■ Acionando {neg_cmd} para JOG {axis_name}-")

if not result.isError():
    self.log(f"■ JOG {axis_name} {direction_text} INICIADO")
    self.update_jog_status(axis_name, f"ATIVO - {direction_text}", "lime")
    # Verifica segurança apenas para informar o usuário (não bloqueia)
    self.check_jog_safety(axis_name)
else:
    self.log(f"■ Erro ao iniciar JOG {axis_name} {direction_text}")

```

```

except Exception as e:
    self.log(f"■ Erro JOG start {axis_name}: {e}")

```

```

def jog_stop(self, axis_name):
    """Para movimento JOG de qualquer eixo"""
    if not self.connected:
        return

    try:
        # Mapeamento de comandos JOG por eixo
        jog_cmd_map = {
            'Y2': ('M70_Y2', 'M80_Y2'),
            'Y1': ('M570_Y1', 'M580_Y1'),
            'X': ('M1070_X', 'M1080_X'),
            'Z': ('M1570_Z', 'M1580_Z')
        }

        pos_cmd, neg_cmd = jog_cmd_map[axis_name]

        # Para ambos os comandos JOG do eixo
        result1 = self.client.write_coil(self.addresses[pos_cmd], False)
        result2 = self.client.write_coil(self.addresses[neg_cmd], False)

        self.log(f"■ Desligando {pos_cmd} e {neg_cmd}")

        self.log(f"■ JOG {axis_name} PARADO")
        self.update_jog_status(axis_name, "PARADO", "gray")

    except Exception as e:
        self.log(f"■ Erro JOG stop {axis_name}: {e}")

```

```

def check_jog_safety(self, axis_name):
    """Monitora status de segurança JOG (apenas informativo - não bloqueia)"""
    try:
        # Mapeamento de flags de segurança por eixo
        safety_map = {
            'Y2': ('M10_Y2', 'M11_Y2'), # Limite positivo, negativo
            'Y1': ('M510_Y1', 'M511_Y1'), # Limite positivo, negativo

```

```

'X': ('M1010_X', 'M1011_X'), # Limite positivo, negativo
'Z': ('M1510_Z', 'M1511_Z') # Limite positivo, negativo
}

```

```

pos_limit_mem, neg_limit_mem = safety_map[axis_name]
result1 = self.client.read_coils(self.addresses[pos_limit_mem], count=1)
result2 = self.client.read_coils(self.addresses[neg_limit_mem], count=1)

```

```

if not result1.isError() and not result2.isError():
    positive_ok = not result1.bits[0] # Invertido: flag ativa = bloqueado
    negative_ok = not result2.bits[0] # Invertido: flag ativa = bloqueado
    safety_ok = positive_ok or negative_ok # Pelo menos uma direção liberada

```

```

safety_indicator = getattr(self, f'safety_indicator_{axis_name}', None)
if not safety_indicator:
    return safety_ok

```

```

# Atualiza indicador visual
if positive_ok and negative_ok:
    status_text = "■■■ Ambas direções OK"
    color = "green"
elif positive_ok:
    status_text = "■■■ Apenas JOG+ permitido"
    color = "orange"
elif negative_ok:
    status_text = "■■■ Apenas JOG- permitido"
    color = "orange"
else:
    status_text = "■■■ Fora dos limites"
    color = "red"

```

```

safety_indicator.setText(status_text)
safety_indicator.setStyleSheet(f"QLabel {{ background-color: {color}; color: white; padding: 3px; font-size: 10px; border: 1px solid black; })")
# Log detalhado para debug
status_key = f'_last_safety_status_{axis_name}'
current_status = (positive_ok, negative_ok)
if hasattr(self, status_key) and getattr(self, status_key) != current_status:
    if not safety_ok:
        self.log(f"■■■ {axis_name} fora dos limites - {pos_limit_mem}={not positive_ok}, {neg_limit_mem}={not negative_ok}")
    else:
        self.log(f"■■■ {axis_name} dentro dos limites - {pos_limit_mem}={not positive_ok}, {neg_limit_mem}={not negative_ok}")

```

```

setattr(self, status_key, current_status)

```

```

    return safety_ok
else:
    return False

```

```

except Exception as e:
    self.log(f"■■■ Erro ao verificar segurança JOG {axis_name}: {e}")
    return False

```

```

def update_jog_status(self, axis_name, status_text, color):
    """Atualiza status visual do JOG"""
    status_label = getattr(self, f'jog_status_label_{axis_name}', None)
    if status_label:
        status_label.setText(f"Status: {status_text}")
        status_label.setStyleSheet(f"QLabel {{ background-color: {color}; color: white; padding: 5px; font-weight: bold; border: 1px solid black; }}")

# Funções específicas para eixo X (compatibilidade com teclado)
def jog_start_x(self, direction): self.jog_start('X', direction)
def jog_stop_x(self): self.jog_stop('X')

# =====
#          CONTROLE VIA TECLADO
# =====

def on_keyboard_jog_toggle(self, state):
    """Callback quando checkbox global de controle via teclado é alterado"""
    enabled = state == Qt.CheckState.Checked.value

    if enabled:
        self.log("■■■ Controle via teclado HABILITADO para todos os eixos")
        self.log("■ Teclas: X(← →) | Y2(↑ ↓) | Y1(W S) | Z(Q E)")
        self.keyboard_jog_checkbox.setStyleSheet("QCheckBox { font-weight: bold; color: #4CAF50; }")
    else:
        self.log("■■■ Controle via teclado DESABILITADO para todos os eixos")
        self.keyboard_jog_checkbox.setStyleSheet("QCheckBox { font-weight: bold; color: #E65100; }")
        # Para qualquer movimento JOG ativo se desabilitar
        for axis in self.keyboard_jog_active:
            if self.keyboard_jog_active[axis]:
                self.jog_stop(axis)
                self.keyboard_jog_active[axis] = False

def keyPressEvent(self, event):
    """Captura teclas pressionadas"""
    # Só processa se checkbox estiver habilitado e conectado
    if not hasattr(self, 'keyboard_jog_checkbox') or not self.keyboard_jog_checkbox.isChecked() or not self.connected:
        super().keyPressEvent(event)
        return

    key = event.key()
    axis_to_start = None
    direction = None

    # Mapeamento de teclas para eixos e direções
    if key == Qt.Key.Key_Left:
        axis_to_start, direction = 'X', '-'
    elif key == Qt.Key.Key_Right:
        axis_to_start, direction = 'X', '+'
    elif key == Qt.Key.Key_Up:
        axis_to_start, direction = 'Y2', '+'

```

```

elif key == Qt.Key.Key_Down:
    axis_to_start, direction = 'Y2', '-'
elif key == Qt.Key.Key_W:
    axis_to_start, direction = 'Y1', '+'
elif key == Qt.Key.Key_S:
    axis_to_start, direction = 'Y1', '-'
elif key == Qt.Key.Key_Q:
    axis_to_start, direction = 'Z', '+'
elif key == Qt.Key.Key_E:
    axis_to_start, direction = 'Z', '-'

```

Inicia JOG se tecla válida e eixo não estiver ativo

```

if axis_to_start and not self.keyboard_jog_active[axis_to_start]:
    self.keyboard_jog_active[axis_to_start] = True
    self.jog_start(axis_to_start, direction)

```

Mapeamento de símbolos para log

```

key_symbols = {

```

```

    'X': {'+': "→", '-': "←"},

```

```

    'Y2': {'+': "↑", '-': "↓"},

```

```

    'Y1': {'+': "W", '-': "S"},

```

```

    'Z': {'+': "Q", '-': "E"}

```

```

}

```

```

symbol = key_symbols[axis_to_start][direction]

```

```

self.log(f"■■■ JOG {axis_to_start}{direction} iniciado via teclado ({symbol})")

```

```

event.accept()

```

```

return

```

Outras teclas passam para o comportamento padrão

```

super().keyPressEvent(event)

```

```

def keyReleaseEvent(self, event):

```

```

    """Captura teclas soltas"""

```

Só processa se checkbox estiver habilitado e conectado

```

if not hasattr(self, 'keyboard_jog_checkbox') or not self.keyboard_jog_checkbox.isChecked() or not self.connected:

```

```

    super().keyReleaseEvent(event)

```

```

    return

```

```

key = event.key()

```

```

axis_to_stop = None

```

```

direction = None

```

Mapeamento de teclas para eixos

```

if key == Qt.Key.Key_Left:

```

```

    axis_to_stop, direction = 'X', '-'

```

```

elif key == Qt.Key.Key_Right:

```

```

    axis_to_stop, direction = 'X', '+'

```

```

elif key == Qt.Key.Key_Up:

```

```

    axis_to_stop, direction = 'Y2', '+'

```

```

elif key == Qt.Key.Key_Down:

```

```

    axis_to_stop, direction = 'Y2', '-'

```

```

elif key == Qt.Key.Key_W:
    axis_to_stop, direction = 'Y1', '+'
elif key == Qt.Key.Key_S:
    axis_to_stop, direction = 'Y1', '-'
elif key == Qt.Key.Key_Q:
    axis_to_stop, direction = 'Z', '+'
elif key == Qt.Key.Key_E:
    axis_to_stop, direction = 'Z', '-'

# Para JOG se tecla válida e eixo estiver ativo
if axis_to_stop and self.keyboard_jog_active[axis_to_stop]:
    self.keyboard_jog_active[axis_to_stop] = False
    self.jog_stop(axis_to_stop)

# Mapeamento de símbolos para log
key_symbols = {
    'X': {'+': "→", '-': "←"},
    'Y2': {'+': "↑", '-': "↓"},
    'Y1': {'+': "W", '-': "S"},
    'Z': {'+': "Q", '-': "E"}
}
symbol = key_symbols[axis_to_stop][direction]
self.log(f"■■ JOG {axis_to_stop}{direction} parado via teclado ({symbol})")
event.accept()
return

# Outras teclas passam para o comportamento padrão
super().keyReleaseEvent(event)

```

```

# -----
# NOVO: pulso momentâneo em M5000 (Y0.10) – “falling-edge trigger”
# -----

def pulse_y010(self):
    """Dispara um pulso de 100 ms em M5000 (Y0.10) e solta o botão."""
    if not self.connected:
        self.log("■ CLP não conectado")
        return

    try:
        # Aciona o coil
        self.log("■üèÑ Pulso M5000 (SHOT)")
        result = self.client.write_coil(self.addresses['M5000'], True)
        if result.isError():
            self.log(f"üâ Erro ao escrever M5000: {result}")
        else:
            # Mantém ON por 100 ms e depois desliga (falling-edge)
            QTimer.singleShot(100,
                              lambda: self.client.write_coil(self.addresses['M5000'], False))
            # Libera o botão na interface um pouco depois
            QTimer.singleShot(120,
                              lambda: self.y010_btn.setChecked(False))
    
```

```
self.log("úÖ Pulso M5000 enviado (100 ms)")
```

```
except Exception as e:
```

```
self.log(f"■ Erro Y0.10: {e}")
```

```
def pulse_m100(self):
```

```
    """Pulso em M100"""
```

```
    if not self.connected:
```

```
        return
```

```
    try:
```

```
        self.client.write_coil(self.addresses['M100'], True)
```

```
        QTimer.singleShot(100, lambda: self.client.write_coil(self.addresses['M100'], False))
```

```
        self.log("M100 pulsado")
```

```
    except Exception as e:
```

```
        self.log(f"■ Erro M100: {e}")
```

```
def emergency_stop(self):
```

```
    """Parada de emergência"""
```

```
    if not self.connected:
```

```
        return
```

```
    try:
```

```
        self.log("■ PARADA DE EMERGÊNCIA!")
```

```
    # Desliga todas as memórias importantes
```

```
    emergency_memories = [
```

```
        'M0_Y2', 'M50_Y2', 'M100_Y2', 'M500_Y1', 'M550_Y1', 'M600_Y1',
```

```
        'M1500_Z', 'M1550_Z', 'M1600_Z', 'M1000_X', 'M1050_X', 'M1100_X',
```

```
        'M5000', 'M350', 'M850', 'M1350', 'M1850',
```

```
        'M70_Y2', 'M80_Y2', 'M570_Y1', 'M580_Y1', 'M1070_X', 'M1080_X', 'M1570_Z', 'M1580_Z'
```

```
    ]
```

```
    for mem in emergency_memories:
```

```
        self.client.write_coil(self.addresses[mem], False)
```

```
    # Para JOG de todos os eixos
```

```
    for axis in ['Y2', 'Y1', 'X', 'Z']:
```

```
        self.jog_stop(axis)
```

```
    # Lê posições uma última vez antes de parar
```

```
    self.read_current_positions()
```

```
    # Para JOG via teclado se estiver ativo para qualquer eixo
```

```
    if hasattr(self, 'keyboard_jog_active'):
```

```
        for axis in self.keyboard_jog_active:
```

```
            if self.keyboard_jog_active[axis]:
```

```
                self.keyboard_jog_active[axis] = False
```

```
                self.jog_stop(axis)
```

```
        self.log("■ JOG via teclado interrompido por emergência")
```

```
    # Desliga o botão Y0.10 na interface
```

```

self.y010_btn.setChecked(False)

self.log("■ Todos os comandos desligados")

except Exception as e:
    self.log(f"■ Erro na parada: {e}")

# =====
# callbacks vindos do MovementControlsWidget
# =====
def _on_step_move_requested(self, axis: str, distance: float, feed: float):
    """Step = movimento incremental curto, em milímetros."""
    pulses = self.pulses_from_mm(axis, distance)
    self.move_relative(axis, pulses)

def _on_widget_jog_start(self, axis: str, direction: int, feed: float):
    self._active_widget_axis = axis
    self.jog_start(axis, '+' if direction > 0 else '-')

def _on_widget_jog_stop(self):
    if hasattr(self, '_active_widget_axis'):
        self.jog_stop(self._active_widget_axis)

def _on_go_to_zero(self):
    # usa homing (já implementado) para Z primeiro e depois Y2,Y1,X
    self.home_all_axes()

# =====
# zerar posição individual (M0/M500/...)
# =====
def zero_axis(self, axis: str):
    """Pulsa a memória M0/M500/M1000/M1500 conforme eixo."""
    mem_map = {'Y2': 'M0_Y2', 'Y1': 'M500_Y1', 'X': 'M1000_X', 'Z': 'M1500_Z'}
    if axis in mem_map:
        self._pulse_coil(mem_map[axis])
        self.log(f"■ Zero absoluto do eixo {axis} solicitado")

def update_status(self):
    """Atualiza status em tempo real"""
    if not self.connected:
        return

    # Lê posições atuais e status de homing (prioridade)
    self.read_current_positions()
    # ----- atualiza MovementControls principal -----
    if hasattr(self, 'mov_widget'):
        self.mov_widget.update_position(
            x=self.current_positions['X'],
            y2=self.current_positions['Y2'],
            y1=self.current_positions['Y1'],
            z=self.current_positions['Z'])

```

```

# ----- atualiza widgets das mesas -----
for mesa_id, tab in getattr(self, 'mesa_tabs', {}).items():
    # envia posição para cada mov_widget da mesa
    if tab.y_axis == 'Y1':
        tab.mov_widget.update_position(
            x=self.current_positions['X'],
            y2=0,
            y1=self.current_positions['Y1'],
            z=self.current_positions['Z'])
    else:
        tab.mov_widget.update_position(
            x=self.current_positions['X'],
            y2=self.current_positions['Y2'],
            y1=0,
            z=self.current_positions['Z'])
self.monitor_homing_status()
# ----- atualiza gráficos das mesas -----
if hasattr(self, "mesa_plot_widgets"):
    self.mesa_plot_widgets[1].update_head_position(
        self.current_positions['X'],
        self.current_positions['Y1'])
    self.mesa_plot_widgets[2].update_head_position(
        self.current_positions['X'],
        self.current_positions['Y2'])
# ajusta foco conforme Z
self.camera_manager.auto_focus(self.current_positions['Z'])

try:
    # Atualiza saídas
    outputs_map = {
        'Y0_0': ('Y00', self.Y0_0_status),
        'Y0_1': ('Y01', self.Y0_1_status),
        'Y0_4': ('Y04', self.Y0_4_status),
        'Y0_5': ('Y05', self.Y0_5_status),
        'Y0_2': ('Y02', self.Y0_2_status),
        'Y0_3': ('Y03', self.Y0_3_status),
        'Y0_6': ('Y06', self.Y0_6_status),
        'Y0_7': ('Y07', self.Y0_7_status),
        'Y0_10': ('Y010', self.Y0_10_status)
    }

    for name, (addr_key, label) in outputs_map.items():
        # Y = coils → FC01 (read_coils)
        result = self.client.read_coils(self.addresses[addr_key], count=1)
        if not result.isError():
            state = result.bits[0]
            label.setText("ON" if state else "OFF")
            label.setStyleSheet(
                "QLabel { background-color: lime; color: black; padding: 5px; }" if state
                else "QLabel { background-color: gray; color: white; padding: 5px; }"

```

```

)

# Sincroniza botão Y0.10 com o estado real
if name == 'Y0_10':
    if self.y010_btn.isChecked() != state:
        self.y010_btn.setChecked(state)

# Atualiza memórias
memories_map = {
    # Atualizado conforme novos endereços
    'M0_Y2': (self.addresses['M0_Y2'], getattr(self, 'M0_Y2_status', None)),
    'M50_Y2': (self.addresses['M50_Y2'], getattr(self, 'M50_Y2_status', None)),
    'M500_Y1': (self.addresses['M500_Y1'], getattr(self, 'M500_Y1_status', None)),
    'M550_Y1': (self.addresses['M550_Y1'], getattr(self, 'M550_Y1_status', None)),
    # Disparo e status da aplicadora
    'M5000': (self.addresses['M5000'], getattr(self, 'M5000_status', None)),
    'M5001': (self.addresses['M5001'], getattr(self, 'M5001_status', None)),

    'M350': (self.addresses['M350'], getattr(self, 'M350_status', None)),
    'M850': (self.addresses['M850'], getattr(self, 'M850_status', None)),
    'M1350': (self.addresses['M1350'], getattr(self, 'M1350_status', None)),
    'M1850': (self.addresses['M1850'], getattr(self, 'M1850_status', None)),
    # MEMÓRIAS DE HOMING REALIZADO - PRIORITÁRIAS
    'M300': (self.addresses['M300'], getattr(self, 'M300_status', None)),
    'M800': (self.addresses['M800'], getattr(self, 'M800_status', None)),
    'M1300': (self.addresses['M1300'], getattr(self, 'M1300_status', None)),
    'M1800': (self.addresses['M1800'], getattr(self, 'M1800_status', None)),
    # Monitoramento JOG (apenas alguns principais)
    'M70_Y2': (self.addresses['M70_Y2'], getattr(self, 'M70_Y2_status', None)),
    'M570_Y1': (self.addresses['M570_Y1'], getattr(self, 'M570_Y1_status', None)),
    'M1070_X': (self.addresses['M1070_X'], getattr(self, 'M1070_X_status', None)),
    'M1570_Z': (self.addresses['M1570_Z'], getattr(self, 'M1570_Z_status', None))
}

for name, (addr, label) in memories_map.items():
    if name in ['M300', 'M800', 'M1300', 'M1800']:
        continue # Pula memórias de homing (tratadas em função dedicada)

    if label is None:
        continue
    result = self.client.read_coils(addr, count=1)
    if not result.isError():
        state = result.bits[0]
        # Atualiza label se existir
        if label is not None:
            label.setText("ON" if state else "OFF")
            # Para M5001 usamos cor laranja enquanto ativo
            if name == 'M5001':
                label.setStyleSheet(
                    "QLabel { background-color: orange; color: black; padding: 5px; }" if state
                    else "QLabel { background-color: gray; color: white; padding: 5px; }"

```

```

    )
else:
    label.setStyleSheet(
        "QLabel { background-color: red; color: white; padding: 5px; }" if state
        else "QLabel { background-color: gray; color: white; padding: 5px; }"
    )

# Atualiza status JOG ativo para todos os eixos
if name in ['M70_Y2', 'M570_Y1', 'M1070_X', 'M1570_Z']:
    jog_status_map = {
        'M70_Y2': ('Y2', 'POSITIVO'),
        'M570_Y1': ('Y1', 'POSITIVO'),
        'M1070_X': ('X', 'POSITIVO'),
        'M1570_Z': ('Z', 'POSITIVO')
    }

    if name in jog_status_map and not result.isError():
        axis, direction = jog_status_map[name]
        status_label = getattr(self, f'jog_status_label_{axis}', None)
        if status_label:
            if state:
                self.update_jog_status(axis, f"ATIVO - {direction}", "lime")
            elif not state and direction in status_label.text():
                self.update_jog_status(axis, "Inativo", "gray")

# Atualiza segurança JOG em tempo real apenas quando necessário
if name in ['M10_Y2', 'M11_Y2', 'M510_Y1', 'M511_Y1', 'M1010_X', 'M1011_X', 'M1510_Z', 'M1511_Z']:
    axis_map = {
        'M10_Y2': 'Y2', 'M11_Y2': 'Y2',
        'M510_Y1': 'Y1', 'M511_Y1': 'Y1',
        'M1010_X': 'X', 'M1011_X': 'X',
        'M1510_Z': 'Z', 'M1511_Z': 'Z'
    }

    if name in axis_map:
        axis = axis_map[name]
        QTimer.singleShot(50, lambda a=axis: self.check_jog_safety(a))

# Atualiza registradores
registers_map = {
    # Registradores dos eixos
    'D100_Y2': (self.addresses['D100_Y2'], getattr(self, 'D100_Y2_status', None)),
    'D20000_Y2': (self.addresses['D20000_Y2'], getattr(self, 'D20000_Y2_status', None)),
    'D1600_Z': (self.addresses['D1600_Z'], getattr(self, 'D1600_Z_status', None)),
    'D21500_Z': (self.addresses['D21500_Z'], getattr(self, 'D21500_Z_status', None)),
    'D1100_X': (self.addresses['D1100_X'], getattr(self, 'D1100_X_status', None)),
    'D21000_X': (self.addresses['D21000_X'], getattr(self, 'D21000_X_status', None))
}

# ----- SENSORES HOME -----
sensors_map = {
    'X04_Y2': ('X04_Y2_status', self.addresses['X04_Y2']),

```

```

'X06_Y1': ('X06_Y1_status', self.addresses['X06_Y1']),
'X08_X': ('X08_X_status', self.addresses['X08_X']),
'X09_Z': ('X09_Z_status', self.addresses['X09_Z'])
}

```

```

for sensor_key, (label_attr, addr) in sensors_map.items():
    lbl = getattr(self, label_attr, None)
    if lbl is None:
        continue
    # X = discrete inputs → FC02 (read_discrete_inputs)
    res = self.client.read_discrete_inputs(addr, count=1)
    if not res.isError():
        state = res.bits[0]
        lbl.setText("ON" if state else "OFF")
        lbl.setStyleSheet(
            "QLabel { background-color: lime; color: black; padding: 5px; }" if state
            else "QLabel { background-color: gray; color: white; padding: 5px; }"
        )

```

```

for name, (addr, label) in registers_map.items():
    try:
        # Todos agora são registradores D normais
        signed = self.read_dword(addr)

        if label is not None:
            label.setText(str(signed))
            # cor laranja se negativo
            label.setStyleSheet(
                "QLabel { background-color: orange; padding: 5px; }"
                if signed < 0 else
                "QLabel { background-color: lightgray; padding: 5px; }"
            )
        except Exception as e:
            self.log(f"■■■ Erro ao ler registrador {name}: {e}")
    except Exception as e:
        self.log(f"■■■ Erro ao atualizar status: {e}")

```

```

# -----
# Fornece posição atual para o widget de inspeção
# -----

```

```

def _get_current_position_dict(self) -> dict:
    """
    Retorna dict {x,y2,y1,z} em pulsos – usado pelo
    InspectionPositionsWidget para 'Adicionar posição atual'.
    """
    return {
        "x": self.current_positions.get("X", 0),
        "y2": self.current_positions.get("Y2", 0),
        "y1": self.current_positions.get("Y1", 0),
        "z": self.current_positions.get("Z", 0),
    }

```

```

# -----
# converte lista do widget para InspectionPosition do sequence_control
# -----
def _positions_to_model(self):
    lst = []
    for p in self.inspect_widget.positions():
        lst.append(
            InspectionPosition(
                name=p.name,
                x=p.x, y2=p.y2, y1=p.y1, z=p.z
            )
        )
    return lst

def force_homing_status_check(self):
    """Força uma verificação imediata do status de homing"""
    try:
        self.log("■ Verificação forçada do status de homing...")
        self.monitor_homing_status()
    except Exception as e:
        self.log(f"■ Erro na verificação forçada: {e}")

def update_homing_led(self, led_widget, homing_status):
    """Atualiza o indicador LED de homing"""
    if homing_status:
        # Homing realizado - LED verde
        led_widget.setStyleSheet("QLabel { background-color: lime; color: darkgreen; padding: 2px; font-size: 16px; font-weight: bold; }")
        led_widget.setText("●")
    else:
        # Homing não realizado - LED cinza
        led_widget.setStyleSheet("QLabel { background-color: gray; color: darkgray; padding: 2px; font-size: 16px; font-weight: bold; }")
        led_widget.setText("●")

def test_position_reading(self):
    """Função de teste para verificar leitura das posições"""
    if not self.connected:
        return

    self.log("■ Testando leitura das posições atuais...")

    # Usa a função dedicada e exibe resultados
    self.read_current_positions()

    # Log das posições lidas
    for axis, position in self.current_positions.items():
        self.log(f"■ Posição {axis}: {position}")
    self.test_homing_status()

def closeEvent(self, event):

```

```
""""Fecha conexão ao sair""""
```

```
if self.client:
```

```
    self.emergency_stop()
```

```
    self.client.close()
```

```
    self.status_bar.showMessage("Desconectado")
```

```
    self.status_bar.setStyleSheet(
```

```
        "QStatusBar { background-color:red; color:white; font-weight:bold; }")
```

```
    self.log("Desconectado do CLP")
```

```
    event.accept()
```

```
# -----
```

```
# BACK-END DE MOVIMENTO para o SequenceControlWidget
```

```
# -----
```

```
class PLCMotionBackend(MotionBackend):
```

```
    """
```

Adaptador simples que converte as chamadas do SequenceControlWidget para os métodos já existentes do MultiAxisMotorController.

```
    """
```

```
def __init__(self, ctrl: MultiAxisMotorController):
```

```
    self._c = ctrl
```

```
    self._off_x = ctrl.camera_nozzle_offset.get("x", 0)
```

```
    self._off_y = ctrl.camera_nozzle_offset.get("y", 0)
```

```
    self._feed = 1000
```

```
    self._apply_offset = True    # default = APPLY
```

```
    self._targets: dict[str,int] = {} # destino mais recente por eixo
```

```
    # registrador de QUANTIDADE de dots
```

```
    self._addr_qty = ctrl.addresses.get('D24100_QTY')
```

```
    # coils para dot
```

```
    self._addr_trig = ctrl.addresses.get('M5000')
```

```
    self._addr_stat = ctrl.addresses.get('M5001')
```

```
# -----
```

```
def move_to_absolute_position(self,
```

```
    x: float | None,
```

```
    y2: float | None,
```

```
    y1: float | None,
```

```
    z: float | None,
```

```
    feed_rate: float = 1000) -> bool:
```

```
    self._feed = feed_rate
```

```
# -----
```

```
# LIMPA destinos da execução anterior ← BUG FIX travamento 2ª run
```

```
# -----
```

```
    self._targets.clear()
```

```
    try:
```

```
        # Escreve apenas eixos cujo valor não é None
```

```
        dx = self._off_x if self._apply_offset else 0
```

```
        dy = self._off_y if self._apply_offset else 0
```

```
        if x is not None: self._move_axis('X', int(x) - dx)
```

```
        if y2 is not None: self._move_axis('Y2', int(y2) - dy)
```

```
        if y1 is not None: self._move_axis('Y1', int(y1) - dy)
```

```

        if z is not None: self._move_axis('Z', int(z))
        return True
    except Exception as exc:
        print(f"Erro move_to_absolute_position: {exc}")
        self._c.log(f"Erro move_to_abs: {exc}")
        return False

# -----
# NOVO: muda modo de offset (True = APPLY, False = VIEW)
# -----
def set_offset_mode(self, apply_offset: bool):
    self._apply_offset = bool(apply_offset)

def _move_axis(self, axis: str, pulses: int):
    spin = getattr(self._c, f"pulsos_spin_{axis}")
    spin.setValue(pulses)
    self._c.move_axis_absolute(axis)
    self._targets[axis] = pulses
    # Não bloqueia aqui – wait_for_idle fará polling

# -----
def wait_for_idle(self) -> bool:
    """Bloqueia até que cada eixo alcance o pulso alvo  $\pm 1$ ."""
    t0 = time.time()
    timeout = 120      # s – eixo pode percorrer longas distâncias
    ok_axes = set()

    pos_regs = {      # registradores D que espelham SR
        'Y2': self._c.addresses['D3100_Y2'],
        'Y1': self._c.addresses['D3400_Y1'],
        'X' : self._c.addresses['D3000_X'],
        'Z' : self._c.addresses['D3200_Z'],
    }

    while time.time() - t0 < timeout:
        all_reached = True
        for axis, target in self._targets.items():
            if axis in ok_axes:
                continue

            try:
                pos = self._c.read_dword(pos_regs[axis])
            except Exception as exc:
                self._c.log(f"Erro ao ler posição do eixo {axis}: {exc}")
                pos = None

            if pos is None or abs(pos - target) > 1:
                all_reached = False
            else:
                ok_axes.add(axis)

```

```

        if all_reached:
            return True
        time.sleep(0.05)

    self._c.log("wait_for_idle: timeout atingido")
    return False

# -----
def set_feed_rate(self, fr: float):
    self._feed = fr

# -----
# DOT – grava SOMENTE a quantidade (D24100)
# -----
def apply_dot_qty(self, qty: int):
    if not self._c.connected:
        return
    try:
        if self._addr_qty is not None:
            self._c.write_dword(self._addr_qty, int(qty))
            self._c.log(f"■ DOT qty → D24100={qty}")
    except Exception as exc:
        self._c.log(f"■ Erro ao escrever DOT cfg: {exc}")

# -----
# DISPARO DO DOT (pulso M5000) + espera M5001
# -----
def trigger_dot(self, pulse_ms: int = 100) -> bool:
    """
    Gera um pulso no coil M5000. Retorna True se escrito com sucesso.
    """
    if not self._c.connected or self._addr_trig is None:
        return False
    try:
        self._c.client.write_coil(self._addr_trig, True)
        time.sleep(pulse_ms / 1000.0)
        self._c.client.write_coil(self._addr_trig, False)
        self._c.log("■ Pulso M5000 enviado")
        return True
    except Exception as exc:
        self._c.log(f"■ Erro no pulso M5000: {exc}")
        return False

def wait_dot_complete(self, timeout: float = 10.0) -> bool:
    """
    Aguarda M5001 ligar (start) e desligar (fim) dentro do timeout.
    """
    if not self._c.connected or self._addr_stat is None:
        return True # nada a esperar – não bloqueia
    t0 = time.time()
    # espera LIGAR

```

```

while time.time() - t0 < timeout:
    try:
        r = self._c.client.read_coils(self._addr_stat, count=1)
        if not r.isError() and r.bits[0]:
            break
    except Exception as exc:
        self._c.log(f"Erro ao esperar M5001=ON: {exc}")
        pass
    time.sleep(0.05)
else:
    self._c.log("■ Timeout esperando M5001=ON")
    return False

```

espera DESLIGAR

```

while time.time() - t0 < timeout:
    try:
        r = self._c.client.read_coils(self._addr_stat, count=1)
        if not r.isError() and not r.bits[0]:
            self._c.log("■ Dot concluído (M5001=OFF)")
            return True
    except Exception as exc:
        self._c.log(f"Erro ao esperar M5001=OFF: {exc}")
        pass
    time.sleep(0.05)
self._c.log("■ Timeout esperando M5001=OFF")
return False

```

def main():

```

app = QApplication(sys.argv)
app.setStyle('Fusion')

```

```

window = MultiAxisMotorController()
window.showMaximized() # abre já maximizado

```

```

print("="*60)
print("CONTROLE MULTI-EIXOS - DELTA AS (ENDEREÇOS LADDER REAIS)")
print("="*60)
print("■ MELHORIAS IMPLEMENTADAS:")
print("• Endereços corrigidos conforme planilha do ladder real")
print("• Controle de 4 eixos: Y2, Y1, Z, X")
print("• Botões HOME corrigidos: M350, M1350, M1850")
print("• Posições atuais via D3000, D3100, D3200")
print("• Indicadores LED de homing e leitura automática de velocidades")
print("• Sistema JOG integrado para TODOS os eixos com segurança")
print("✓ Controle via teclado para TODOS os eixos:")
print("• X: ← → • Y2: ↑ ↓ • Y1: W S • Z: Q E")
print("• Movimentos absolutos com verificação de limites")
print("• Configurações de homing e limites salvos em ROM")
print("• Interface redesenhada para melhor usabilidade")
print("• Leitura em tempo real das posições dos motores corrigida")
print("• Status inicial e atualização automática dos LEDs de homing")
print("="*60)

```

```
sys.exit(app.exec())

if __name__ == '__main__':
    main()
```

File: programa_exemplo.txt

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/teste_clp/Projetos

```
[
  {
    "name": "1",
    "x": 45230.0,
    "y": 363.0,
    "z": 700.0,
    "meta": {
      "action": "fiducial",
      "window": 50,
      "radius": 80,
      "threshold": 80,
      "template_png_b64": "iVBORw0KGgoAAAANSUhEUgAAADIAAAAYCAIAAACR XR/mAAARx0IEQVRYCTXBS6ymV3YV
    }
  },
  {
    "name": "2",
    "x": 32262.0,
    "y": 14937.0,
    "z": 700.0,
    "meta": {
      "action": "fiducial",
      "window": 50,
      "radius": 80,
      "threshold": 80,
      "template_png_b64": "iVBORw0KGgoAAAANSUhEUgAAADIAAAAYCAIAAACR XR/mAAATKUIEQVRYCTXBa6xmV3kf
    }
  },
  {
    "name": "3",
    "x": 39109.0,
    "y": 3625.0,
    "z": 700.0,
    "meta": {
      "action": "barcode",
      "width": 200,
      "height": 400
    }
  },
  {
    "name": "4",
```

```
"x": 41396.0,
"y": 8810.0,
"z": 700.0,
"meta": {
  "action": "dot",
  "dot_id": 2,
  "dot_name": "processador",
  "dot_qty": 100
}
},
{
  "name": "5",
  "x": 42178.0,
  "y": 8484.0,
  "z": 700.0,
  "meta": {
    "action": "dot",
    "dot_id": 2,
    "dot_name": "processador",
    "dot_qty": 100
  }
},
{
  "name": "6",
  "x": 43280.0,
  "y": 8478.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 2,
    "dot_name": "processador",
    "dot_qty": 100
  }
},
{
  "name": "7",
  "x": 44089.0,
  "y": 8843.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 2,
    "dot_name": "processador",
    "dot_qty": 100
  }
},
{
  "name": "8",
  "x": 44108.0,
  "y": 9602.0,
  "z": 1000.0,
```

```
"meta": {
  "action": "dot",
  "dot_id": 2,
  "dot_name": "processador",
  "dot_qty": 100
}
},
{
  "name": "9",
  "x": 43260.0,
  "y": 9987.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 2,
    "dot_name": "processador",
    "dot_qty": 100
  }
},
{
  "name": "10",
  "x": 42250.0,
  "y": 9979.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 2,
    "dot_name": "processador",
    "dot_qty": 100
  }
},
{
  "name": "11",
  "x": 41406.0,
  "y": 9606.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 2,
    "dot_name": "processador",
    "dot_qty": 100
  }
},
{
  "name": "12",
  "x": 42770.0,
  "y": 9235.0,
  "z": 0.0,
  "meta": {
    "action": "inspect",
    "componentes": {
```

```
"UH1": {
  "posicao": [
    149,
    67
  ],
  "dimensoes": [
    319,
    341
  ],
  "inspecoes": [
    {
      "nome": "w1",
      "posicao": [
        13,
        10
      ],
      "tamanho": [
        94,
        94
      ],
      "similaridade": 0.95
    },
    {
      "nome": "w2",
      "posicao": [
        202,
        6
      ],
      "tamanho": [
        95,
        101
      ],
      "similaridade": 0.95
    },
    {
      "nome": "w3",
      "posicao": [
        12,
        227
      ],
      "tamanho": [
        96,
        106
      ],
      "similaridade": 0.95
    },
    {
      "nome": "w4",
      "posicao": [
        187,
        215
      ],
```

```
    ],
    "tamanho": [
      117,
      111
    ],
    "similaridade": 0.95
  }
]
}
}
}
},
{
  "name": "13",
  "x": 31740.0,
  "y": 7897.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
{
  "name": "14",
  "x": 32054.0,
  "y": 7751.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
{
  "name": "15",
  "x": 32996.0,
  "y": 7740.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
{
  "name": "16",
  "x": 33317.0,
```

```
"y": 7904.0,  
"z": 1000.0,  
"meta": {  
  "action": "dot",  
  "dot_id": 1,  
  "dot_name": "memoria",  
  "dot_qty": 50  
}  
},  
{  
  "name": "17",  
  "x": 33346.0,  
  "y": 8265.0,  
  "z": 1000.0,  
  "meta": {  
    "action": "dot",  
    "dot_id": 1,  
    "dot_name": "memoria",  
    "dot_qty": 50  
  }  
},  
{  
  "name": "18",  
  "x": 32904.0,  
  "y": 8436.0,  
  "z": 1000.0,  
  "meta": {  
    "action": "dot",  
    "dot_id": 1,  
    "dot_name": "memoria",  
    "dot_qty": 50  
  }  
},  
{  
  "name": "19",  
  "x": 32098.0,  
  "y": 8443.0,  
  "z": 1000.0,  
  "meta": {  
    "action": "dot",  
    "dot_id": 1,  
    "dot_name": "memoria",  
    "dot_qty": 50  
  }  
},  
{  
  "name": "20",  
  "x": 31748.0,  
  "y": 8279.0,  
  "z": 1000.0,  
  "meta": {
```

```
"action": "dot",
"dot_id": 1,
"dot_name": "memoria",
"dot_qty": 50
},
{
"name": "21",
"x": 31755.0,
"y": 8731.0,
"z": 1000.0,
"meta": {
"action": "dot",
"dot_id": 1,
"dot_name": "memoria",
"dot_qty": 50
},
},
{
"name": "22",
"x": 32076.0,
"y": 8574.0,
"z": 1000.0,
"meta": {
"action": "dot",
"dot_id": 1,
"dot_name": "memoria",
"dot_qty": 50
},
},
{
"name": "23",
"x": 32882.0,
"y": 8570.0,
"z": 1000.0,
"meta": {
"action": "dot",
"dot_id": 1,
"dot_name": "memoria",
"dot_qty": 50
},
},
{
"name": "24",
"x": 33332.0,
"y": 8698.0,
"z": 1000.0,
"meta": {
"action": "dot",
"dot_id": 1,
"dot_name": "memoria",
```

```
"dot_qty": 50
}
},
{
  "name": "25",
  "x": 33368.0,
  "y": 9059.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
{
  "name": "26",
  "x": 32918.0,
  "y": 9252.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
{
  "name": "27",
  "x": 32140.0,
  "y": 9259.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
{
  "name": "28",
  "x": 31748.0,
  "y": 9073.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
}
```

```
{
  "name": "29",
  "x": 33539.0,
  "y": 9493.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
{
  "name": "30",
  "x": 33853.0,
  "y": 9347.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
{
  "name": "31",
  "x": 34524.0,
  "y": 9343.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
{
  "name": "32",
  "x": 34895.0,
  "y": 9489.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
{
  "name": "33",
  "x": 34924.0,
```

```
"y": 9937.0,
"z": 1000.0,
"meta": {
  "action": "dot",
  "dot_id": 1,
  "dot_name": "memoria",
  "dot_qty": 50
}
},
{
  "name": "34",
  "x": 34524.0,
  "y": 10094.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
{
  "name": "35",
  "x": 33889.0,
  "y": 10090.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
{
  "name": "36",
  "x": 33518.0,
  "y": 9959.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
{
  "name": "37",
  "x": 35095.0,
  "y": 9481.0,
  "z": 1000.0,
  "meta": {
```

```
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
{
  "name": "38",
  "x": 35495.0,
  "y": 9332.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
{
  "name": "39",
  "x": 36123.0,
  "y": 9321.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
{
  "name": "40",
  "x": 36523.0,
  "y": 9481.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
{
  "name": "41",
  "x": 36573.0,
  "y": 9919.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
```

```
"dot_qty": 50
}
},
{
  "name": "42",
  "x": 36188.0,
  "y": 10094.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
{
  "name": "43",
  "x": 35553.0,
  "y": 10094.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
{
  "name": "44",
  "x": 35161.0,
  "y": 9959.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
{
  "name": "45",
  "x": 35875.0,
  "y": 9274.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
}
```

```
{
  "name": "46",
  "x": 36531.0,
  "y": 8880.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
{
  "name": "47",
  "x": 36538.0,
  "y": 8118.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
{
  "name": "48",
  "x": 35810.0,
  "y": 7702.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
{
  "name": "49",
  "x": 34247.0,
  "y": 7698.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
{
  "name": "50",
  "x": 33434.0,
```

```
"y": 8095.0,
"z": 1000.0,
"meta": {
  "action": "dot",
  "dot_id": 1,
  "dot_name": "memoria",
  "dot_qty": 50
}
},
{
  "name": "51",
  "x": 33448.0,
  "y": 8886.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
{
  "name": "52",
  "x": 34290.0,
  "y": 9276.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 1,
    "dot_name": "memoria",
    "dot_qty": 50
  }
},
{
  "name": "53",
  "x": 34090.0,
  "y": 8929.0,
  "z": 0.0,
  "meta": {
    "action": "inspect",
    "componentes": {
      "UV1": {
        "posicao": [
          234,
          0
        ],
        "dimensoes": [
          308,
          305
        ],
        "inspecoes": [
```

```
{
  "nome": "w1",
  "posicao": [
    3,
    3
  ],
  "tamanho": [
    76,
    80
  ],
  "similaridade": 0.95
},
{
  "nome": "w2",
  "posicao": [
    213,
    1
  ],
  "tamanho": [
    92,
    100
  ],
  "similaridade": 0.95
},
{
  "nome": "w3",
  "posicao": [
    220,
    209
  ],
  "tamanho": [
    85,
    91
  ],
  "similaridade": 0.95
},
{
  "nome": "w4",
  "posicao": [
    2,
    210
  ],
  "tamanho": [
    89,
    88
  ],
  "similaridade": 0.95
}
]
},
"UV12": {
```

```
"posicao": [
  75,
  1
],
"dimensoes": [
  183,
  152
],
"inspecoes": [
  {
    "nome": "w1",
    "posicao": [
      2,
      1
    ],
    "tamanho": [
      56,
      55
    ],
    "similaridade": 0.95
  },
  {
    "nome": "w2",
    "posicao": [
      110,
      1
    ],
    "tamanho": [
      66,
      63
    ],
    "similaridade": 0.95
  },
  {
    "nome": "w3",
    "posicao": [
      111,
      90
    ],
    "tamanho": [
      66,
      53
    ],
    "similaridade": 0.95
  },
  {
    "nome": "w4",
    "posicao": [
      2,
      91
    ],
```

```
    "tamanho": [
      59,
      55
    ],
    "similaridade": 0.95
  }
]
},
"UV11": {
  "posicao": [
    71,
    150
  ],
  "dimensoes": [
    189,
    159
  ],
  "inspecoes": [
    {
      "nome": "w1",
      "posicao": [
        5,
        1
      ],
      "tamanho": [
        73,
        60
      ],
      "similaridade": 0.95
    },
    {
      "nome": "w2",
      "posicao": [
        107,
        1
      ],
      "tamanho": [
        75,
        66
      ],
      "similaridade": 0.95
    },
    {
      "nome": "w3",
      "posicao": [
        107,
        85
      ],
      "tamanho": [
        76,
        70
      ],
      "similaridade": 0.95
    }
  ]
}
```

```
    ],
    "similaridade": 0.95
  },
  {
    "nome": "w4",
    "posicao": [
      5,
      84
    ],
    "tamanho": [
      72,
      69
    ],
    "similaridade": 0.95
  }
]
},
"UV14": {
  "posicao": [
    252,
    280
  ],
  "dimensoes": [
    144,
    177
  ],
  "inspecoes": [
    {
      "nome": "w1",
      "posicao": [
        3,
        17
      ],
      "tamanho": [
        41,
        61
      ],
      "similaridade": 0.95
    },
    {
      "nome": "w2",
      "posicao": [
        79,
        18
      ],
      "tamanho": [
        57,
        56
      ],
      "similaridade": 0.95
    }
  ],
}
```

```
{
  "nome": "w3",
  "posicao": [
    80,
    112
  ],
  "tamanho": [
    59,
    59
  ],
  "similaridade": 0.95
},
{
  "nome": "w4",
  "posicao": [
    4,
    109
  ],
  "tamanho": [
    52,
    65
  ],
  "similaridade": 0.95
}
]
},
"UV13": {
  "posicao": [
    391,
    279
  ],
  "dimensoes": [
    156,
    179
  ],
  "inspecoes": [
    {
      "nome": "w1",
      "posicao": [
        3,
        13
      ],
      "tamanho": [
        61,
        60
      ],
      "similaridade": 0.95
    },
    {
      "nome": "w2",
      "posicao": [
```

```

        77,
        13
    ],
    "tamanho": [
        72,
        62
    ],
    "similaridade": 0.95
},
{
    "nome": "w3",
    "posicao": [
        85,
        111
    ],
    "tamanho": [
        67,
        65
    ],
    "similaridade": 0.95
},
{
    "nome": "w4",
    "posicao": [
        2,
        108
    ],
    "tamanho": [
        63,
        66
    ],
    "similaridade": 0.95
}
]
}
}
},
{
    "name": "54",
    "x": 35404.0,
    "y": 6300.0,
    "z": 1000.0,
    "meta": {
        "action": "dot",
        "dot_id": 2,
        "dot_name": "processador",
        "dot_qty": 100
    }
},
{

```

```
"name": "55",
"x": 34429.0,
"y": 5887.0,
"z": 1000.0,
"meta": {
  "action": "dot",
  "dot_id": 2,
  "dot_name": "processador",
  "dot_qty": 100
}
},
{
  "name": "56",
  "x": 34390.0,
  "y": 4370.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 2,
    "dot_name": "processador",
    "dot_qty": 100
  }
},
{
  "name": "57",
  "x": 35261.0,
  "y": 3945.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 2,
    "dot_name": "processador",
    "dot_qty": 100
  }
},
{
  "name": "58",
  "x": 37262.0,
  "y": 3927.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 2,
    "dot_name": "processador",
    "dot_qty": 100
  }
},
{
  "name": "59",
  "x": 38337.0,
  "y": 4374.0,
```

```
"z": 1000.0,
"meta": {
  "action": "dot",
  "dot_id": 2,
  "dot_name": "processador",
  "dot_qty": 100
}
},
{
  "name": "60",
  "x": 38349.0,
  "y": 5742.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 2,
    "dot_name": "processador",
    "dot_qty": 100
  }
},
{
  "name": "61",
  "x": 37393.0,
  "y": 6282.0,
  "z": 1000.0,
  "meta": {
    "action": "dot",
    "dot_id": 2,
    "dot_name": "processador",
    "dot_qty": 100
  }
},
{
  "name": "62",
  "x": 36261.0,
  "y": 5132.0,
  "z": 0.0,
  "meta": {
    "action": "inspect",
    "componentes": {
      "UZ12": {
        "posicao": [
          104,
          0
        ],
        "dimensoes": [
          419,
          460
        ],
        "inspecoes": [
          {
```

```
"nome": "w1",
"posicao": [
  8,
  3
],
"tamanho": [
  134,
  112
],
"similaridade": 0.95
},
{
  "nome": "w2",
  "posicao": [
    275,
    3
  ],
  "tamanho": [
    139,
    116
  ],
  "similaridade": 0.95
},
{
  "nome": "w3",
  "posicao": [
    284,
    341
  ],
  "tamanho": [
    128,
    116
  ],
  "similaridade": 0.95
},
{
  "nome": "w4",
  "posicao": [
    5,
    341
  ],
  "tamanho": [
    128,
    114
  ],
  "similaridade": 0.95
}
]
}
}
```

}
]